Guía completa de Galaga 3D

Funcionamiento, estructura, decisiones y código fuente explicado

Versión técnica del 1 de noviembre de 2026

Este documento explica cómo funciona la demo 3D integrada en EntornVGI. Está pensado para una persona que empieza con C++, OpenGL o programación de videojuegos. Cada concepto se conecta con los archivos y funciones que lo utilizan.

La guía cubre el código propio del juego, su integración con la plantilla docente, los recursos, la compilación y las comprobaciones. El anexo reproduce los módulos propios completos y fragmentos identificados de la integración. La plantilla general y las dependencias se describen por su función dentro del proyecto.

La versión documentada tiene una ronda con 18 enemigos, tres vidas y movimiento lateral. El GDD contiene ampliaciones que se encuentran en fases distintas de desarrollo. Las secciones de estado y limitaciones explican esa relación.

El código se ha desarrollado con asistencia de Codex. La guía docente establece restricciones para el uso académico de IA en programación. La revisión técnica y las pruebas deben acompañarse de comprensión personal y de la validación que corresponda con el profesorado.



Captura real de la demo con un objetivo seleccionado.

Índice de lectura

Índice de lectura 2
1 Cómo leer esta guía y qué versión describe 4
2 Qué hace el jugador durante una partida 5
3 Estructura general y responsabilidades 6
4 Carpetas y archivos que aparecen en el proyecto 7
5 Ideas de C++ necesarias para seguir el código 8
6 Desde abrir el EXE hasta ver el menú 9
7 Teclas y cambios de estado de la interfaz 10
8 El reloj y el paso fijo de simulación 11
9 Datos de la partida y significado de cada grupo 12
10 Crear y reiniciar la formación 13
11 Movimiento del jugador y límites del campo 14
12 Selección de objetivo y asistencia de apuntado 15
13 Cómo se crea y se mueve una bala aliada 16
14 La IA básica y los disparos enemigos 17
15 Detectar una colisión entre dos pasos 18
16 Resolver impactos sin duplicar daño ni puntos 19
17 Vidas, protección, puntos y resultado 20
18 Cómo se construye una imagen completa 21
19 Cámara en tercera persona y sistemas de coordenadas 22
20 Matrices y normales explicadas con un ejemplo 23
21 Cómo se fabrican las mallas de las naves 24
22 VBO y VAO como almacenamiento y descripción de datos 25
23 La biblioteca de modelos y sus nueve cargas 26
24 Vertex shader y transformación de cada vértice 27
25 Fragment shader y modelo de iluminación Phong 28
26 El HUD y las texturas del proyecto 29
27 Proyectar la marca de objetivo y comprobar el encuadre 30
28 Gestión de recursos y cierre de la aplicación 31
29 Pruebas de reglas y comprobación gráfica 32
30 Compilación y contenido de la carpeta ejecutable 33
31 Mesa y las dependencias explicadas 34
32 Decisiones de diseño y evolución del código 35
33 Cómo modificar el juego con criterio 36
34 Diagnóstico de problemas y límites actuales 37
35 Correspondencia con la asignatura VGI 38
36 Glosario para leer el código 39
Referencia de funciones del juego 40

Mapa de parámetros y valores actuales 46
Anexo de código y trazabilidad 47
Código Game3D.h 47
Código Game3D.cpp 48
Código GameView3D.cpp 51
Código Mesh3D.h 52
Código Mesh3D.cpp 53
Código Renderer3D.h 58
Código Renderer3D.cpp 58
Código Renderer2D.h 62
Código Renderer2D.cpp 62
Código galaga3d.vert 65
Código galaga3d.frag 65
Código DemoVerification.cpp 66
Código Game3DTests.cpp 68
Código build-demo.cmd 70
Código build-demo.ps1 70
Código setup-mesa.ps1 71
Integración con la plantilla MFC 72
Configuración de los proyectos 76
Identificación de los archivos documentados 82

1 Cómo leer esta guía y qué versión describe

La aplicación se puede entender como una cadena de trabajo. Windows entrega las teclas. La simulación decide dónde están las naves y qué impactos ocurren. El renderizador convierte ese estado en una imagen. El HUD añade los puntos, las vidas y las instrucciones. La compilación reúne todos esos archivos en un ejecutable.

Para una primera lectura conviene seguir los capítulos en orden hasta las colisiones. Después pueden estudiarse las mallas, la cámara y los shaders. La referencia de funciones sirve para resolver una duda concreta. El anexo permite comparar la explicación con el código exacto y sus números de línea.

Qué está implementado

La demo presenta una selección entre Flecha, Delta y Doble Motor. Las tres opciones utilizan las mismas reglas. La ronda contiene dieciocho enemigos estáticos en tres filas con distintas posiciones tridimensionales. El jugador se desplaza lateralmente, dispara, recibe daño y puede terminar en victoria o derrota. La pausa y la vuelta al menú forman parte del ciclo completo.

El objetivo marcado se obtiene de la posición lateral del jugador. Si se dispara con un objetivo seleccionado, la velocidad se calcula para pasar por su centro. El proyectil continúa por esa dirección aunque después el jugador se mueva. Los enemigos disparan hacia la posición que tenía la nave cuando salió su bala.

Cómo distinguir diseño y estado real

El GDD recoge propuestas más amplias, como varias oleadas, jefe, armas adicionales, cajas de skins y un modo de esquiva. Este manual identifica las funciones activas y las estructuras que han quedado preparadas o conservadas. Por ejemplo, existe una malla de jefe en la biblioteca gráfica, pero la ronda actual instancia solamente dos tipos de enemigo.

El código exacto de una versión es la referencia final para sus valores. Por eso el anexo incluye hashes de los archivos. Las cifras de pruebas y capturas se refieren a esta revisión. El comportamiento de una futura versión puede cambiar y requerirá actualizar su explicación.

2 Qué hace el jugador durante una partida

El menú permite elegir nave con 1, 2 o 3. Enter crea una partida limpia. A y D, o las flechas, desplazan la nave. Espacio mantiene el disparo mientras se respete la recarga. P alterna juego y pausa. Esc pausa durante el combate y permite volver al menú desde pausa o resultado. R vuelve a empezar después de victoria o derrota.

El movimiento tiene una única entrada espacial, la posición X. El mundo también tiene Y y Z. La formación se encuentra hacia delante en Y y sus filas están a distintas alturas Z. Esta combinación da distancia y desnivel reales. El motor representa los tres componentes al dibujar y al resolver impactos.

La ayuda de apuntado compensa esa combinación de controles sencillos y escena tridimensional. Dentro del margen lateral se escoge el enemigo vivo situado más cerca hacia delante. La marca amarilla lo identifica. Al desaparecer ese enemigo, el siguiente de la columna puede convertirse en objetivo. Los tiros ya emitidos conservan su rumbo.

El jugador empieza con tres vidas. Un impacto válido provoca una espera de reaparición y un periodo de protección. La nave vuelve al centro. El HUD muestra el estado de protección con ESCUDO. Los puntos suben al destruir enemigos. La racha aumenta el multiplicador y puede romperse por daño o por un disparo aliado fallado que abandone el campo.

La victoria exige que no queden enemigos vivos y que el jugador tenga vidas. Si ambas condiciones finales coinciden con la pérdida de la última vida, la simulación elige derrota. La decisión está al final de update, después de resolver todos los contactos de ese paso.

El fondo negro, la posición de la cámara y los tamaños de malla ayudan a leer la formación. Son valores de presentación ajustados durante el desarrollo. Los controles, la cadencia y los radios de contacto pertenecen a las reglas del juego. Conviene revisar las dos partes juntas cuando se cambia el tamaño de una nave.

3 Estructura general y responsabilidades

La solución Galaga3D.sln organiza la compilación. El proyecto EntornVGI.vcxproj conserva la base MFC del curso y añade los módulos de la demo. El proyecto Game3DTests.vcxproj compila pruebas de la simulación. GLEW y SOIL2 se construyen como dependencias. Los shaders y recursos se copian junto al ejecutable.

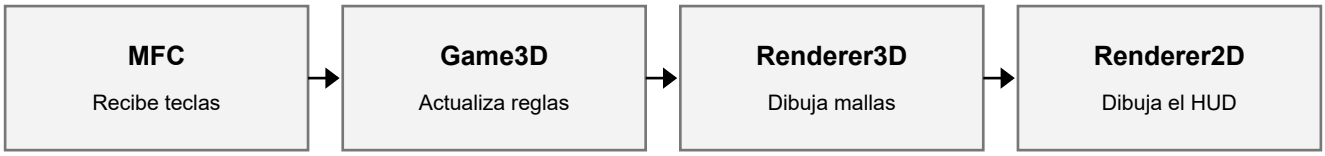
Game3D.h declara los datos y las operaciones del juego. Game3D.cpp implementa sus reglas. Esta clase puede ejecutarse sin abrir una ventana porque sus cálculos no dependen de MFC, OpenGL ni del reloj del sistema. Esto permite repetir partidas de prueba con entradas conocidas.

GameView3D.cpp actúa como conexión con Windows. Conserva el estado de las teclas, transforma los eventos en un Input y decide cuántos pasos de simulación ejecutar según el tiempo transcurrido. También se ocupa de pausar al perder el foco y de limpiar entradas al cambiar de estado.

Mesh3D.cpp construye la geometría y guarda sus buffers. Renderer3D.cpp selecciona cámara, matrices, iluminación y mallas. Renderer2D.cpp dibuja el fondo y los elementos de pantalla. Los shaders galaga3d.vert y galaga3d.frag hacen las transformaciones y los cálculos de color del dibujo 3D.

EntornVGI.cpp crea la aplicación y el marco principal. MainFrm.cpp organiza menús y paneles de la plantilla. EntornVGIView.cpp crea el contexto OpenGL, conecta el repintado, arranca el temporizador y libera recursos al cerrar. EntornVGIDoc y ChildFrm conservan la estructura documental y de ventanas de MFC.

Esta distribución evita que una función de dibujo cambie las vidas o que una tecla tenga que conocer cómo se configura un VBO. Cuando aparece un fallo se puede seguir el recorrido del dato. Una bala que nace mal se investiga en Game3D. Una bala orientada mal se investiga en Renderer3D. Un icono ausente se investiga en Renderer2D y en la copia de recursos.



La simulación produce datos. El renderizado consulta esos datos.

La misma malla puede dibujarse muchas veces con distintas matrices.

4 Carpetas y archivos que aparecen en el proyecto

Entrega_Amin_VGI/entornsGrafics contiene la solución y el entorno del curso. Dentro, EntornVGI reúne los archivos C++, los shaders y recursos de la aplicación. tests contiene el ejecutable de pruebas. include y third_party aportan cabeceras y dependencias. Las carpetas bin, build y lib reciben resultados de compilación.

Recursos_Galaga3D reúne modelos de Blender, scripts de creación, imágenes del HUD, fuentes y audio. En la demo actual, las naves se construyen en Mesh3D.cpp y los PNG del HUD sí se cargan desde esta carpeta. Los recursos existentes permiten seguir trabajando con una estética compartida.

Recursos_Galaga2D contiene sprites, un atlas, descripciones JSON y variantes vectoriales. Estos archivos pertenecen a los prototipos y propuestas del proyecto. El ejecutable 3D de esta guía utiliza la geometría de su biblioteca de modelos y los recursos de interfaz que se indican en el código.

Actes conserva las reuniones del grupo. Gestion_y_diseno reúne documentos de diseño y planificación. REVISION_JUEGO.txt relaciona la revisión funcional con la documentación docente. DEMO_3D.md contiene las instrucciones prácticas para compilar y ejecutar.

Los scripts de la raíz y de scripts preparan el entorno de ejecución. build-demo.cmd abre PowerShell con una política limitada a ese proceso. build-demo.ps1 compila, prueba y crea el paquete. setup-mesa.ps1 comprueba o descarga el motor OpenGL local y verifica sus hashes.

dist/Galaga3D contiene la carpeta lista para ejecutar en este equipo. Incluye el EXE, bibliotecas de ejecución, shaders, imágenes y licencias. .gitignore excluye dist y los resultados de compilación. El repositorio conserva el código y las instrucciones necesarias para reconstruirlos. La carpeta MVP tiene su propia exclusión local, establecida por la decisión del usuario.

Los archivos de la plantilla, como visualitzacio.cpp, escena.cpp, material.cpp, shader.cpp, objLoader.cpp y glut_geometry.cpp, permiten utilizar el entorno docente. Su lógica general permanece disponible mediante el menú Juego. La demo activa dirige el dibujo hacia Renderer3D.

5 Ideas de C++ necesarias para seguir el código

Una variable guarda un valor. int se usa para contadores e identificadores. float representa posiciones y tiempos con decimales. bool representa una condición verdadera o falsa. Un enum class reúne estados con nombre, como Playing o Paused, y hace más legibles las decisiones.

Una struct agrupa datos relacionados. Enemy guarda posición, tipo e información de estado. Bullet guarda posición, dirección y tiempo de vida. Una class reúne datos y funciones. Game3D expone operaciones como reset y update, mientras conserva sus relojes internos en la parte private.

std::vector es una colección cuyo tamaño puede variar. Las balas se añaden con push_back y se retiran al final del paso. Es importante evitar borrar posiciones mientras todavía se usan sus índices. El proyecto reúne los contactos, los resuelve y después elimina las balas consumidas.

La palabra const expresa que una operación debe consultar un dato sin modificarlo. Un parámetro como const Input& permite leer la entrada sin copiarla. El signo & expresa una referencia al objeto original. Un puntero como const Enemy* puede señalar un enemigo o valer nullptr cuando no existe un objetivo.

El puntero devuelto por aimTarget se utiliza inmediatamente. Añadir elementos a un vector puede mover su memoria y dejar obsoletos los punteros anteriores. Por eso una marca de objetivo se calcula cada vez que hace falta, en lugar de conservar un puntero durante toda la partida.

Las funciones pequeñas sin nombre se llaman lambdas. El código las utiliza para contar enemigos vivos, ordenar contactos o eliminar proyectiles. Por ejemplo, una condición que devuelve !b.alive selecciona las balas marcadas como consumidas.

std::clamp limita un valor entre dos extremos. std::max impide que un temporizador quede negativo. std::sqrt calcula una raíz cuadrada. try y catch permiten capturar fallos de recursos gráficos y terminar la inicialización de manera controlada. Los identificadores GLuint representan objetos administrados por OpenGL y deben liberarse con su contexto activo.

6 Desde abrir el EXE hasta ver el menú

MFC organiza el arranque mediante CEntornVGIApp::InitInstance. La función sitúa el directorio de trabajo junto al ejecutable para que los archivos de apoyo se encuentren desde una ubicación estable. El registro startup.log guarda las etapas iniciales y ayuda a localizar un fallo anterior al dibujo.

Se inicializan controles de Windows, MFC y OLE. Se configuran administradores de menús, teclado y ayudas. La aplicación crea una plantilla documental que relaciona CEntornVGIDoc, CChildFrame y CEntornVGIView. Después crea CMainFrame y carga sus recursos de ventana.

Cuando se crea la vista se ejecuta OnCreate. Esta función obtiene un contexto de dispositivo de Windows, elige un formato de píxel con doble buffer y profundidad y prepara OpenGL. Un contexto es la conexión que mantiene el estado de OpenGL para esa superficie de dibujo.

El código comienza con un contexto temporal para obtener las extensiones necesarias. Después intenta crear un contexto Core 4.6 y, como alternativa técnica, uno 3.3. GLEW proporciona el acceso a las funciones modernas. Renderer3D::initialize carga programas, mallas y HUD. El récord se recupera del perfil de la aplicación y se instala el temporizador 45003.

En un arranque habitual se activa el diseño de demo, se muestra el marco, se maximiza la ventana documental y se da el foco a la vista. En un arranque con /verify-render se ejecuta la comprobación automática y se cierra con un código de resultado.

MainFrm::SetDemoLayout oculta o muestra los paneles docentes según el modo. La aplicación conserva así la estructura del entorno y puede volver a sus herramientas. Las trazas de frame.log y context.log completan startup.log para distinguir creación de ventana, creación de OpenGL y carga del renderizador.

Se ha observado un fallo intermitente al crear el marco MFC. Las últimas aperturas verificadas funcionaron, pero su causa raíz sigue abierta. Este punto se conserva explícitamente en el informe de revisión.

7 Teclas y cambios de estado de la interfaz

PreTranslateMessage intercepta las teclas de la demo antes de que otros elementos MFC las consuman como aceleradores o navegación. OnKeyDown termina llamando a GameKeyDown y OnKeyUp limpia la posición correspondiente del arreglo gameKeys. Cada entrada del arreglo indica si una tecla permanece pulsada.

El juego distingue acciones sostenidas y acciones de una sola pulsación. Mover y disparar se consultan en cada paso. Pausar, seleccionar o reiniciar requieren una pulsación nueva. La variable fresh combina el estado anterior de la tecla con la marca de repetición que entrega Windows.

GameKeyDown aplica las transiciones permitidas. P llama a pause. Las teclas 1-3 llaman a selectShip, que acepta cambios en Menu. Enter inicia desde el menú. R reinicia desde Victory o Defeat. Esc pausa durante Playing y vuelve al menú desde otros estados de partida.

ClearGameInput pone todas las teclas a false, vacía el acumulador y vuelve a tomar el reloj actual. De esta manera, una tecla retenida antes de cambiar de pantalla no queda guardada como una acción futura. También evita acumular el tiempo real que transcurrió mientras se estuvo pausado.

OnKillFocus pausa una partida activa y limpia la entrada. GameTick comprueba además que la aplicación y la vista sigan siendo las activas. Son dos puntos de protección para situaciones como cambiar de ventana o seleccionar otro documento MDI.

El menú Juego conecta OnGameDemo y OnGameEnvironment. Al entrar en el entorno docente se revisa la disponibilidad de OpenGL 4.6, se pausa el combate y se muestran sus paneles. La vista conserva sus objetos de juego y de renderizado. Volver a la demo recupera el diseño de pantalla y solicita un nuevo dibujo.

La entrada representa la intención del jugador. Las consecuencias físicas se aplican en Game3D::update. Esta separación permite que las pruebas construyan un Input directamente sin enviar teclas reales al sistema operativo.

8 El reloj y el paso fijo de simulación

Una ventana puede repintarse con intervalos distintos. Si cada repintado avanzara la nave una cantidad fija, el juego iría más rápido en un equipo que dibuja más imágenes. GameTick mide el tiempo transcurrido con steady_clock y lo guarda en gameAccumulator.

El paso lógico vale 1/60 de segundo. Mientras el acumulador contenga al menos esa cantidad se ejecuta update y se resta un paso. Un intervalo real de 0,05 segundos permite aproximadamente tres actualizaciones de 1/60. El render puede representar el estado resultante al terminar esas actualizaciones.

El tiempo de un evento se limita a 0,1 segundos y se permiten como máximo cinco pasos por llamada. Si todavía queda un exceso, se conserva solamente la fracción inferior a un paso. Esto evita intentar recuperar cientos de movimientos después de una interrupción larga. Durante una sobrecarga muy grande puede perderse tiempo simulado, que es una limitación deliberada de esta versión sencilla.

La simulación abandona la actualización si la fase no es Playing o si dt es cero o negativo. En pausa se vacía el acumulador. Al reanudar se toma una referencia de reloj nueva. Una espera de diez segundos en pausa no se transforma en diez segundos de disparos al continuar.

Los temporizadores se reducen con dt y se limitan a cero. fireClock arranca en 0,22 segundos después de disparar. Con pasos discretos, trece pasos son unos 0,2167 segundos y todavía falta una fracción. El siguiente disparo suele admitirse al paso catorce, alrededor de 0,2333 segundos. El test de cadencia comprueba nueve emisiones en dos segundos cuando las otras restricciones lo permiten.

Esta arquitectura usa una simulación secuencial en la vista MFC. Su coste es pequeño para dieciocho enemigos. Las reglas se benefician de un orden reproducible, útil para analizar una colisión o repetir una partida con las mismas entradas.

9 Datos de la partida y significado de cada grupo

Phase describe Menu, Playing, Paused, Victory y Defeat. Cada valor gobierna qué entradas y actualizaciones tienen efecto. Input contiene left, right y fire. Vec2 reúne X e Y. Aunque se use esa estructura, enemigos y balas añaden sus componentes Z, de modo que su estado completo es tridimensional.

Game3D guarda player y previousPlayer. La posición anterior permite medir un recorrido entre dos instantes. lives, score, shots y kills registran vidas, puntos, disparos emitidos y bajas. selectedShip identifica el modelo elegido. bestScore conserva el máximo conocido durante la sesión y puede guardarse al cerrar normalmente.

hitStreak cuenta bajas de la racha y multiplier expresa su multiplicador. time es el tiempo lógico acumulado. invulnerable y respawn controlan protección y espera. fireClock limita al jugador y attackClock controla cuándo vuelve a disparar un enemigo. nextId produce identificadores para nuevas balas. nextAttacker recuerda la posición de búsqueda de la IA.

Cada Enemy tiene id, type y alive. pos, previous y z/previousZ se usan en dibujo y colisiones. slot conserva el lugar de formación. La versión actual asigna Formation al iniciar. También conserva nombres y campos de variantes anteriores, como Entering, Warning, Diving, Returning, start, shot, timer y targetX. El ciclo actual mantiene la formación estática.

Bullet conserva id, ally y alive para distinguir el proyectil, su equipo y su estado. pos/previous y z/previousZ permiten barrer el recorrido. velocityX, velocityY y velocityZ representan su movimiento. ttl guarda el tiempo de vida restante. Cuando velocityY vale cero, forwardSpeed utiliza la velocidad base asociada a su equipo. Esta convención admite los proyectiles simples de los casos de prueba y el disparo sin objetivo.

Spark es una estructura conservada. update vacía su colección y Renderer3D no dibuja explosiones de esa colección en esta revisión. Al leer una cabecera conviene comprobar dónde se usa cada dato, porque una declaración puede seguir existiendo después de simplificar una mecánica.

10 Crear y reiniciar la formación

reset conserva la nave seleccionada y el récord, y después asigna un Game3D nuevo sobre el objeto actual. Eso recupera los valores iniciales de contadores, colecciones y temporizadores. A continuación restaura la selección y el récord y pone la fase en Playing.

El bucle crea dieciocho enemigos. i%6 calcula la columna y i/6, usando división entera, calcula la fila. La expresión de X es (columna - 2,5) por 2,6. Así se obtienen -6,5, -3,9, -1,3, 1,3, 3,9 y 6,5. Las seis columnas quedan centradas alrededor de cero.

La posición hacia delante es Y = 14 + fila por 4. Las filas se sitúan en 14, 18 y 22. La altura es Z = (fila - 1) por 2,5. Sus valores son -2,5, 0 y 2,5. Las coordenadas anteriores se igualan a las actuales al crear cada enemigo. Esto evita que el primer barrido interprete un desplazamiento artificial desde el origen.

Los doce primeros enemigos tienen type=0 y los seis restantes type=1. Renderer3D utiliza ese dato para elegir entre las dos mallas activas. Todos empiezan vivos y en formación. La malla tercera de la biblioteca queda reservada y también se utiliza en la galería de comprobación.

returnToMenu aplica una reinicialización parecida y termina con la fase inicial Menu. selectShip limita la selección a índices de cero a dos y requiere estar en el menú. pause cambia exclusivamente entre Playing y Paused. remaining cuenta las marcas alive reales, por lo que la victoria depende del estado de los enemigos y no de borrar sus posiciones del vector.

El reinicio elimina balas y temporizadores de la sesión anterior. Las pruebas repiten esta secuencia varias veces y comprueban que vuelven las tres vidas, los dieciocho enemigos y el tiempo cero.

11 Movimiento del jugador y límites del campo

Al empezar cada paso se copia player en previousPlayer. Si hay espera de reaparición, el juego reduce ese tiempo. Cuando llega a cero, ambas posiciones se sitúan en el centro con Y=-4. Al igualar las dos posiciones se evita representar el regreso al centro como un movimiento físico que pudiera chocar durante su recorrido.

Cuando la nave está disponible, se reduce invulnerable y se calcula el movimiento. La expresión int(right)-int(left) da 1 al ir a la derecha, -1 al ir a la izquierda y 0 al soltar ambas o pulsarlas a la vez. Se multiplica por 6 y por dt. En un paso de 1/60, el desplazamiento es 0,1 unidades.

clamp mantiene X entre -7,68 y 7,68. Esos límites se han relacionado con la cámara y se comprueban mediante capturas de ambos extremos. Un límite del mundo y un borde de pantalla son conceptos conectados por la proyección. Si se modifica la cámara es necesario volver a comprobar que la nave siga completamente visible.

El modelo elegido cambia la geometría mostrada. La simulación conserva las mismas velocidades, límites y vidas para las tres naves. El radio defensivo de contacto con balas hostiles es un valor efectivo de 0,42. Las colisiones utilizan una aproximación sencilla al volumen vulnerable y no recorren cada triángulo del modelo.

Mientras dura respawn se suspende la entrada de movimiento y disparo de la nave. El mundo continúa actualizando enemigos y proyectiles. Durante invulnerable se admite el control y los contactos hostiles consumen la bala, pero damage descarta una nueva pérdida de vida.

Estos detalles explican situaciones observables. Dos impactos seguidos pueden dejar una sola vida menos. Una bala ya disparada conserva su trayectoria aunque el jugador cambie de dirección. La tecla de movimiento afecta a la posición del jugador y no arrastra los proyectiles existentes.

12 Selección de objetivo y asistencia de apuntado

aimTarget es el único criterio compartido por la marca visual y el disparo dirigido. Recorre los enemigos y descarta los muertos y los que no están delante de la salida de la bala. Después exige que la distancia lateral absoluta sea como máximo AimTolerance, cuyo valor es 0,65.

Entre los candidatos se elige el de menor Y. Esta regla hace que el enemigo delantero de una columna se seleccione antes que el situado detrás. La marca cambia cuando desaparece el objetivo o cuando el jugador se desplaza a otra columna. La búsqueda devuelve un puntero temporal o nullptr.

La asistencia es una decisión de jugabilidad para combinar movimiento exclusivamente lateral con blancos a distintas alturas. El jugador necesita alinearse horizontalmente. El código completa la dirección tridimensional hacia el centro del blanco seleccionado. El margen no se amplía hasta cubrir toda la pantalla.

La primera implementación aceptaba hasta 0,55 unidades laterales, pero mantenía velocidad X igual a cero y usaba un radio efectivo menor para el impacto. Por eso podía marcar conceptualmente un objetivo y pasar a su lado. La versión actual calcula las tres componentes de velocidad y utiliza la misma selección para el HUD.

Renderer3D consulta aimTarget para dibujar la marca. Si el enemigo ya ha muerto, la búsqueda visual refleja el nuevo estado. Una bala que salió antes de esa muerte continúa por su vector original. No se conserva el puntero al enemigo dentro de Bullet, lo que también evita depender de la duración de memoria de ese objeto.

Fuera del margen el jugador puede disparar hacia delante. Una coincidencia visual entre objetos a distinta distancia no garantiza contacto. La marca comunica el destino asistido y las esferas del mundo resuelven los impactos reales. Las pruebas incluyen tanto aciertos dentro del margen como un fallo auténtico fuera de él.

13 Cómo se crea y se mueve una bala aliada

El jugador dispara cuando fire es verdadero, fireClock ha llegado a cero y existen menos de ocho balas aliadas activas. La nueva posición nace en X del jugador, Y del jugador más 0,85 y Z=0. El desplazamiento hacia delante coloca la bala por delante del morro de los tres modelos.

Si hay objetivo, se calculan dx, dy y dz desde la salida hasta su centro. La longitud de ese vector es la raíz cuadrada de $dx^2+dy^2+dz^2$. Dividir esa longitud entre PlayerShotSpeed produce el tiempo de viaje ideal. Cada componente de velocidad se obtiene dividiendo su desplazamiento por ese tiempo.

Por ejemplo, con jugador X=0,5 y un enemigo en (0,14,-2,5), la bala nace en (0,5,-3,15,0). El vector hacia el enemigo es (-0,5,17,15,-2,5). La distancia es aproximadamente 17,338 unidades. A 24 unidades por segundo el tiempo es aproximadamente 0,722 segundos. La velocidad es aproximadamente (-0,692,23,742,-3,461). Sus tres componentes combinadas tienen rapidez 24.

En cada paso se guardan las posiciones anteriores y se avanza posición += velocidad por tiempo activo. El tiempo activo se limita al ttl restante. Una bala que solo conserva 0,005 segundos de vida avanza durante esa fracción aunque el paso completo sea mayor. Después se descuenta dt del ttl y se conserva el recorrido para los contactos.

Si no hay objetivo, velocityY permanece en cero y forwardSpeed devuelve la velocidad base positiva. Las otras componentes permanecen en cero. La bala viaja recta hacia delante. Los proyectiles creados con esa forma sencilla siguen siendo válidos para la simulación y para las pruebas.

Al emitir se pone fireClock a 0,22 y aumenta shots. El identificador nextId se incrementa para distinguir la nueva bala. El límite de ocho evita una colección ilimitada si el jugador mantiene Espacio. La retirada por delante de Y=28 elimina tiros que ya han atravesado la zona de los enemigos.

14 La IA básica y los disparos enemigos

Los enemigos de esta revisión permanecen en su lugar. La IA se limita a decidir cuándo y desde dónde emitir una bala. attackClock empieza en 2,4 segundos. Cuando se agota, el código busca un enemigo vivo a partir de nextAttacker y recorre circularmente el vector si hace falta.

Después de encontrar un emisor se calcula el desplazamiento desde ese enemigo hasta la posición actual del jugador. Se obtiene el tiempo ideal de viaje dividiendo la distancia tridimensional entre EnemyShotSpeed, que vale 7. La velocidad X, Y y Z resulta de dividir cada desplazamiento por ese tiempo.

El disparo registra las coordenadas reales del emisor como posición actual y anterior. Así, el primer barrido comienza en el lugar del enemigo. ttl recibe el tiempo de viaje más un segundo. El temporizador de la siguiente emisión pasa a 1,2 segundos y nextAttacker avanza al índice siguiente.

La puntería se fija una vez al emitir. Después la IA puede disparar otra bala, pero la bala anterior conserva su vector. Esto permite esquivarla mediante desplazamiento lateral. El emisor utiliza información presente en el momento de su decisión y una cadencia limitada.

La rapidez total se normaliza en XYZ. Si solo se fijara una velocidad Y y se añadiesen componentes laterales, una bala diagonal sería más rápida que una frontal. La normalización mantiene la misma rapidez general para posiciones de disparo distintas.

Cuando un tiro pasa por detrás de Y=-4,8 se retira. Esa frontera está un poco detrás de la nave. Evita que un proyectil fallado siga viajando hacia la cámara y aparezca desproporcionadamente grande en primer plano. La prueba de retirada cubre este comportamiento.

Esta IA es un director reactivo de disparos. La formación estática, los tiempos y las trayectorias se eligieron para la demo sencilla. Los picados y otras estrategias del proyecto completo requerirán sus estados y comprobaciones adicionales.

15 Detectar una colisión entre dos pasos

Comprobar únicamente la posición final de una bala puede perder un impacto. Si en un paso aparece delante del enemigo y en el siguiente ya está detrás, las dos posiciones pueden estar fuera de su volumen aunque el recorrido lo atraviese. hitTime3D calcula el primer instante de contacto dentro de ese recorrido.

La función recibe dos posiciones anteriores, dos posiciones actuales y un radio efectivo r. Primero expresa la posición relativa inicial p entre los dos objetos. Después calcula el movimiento relativo v, restando sus desplazamientos. De esta manera también se puede detectar un choque si ambos objetos se mueven.

El punto relativo durante el paso es p + t·v, con t entre cero y uno. Hay contacto cuando su distancia al origen alcanza r. Al elevar al cuadrado se obtiene $a·t^2 + b·t + c = 0$. El código calcula a=v·v, b=2·p·v y c=p·p-r^2. El punto indica producto escalar, es decir, la suma de los productos de sus componentes.

Si c es menor o igual que cero, los volúmenes ya están en contacto al inicio y se devuelve cero. Si a es casi cero y estaban separados, no hay movimiento relativo útil y se devuelve -1. Si el discriminante b^2-4ac es negativo, la trayectoria no corta la esfera. En los demás casos se toma la raíz menor y se admite solo si está entre cero y uno.

Un ejemplo sencillo coloca una bala en Y=-2 y después en Y=2, con enemigo en el origen y r=0,76. El contacto inicial ocurre cuando la bala alcanza Y=-0,76. Ha recorrido 1,24 de las 4 unidades del tramo, por lo que t=0,31. Ese valor permite ordenar este impacto respecto a otros del mismo paso.

EnemyHitRadius=0,76 se usa como radio combinado efectivo para bala aliada y enemigo. Los otros contactos usan sus propios valores. La aproximación es una envolvente de juego. El curso presenta esferas como volumen simple y la demo añade el barrido temporal para proteger los disparos rápidos.

16 Resolver impactos sin duplicar daño ni puntos

Después de mover las balas, update construye una lista de Contact. Cada elemento contiene el tiempo t, el índice de bala, el índice de enemigo y un identificador para desempatar. El valor de enemigo -1 representa un contacto con el jugador.

Las balas aliadas se comparan con los enemigos vivos. Las hostiles se comparan con el jugador cuando no está esperando reaparecer. hitTime3D utiliza las posiciones reales XYZ. La proyección de cámara no interviene en esta decisión.

La lista se ordena por tiempo de contacto. Si dos contactos tienen el mismo tiempo, se usa el identificador y después el índice de bala. Esto establece un orden reproducible para casos simultáneos. Los vectores principales siguen intactos mientras se recorren esos índices.

Antes de aplicar un contacto se comprueba que la bala siga viva. Si otro contacto ya la consumió, se ignora. Si el enemigo ya ha muerto por otro impacto anterior, se ignora ese objetivo y se permite que la bala conserve su estado para otros contactos. Cuando el golpe mata a un enemigo, se consume la bala y se marca alive=false en el enemigo.

Para el jugador, se consume la bala y se llama a damage. Esa función comprueba protección, reaparición y vidas. Si ya se aplicó daño, la protección impide que varios impactos simultáneos resten varias vidas. Una bala que toque al jugador protegido también se consume.

Una segunda comprobación conserva el caso de contacto directo entre nave enemiga y jugador. En la formación estática habitual sus posiciones están separadas. El caso sirve como defensa lógica y como base para una posible mecánica futura. Se retira el enemigo de contacto y se aplica damage sin sumar puntos por ese contacto.

Finalmente se eliminan las balas muertas, caducadas o fuera del campo. Una bala con ttl ya agotado se marca como muerta antes de moverse. Su presencia residual en una colección de contactos no puede causar daño porque la resolución vuelve a consultar alive.

17 Vidas, protección, puntos y resultado

damage termina inmediatamente si hay invulnerabilidad, reaparición o cero vidas. En caso válido resta una vida, reinicia el combo, establece respawn=0,75 e invulnerable=1,5. Durante la espera la nave no se dibuja. Al terminar reaparece en el centro y empieza a consumir su periodo de protección mientras se juega.

La baja de un enemigo aumenta kills y hitStreak. El multiplicador se calcula como el mínimo entre 3 y 1 + hitStreak/3, con división entera. Las dos primeras bajas de una racha valen ×1. La tercera, cuarta y quinta valen ×2. Desde la sexta se usa ×3.

El tipo 0 da 100 puntos base y el tipo 1 da 150. El código conserva una bonificación para EnemyPhase::Diving, pero la formación estática actual utiliza Formation. Por eso esa rama no se activa en las partidas normales de esta versión.

Con siete enemigos de tipo 0 eliminados en una racha, la secuencia de puntos es 100, 100, 200, 200, 200, 300 y 300. El total es 1.400. El récord se actualiza con el máximo entre su valor anterior y score. Al reiniciar, el récord se conserva y score vuelve a cero.

El combo también se rompe cuando una bala aliada viva agota su ttl o sale por delante. Esto significa que una bala antigua fallada puede reiniciar la racha aunque entre tanto haya ocurrido otra baja. Es el comportamiento concreto de esta política de precisión. Su implementación se puede localizar al final de update.

La derrota tiene prioridad cuando lives llega a cero. Si quedan vidas y remaining devuelve cero, se elige Victory. Cualquier resultado final vacía las balas. Las llamadas posteriores a update dejan la simulación detenida porque la fase ya no es Playing.

El récord se lee mediante GetProfileInt durante la creación de la vista. Se escribe al cerrar normalmente, salvo en /verify-render, para que las pruebas gráficas no alteren los datos del jugador. La persistencia pertenece a la integración MFC, mientras que el máximo en memoria pertenece a Game3D.

18 Cómo se construye una imagen completa

OnPaint obtiene un contexto de dibujo de Windows. Si la vista está en modo demo y el renderizador está preparado, activa el contexto OpenGL, llama a gameRenderer.draw, intercambia los buffers y desactiva la asociación actual del contexto. El intercambio presenta la imagen que se terminó de dibujar.

Renderer3D::draw recibe el estado del juego como referencia constante. Primero descarta ventanas de tamaño cero o un programa gráfico ausente. Después dibuja el fondo con Renderer2D, prepara cámara y proyección y activa los estados necesarios para geometría 3D.

begin3D activa profundidad, escritura de profundidad, descarte de caras posteriores y el programa 3D. También envía las matrices comunes y los datos de la luz. En el menú se dibujan tres modelos con las transformaciones de sus tarjetas. En partida se dibujan enemigos vivos, balas y la nave cuando está disponible.

Cada llamada a instance recibe una malla y una matriz model. Envía los datos variables de esa instancia y llama a draw de la malla. La geometría ya reside en su VBO. Dos enemigos del mismo tipo usan la misma malla con posiciones distintas.

Al terminar la geometría se calcula la posición de pantalla del objetivo y se llama a overlay. El HUD desactiva profundidad y dibuja en coordenadas de pantalla. Sus textos permanecen legibles sobre el campo y sus paneles de resultado pueden cubrir la escena.

El orden importa. Dibujar el HUD antes de las naves podría permitir que la geometría lo tapara. Olvidar limpiar profundidad podría dejar valores de un frame anterior. Mantener un estado OpenGL sin conocerlo puede afectar al siguiente dibujo. Por eso begin y begin3D restablecen los estados que sus respectivas pasadas necesitan.

La aplicación utiliza doble buffer. El jugador ve una imagen terminada mientras se prepara la siguiente. La verificación gráfica emplea un framebuffer fuera de pantalla para inspeccionar el mismo pipeline sin depender de una captura del escritorio.

19 Cámara en tercera persona y sistemas de coordenadas

Las mallas se construyen alrededor de su origen local. La matriz model las sitúa en coordenadas del mundo. En esta demo, X recorre el campo de izquierda a derecha, Y apunta hacia delante y Z representa altura. Las filas tienen distintas distancias Y y distintos niveles Z.

La cámara de partida se encuentra en (0,15·player.x,-16,6) y mira hacia (0,15·player.x,8,0). El vector vertical es (0,0,1). El seguimiento de X es parcial. Si la nave se mueve una unidad, cámara y punto de mirada se desplazan 0,15 unidades. Así se percibe seguimiento sin arrastrar toda la formación al mismo ritmo.

glm::lookAt crea la transformación desde mundo a cámara. glm::perspective utiliza 38 grados de apertura vertical, una relación 1280/720, near=0,1 y far=100. near y far limitan el volumen visible. También influyen en cómo se distribuye la precisión del buffer de profundidad.

La perspectiva hace que un modelo lejano ocupe menos píxeles. Alejar enemigos y mantener tamaños muy pequeños produjo una formación difícil de leer. La revisión combinó distancia real, apertura y escala de los modelos. La comprobación de encuadre mide la proyección del volumen aproximado de cada objetivo y los límites de la nave.

El menú usa una cámara distinta. Se sitúa delante del origen y utiliza proyección ortográfica entre -6,4 y 6,4 en horizontal y -3,6 y 3,6 en vertical. Esto facilita colocar las tres tarjetas y los modelos de selección con tamaños comparables.

El lienzo de referencia del HUD es 1280 por 720. Al cambiar el tamaño de ventana se toma la menor escala que permite encajarlo completo y se centra el viewport. Las bandas restantes conservan la composición. El campo jugable usa unidades del mundo, de modo que una ventana más grande no aumenta los límites de movimiento.

20 Matrices y normales explicadas con un ejemplo

Una matriz 4×4 permite combinar posición, rotación y tamaño. Se utiliza un punto homogéneo (x,y,z,1). El último componente permite expresar la traslación con la misma operación matricial que las demás transformaciones.

transform construye translate por rotate por scale. Con vectores columna, la operación situada a la derecha se aplica primero. Un punto se escala en su modelo local, después gira y al final se coloca en el mundo. La documentación GL-4 explica precisamente ese orden inverso de escritura.

Imaginemos un punto local (1,0,0). Una escala por dos lo lleva a (2,0,0). Una rotación de noventa grados alrededor de Z lo lleva aproximadamente a (0,2,0). Una traslación de cinco unidades en X produce (5,2,0). Cambiar el orden puede hacer que la traslación también gire o se escale.

El vertex shader aplica projection * view * model. model lleva del objeto al mundo, view lleva del mundo a la cámara y projection produce coordenadas de recorte. La división por w y el viewport completan la posición de pantalla.

Una normal describe hacia dónde mira una superficie. Para calcular la luz debe transformarse de forma consistente. Renderer3D utiliza transpose(inverse(mat3(model))). La parte 3×3 representa la transformación lineal. La inversa transpuesta conserva la perpendicularidad, especialmente cuando se utilizan escalas diferentes en cada eje.

El shader vuelve a normalizar la normal antes de usarla. Un vector normal unitario tiene longitud uno y permite interpretar su producto escalar con la dirección de luz como un coseno. La luz, el observador y las posiciones se expresan en coordenadas de mundo durante esta evaluación.

Las balas necesitan orientar su eje largo según su velocidad. glm::rotation calcula un cuaternión entre el eje local +Y y la dirección normalizada de la bala. glm::toMat4 lo convierte en una matriz de rotación. Así la orientación dibujada sigue las tres componentes del desplazamiento.

21 Cómo se fabrican las mallas de las naves

Mesh3D.cpp construye mallas mediante triángulos. triangle recibe tres puntos y un color. Calcula el producto vectorial entre dos lados para obtener la normal. Si la longitud es casi cero, los puntos formarían un triángulo degenerado y se descarta. Los tres vértices se guardan con esa normal y color.

quad forma una superficie de cuatro esquinas usando dos triángulos. box construye las seis caras de un bloque a partir de un centro y sus semitamaños. Estos bloques sirven para detalles y para la geometría de comprobación.

hull recibe un contorno convexo en XY y alturas inferior y superior. Calcula el área con signo del contorno y corrige el orden si hace falta. Obtiene un centro promedio, crea un borde superior interior y conecta tapa, bisel, laterales y base. El bisel ayuda a distinguir caras cuando reciben luz.

engine crea un motor de seis lados cuyo eje largo es Y. Cada sector conecta la parte delantera con la trasera y añade las tapas. cockpit crea una pequeña pieza elevada para la cabina. eyes usa bloques pequeños para los ojos de los insectos. Estas funciones reutilizan la misma idea de geometría simple y colores por vértice.

flecha combina alas blancas, cuerpo azul, detalles rojos, motores y cabina. delta utiliza un ala triangular roja con paneles claros. twinEngine tiene dos cuerpos laterales y una unión central. bee y butterfly combinan alas, cuerpo y ojos con siluetas distintas. boss conserva una variante adicional para ampliaciones y pruebas visuales.

projectile construye una pieza estrecha orientada en Y y una franja clara. La versión aliada utiliza cian y la hostil naranja rojizo. El renderizador aplica una transformación a esa geometría para seguir el vector de velocidad.

Los modelos usan normales por cara. El resultado mantiene una apariencia facetada que ayuda a reconocer el volumen con poca geometría. La biblioteca completa realiza nueve cargas estáticas y contiene 1.524 triángulos en la revisión verificada. El número de llamadas de dibujo puede ser mayor porque una malla se instancia varias veces.

22 VBO y VAO como almacenamiento y descripción de datos

Un VBO es un objeto OpenGL que guarda una secuencia de datos de vértices. En esta implementación, cada MeshVertex contiene posición, normal, coordenadas UV y color. Los datos están intercalados, de modo que la información completa de un vértice aparece seguida de la del siguiente.

Un VAO recuerda cómo interpretar esos datos. El atributo 0 lee tres componentes de posición. El 1 lee tres de normal. El 2 lee dos coordenadas UV. El 3 lee cuatro componentes de color. glVertexAttribPointer indica tamaño, tipo, separación entre vértices y desplazamiento del atributo dentro de MeshVertex.

offsetof calcula esos desplazamientos a partir de la estructura real de C++. sizeof(MeshVertex) define la separación entre vértices. La comprobación de tipo standard_layout reduce el riesgo de que la organización de memoria resulte incompatible con esta lectura.

upload comprueba que la colección tenga triángulos completos, que sus posiciones y normales sean finitas y que las normales tengan longitud razonable. Después libera una carga anterior, crea el VAO y VBO y envía la colección con GL_STATIC_DRAW. Esa marca expresa que la geometría se reutilizará sin modificarla continuamente.

draw enlaza el VAO y llama a glDrawArrays con GL_TRIANGLES. El número de elementos guardado es un número de vértices, por lo que el número de triángulos se obtiene dividiendo entre tres. La versión actual expande cada triángulo en sus vértices. El material docente también explica EBO para compartir índices cuando conviene.

verifyGpu consulta los objetos reales. Comprueba el tamaño del buffer, el modo de uso, los cuatro atributos, sus offsets y la asociación al VBO. Después restaura los objetos que estaban enlazados. Este cuidado impide que una comprobación cambie accidentalmente el estado gráfico de otra operación.

release elimina los identificadores y pone los contadores a cero. Las operaciones se realizan con un contexto OpenGL activo. La clase prohíbe copiar Mesh3D para evitar que dos objetos crean ser propietarios del mismo VAO y VBO y los liberen dos veces.

23 La biblioteca de modelos y sus nueve cargas

ModelLibrary reúne tres mallas de naves, tres mallas de enemigos, un proyectil aliado, uno hostil y un cubo auxiliar. initialize llama a las funciones de construcción y a upload para cada una. uploads cuenta las cargas que han terminado correctamente.

Si la biblioteca ya tiene sus nueve cargas, initialize puede terminar sin repetirlas. Si necesita reconstruirse, primero libera su contenido. Un fallo durante el proceso activa la liberación de las mallas ya creadas y deja la biblioteca en un estado recuperable para informar del error.

El cubo auxiliar recibe el nombre particle por su origen en versiones anteriores. En esta revisión sirve para comprobar numéricamente los términos de iluminación. La galería también utiliza las mallas del juego para demostrar que sus matrices y su volumen producen imágenes distintas desde dos orientaciones.

En partida hay dieciocho enemigos, pero cada tipo reutiliza su objeto Mesh3D. La posición, el tamaño y la rotación pertenecen a la matriz de la instancia. Esto evita enviar de nuevo todos los vértices para cada enemigo en cada frame.

verifyGpu revisa todas las mallas, incluso si una de las anteriores ya había fallado. Así se pueden localizar problemas adicionales de atributos o memoria. vertexCount suma los vértices de toda la biblioteca. release recorre las colecciones, libera cada malla y reinicia uploads.

Una ampliación que añada otra malla debe revisar el contador esperado, las pruebas, la selección del modelo y la liberación. Cambiar solamente el tamaño de un arreglo dejaría incompleto el contrato entre biblioteca y verificación. Si se incorporan modelos externos, la carga debe producir la misma descripción de atributos o adaptar también los shaders.

24 Vertex shader y transformación de cada vértice

galaga3d.vert recibe cuatro atributos con las mismas posiciones declaradas en Mesh3D. position representa el punto local de la malla. normal expresa la orientación de la superficie. uv conserva el atributo de textura de la estructura, aunque este shader 3D usa el color del vértice para su material. color contiene RGBA.

El shader recibe model, view, projection y normalMatrix. La primera operación construye world = model * vec4(position,1). El resultado world.xyz pasa al fragment shader, que lo necesita para calcular la dirección hacia el observador y, en el caso general de la expresión, hacia una fuente puntual.

worldNormal se obtiene con normalMatrix por normal. vertexColor transmite el color. El sistema de rasterización interpolará las variables entre los vértices del triángulo antes de ejecutar el fragment shader. En una cara plana cuyos vértices comparten normal, esa normal se mantiene coherente dentro de la cara.

gl_Position recibe projection * view * world. Es una salida especial que OpenGL utiliza para recortar la geometría y encontrar su posición final en pantalla. Los objetos que quedan fuera del volumen de visualización se descartan o recortan durante el pipeline.

El tipo uniform significa que un valor se mantiene común para los vértices de una llamada de dibujo. Por eso se puede enviar una matriz model y dibujar toda una nave. in y out representan entradas y salidas entre etapas. La aplicación no llama al shader una vez por vértice de manera manual. La llamada de dibujo organiza ese trabajo sobre los datos del buffer.

La versión GLSL declarada es 330 core y el contexto verificado admite OpenGL 4.6 Core. Esta combinación utiliza funciones modernas disponibles en ese contexto. La extensión técnica de otras versiones del entorno se conserva en sus propios shaders docentes.

25 Fragment shader y modelo de iluminación Phong

galaga3d.frag calcula el color de cada fragmento. Normaliza worldNormal y obtiene la dirección de la luz y del observador. El color base es el producto del color del vértice y tint, que permite modificar una instancia sin reconstruir su malla.

Light contiene position, diffuse y specular. position es un vec4. En la escena actual se envía w=0 y la dirección (-0,45,0,7,1,1). El shader evalúa light.position.xyz - worldPosition * light.position.w. Con w=0 el vector es común a la escena. La estructura permite expresar la distinción habitual entre dirección y posición.

Material contiene emission, ambient, diffuse, specular y shininess. Son valores enviados por C++. Las naves usan coeficientes reflectantes de 0,37, 0,63 y 0,16. Las balas reciben material emisivo. El exponente especular es 32.

La emisión añade color propio del objeto. El ambiente aporta una iluminación de base común. La difusión Lambert depende del máximo entre cero y el producto escalar de la normal y la dirección de luz. Una superficie orientada hacia la luz recibe más componente difusa que otra que la recibe de lado.

Para el reflejo especular se obtiene reflected = reflect(-lightDirection,n). Después se calcula el producto con viewDirection, se limita a cero y se eleva a shininess. El término se aplica cuando la superficie recibe luz por su cara frontal. Un exponente mayor concentra el brillo en una zona más estrecha.

El color final suma emisión, ambiente, difusión y reflexión especular. clamp mantiene los componentes entre cero y uno. El alfa procede del color por vértice y tint. La pasada de geometría opaca mantiene la mezcla desactivada.

La versión anterior utilizaba un vector intermedio de Blinn y una mezcla simplificada para la emisión. La revisión adoptó la formulación reflect/pow y las propiedades de material del seminario GL-6. Las seis pruebas de iluminación aíslan cada término y comparan píxeles reales con valores esperados.

26 El HUD y las texturas del proyecto

Renderer2D dibuja una interfaz en un lienzo lógico de 1280 por 720. Su vertex shader transforma directamente píxeles de ese lienzo a coordenadas normalizadas. X se divide entre 640 y se desplaza a partir de -1. Y se divide entre 360 y se invierte para que el origen de la interfaz quede arriba a la izquierda.

La estructura Vertex del HUD guarda posición 2D, coordenadas UV y color RGBA. quad crea dos triángulos. La función image prepara un rectángulo y lo asocia a una textura. border crea cuatro tiras. text construye rectángulos para los caracteres de un atlas.

La textura white es un píxel blanco. Al multiplicarlo por el color de cada vértice permite dibujar paneles y líneas de color. El atlas font se genera una vez a partir de filas binarias de glifos de cinco por siete píxeles. La tabla contiene letras, cifras y algunos símbolos. text convierte minúsculas y sustituye caracteres fuera de ASCII por espacios.

Los siete PNG projectHud se cargan mediante SOIL2. Incluyen etiquetas de puntos, récord y vidas, un icono de nave y las instrucciones de mover, disparar y pausar. Sus archivos se buscan junto al EXE dentro de Recursos_Galaga3D/hud.

El HUD conserva una lista de vértices y otra de lotes. flush cierra un lote indicando qué textura utiliza y qué rango de vértices le pertenece. finish envía una vez todos los vértices de la pasada con GL_STREAM_DRAW y después dibuja los lotes en orden. Esta parte es dinámica porque los textos y paneles pueden cambiar.

begin fija viewport, mezcla alfa y el programa de interfaz. glBlendFunc utiliza GL_SRC_ALPHA y GL_ONE_MINUS_SRC_ALPHA, adecuados para los PNG con alfa recto. El fragment shader multiplica el color de la textura por el color del vértice.

overlay presenta el menú o los indicadores de partida. También dibuja los paneles de pausa, victoria y derrota. La marca de objetivo se construye con pequeñas esquinas amarillas. Cada elemento consulta los datos de Game3D, mientras los cambios de puntuación y vidas pertenecen a la simulación.

27 Proyectar la marca de objetivo y comprobar el encuadre

La marca amarilla necesita seguir al enemigo en la imagen. Renderer3D obtiene la posición central del objetivo y la multiplica por projection * view. El resultado clip conserva cuatro componentes. Si w es positivo, se divide X e Y entre w para obtener coordenadas normalizadas.

La conversión al lienzo del HUD es x = (clip.x/clip.w + 1) por 640 y y = (1 - clip.y/clip.w) por 360. Es la misma región de pantalla que usa la interfaz. La coordenada Y se invierte porque el HUD crece desde arriba hacia abajo.

El semitamaño de la marca se aproxima usando 360 por projection[1][1] por EnemyHitRadius dividido entre w, más cinco píxeles de margen. A mayor distancia, w aumenta y la marca se reduce. La marca se calcula únicamente durante juego activo y cuando la nave está disponible.

verifyFraming calcula una proyección equivalente para comprobar la composición. Mide el ancho mínimo aproximado de un enemigo, el ancho de la formación y las posiciones verticales de sus centros. Después proyecta las esquinas de un volumen aproximado del jugador y comprueba que queden dentro del campo útil.

Los criterios actuales exigen al menos 34 píxeles por objetivo y 400 píxeles de formación en el lienzo de referencia. También establecen márgenes frente al HUD y a los bordes. En la revisión medida se obtuvieron aproximadamente 42 píxeles de objetivo mínimo y 487 píxeles de formación.

Estas mediciones detectan regresiones concretas, como empequeñecer accidentalmente las naves al cambiar el campo de visión. La valoración visual se completa inspeccionando capturas al centro, a ambos extremos y en formato 4:3. Las mediciones no describen por sí solas la diversión o la comodidad de una partida manual.

28 Gestión de recursos y cierre de la aplicación

OpenGL mantiene objetos identificados por números. Un programa de shaders, una textura o un buffer deben crearse y eliminarse de forma ordenada. El contexto debe estar asociado al hilo que realiza esas operaciones.

Renderer3D::initialize compila los dos shaders, los enlaza y consulta las posiciones de sus uniform. Después inicializa modelos y HUD. Si ocurre una excepción, libera los objetos que la función conserva y propaga el error para informar del fallo de arranque.

Mesh3D::upload libera una malla anterior antes de asignar sus nuevos objetos. La clase impide la copia para expresar que la propiedad de los identificadores es única. ModelLibrary::release recorre todas sus mallas. Renderer2D::release elimina texturas, VAO, VBO y programa del HUD.

OnDestroy detiene el temporizador. En un cierre normal guarda el récord. Si existe contexto, lo activa, libera Renderer3D y después destruye ese contexto. Finalmente libera el contexto de dispositivo y termina la destrucción de la vista MFC.

El orden de cierre es relevante. Eliminar un contexto antes de liberar los objetos gráficos impediría hacerlo sobre el contexto que los posee. Dejar un temporizador activo mientras se destruye la vista podría generar llamadas sobre datos que ya no están disponibles.

La verificación fuera de pantalla también reserva objetos temporales. Su función cleanup desenlaza el framebuffer, elimina textura, renderbuffer y framebuffer, y desactiva el contexto actual. Se llama al terminar correctamente y al capturar un error.

Los fallos de inicialización se registran en startup.log, frame.log y context.log. La existencia de un mensaje Renderer ready indica que la inicialización gráfica terminó en esa ejecución. Los logs de una ejecución antigua deben distinguirse por su fecha cuando se investiga un arranque posterior que se detuvo antes.

29 Pruebas de reglas y comprobación gráfica

Game3DTests crea estados de juego y ejecuta update directamente. check incrementa un contador y produce PASS o lanza una excepción con el nombre del caso. ticks repite un número conocido de pasos. pilot selecciona un enemigo y genera únicamente entradas de movimiento y disparo.

La suite actual realiza 58 comprobaciones. Cubre inicio, formación estática, tres niveles, velocidad, bordes, teclas opuestas, cadencia, pausa, colisiones XYZ, objetivos en el margen, velocidad de bala, fallo fuera de margen, cambio de blanco, puntuación única, caducidad, protección y reinicios.

Las pruebas de margen colocan blancos a distintas profundidades y con descentrados positivos y negativos. Tras emitir, desplazan al jugador y dejan que la bala termine su recorrido. De esa manera comprueban que la dirección quedó fijada y que el objetivo marcado resulta alcanzable.

Una partida completa usa el piloto y exige victoria con dieciocho bajas. Otra mantiene al jugador inmóvil y exige derrota natural. Estos casos pasan por las reglas normales de simulación, aunque no reproducen la experiencia subjetiva de una persona usando el teclado.

VerifyDemoRendering crea un framebuffer con textura RGBA8 y profundidad de 24 bits. Los manejadores MFC se invocan para seleccionar nave, comenzar, pausar, reanudar, volver al menú y reiniciar. Las capturas de victoria y derrota identificadas como fixture preparan esos estados para comprobar su presentación.

verifyLighting usa una cara de cubo y seis combinaciones aisladas de material y luz. Lee un píxel y lo compara con el color esperado. verifyGpu comprueba buffers y atributos. verifyFraming comprueba tamaños y márgenes. Finalmente se dibujan sesenta frames para medir un tiempo medio orientativo y confirmar que las mallas no vuelven a cargarse.

Los resultados se guardan en verification/render-report.txt y en PNG numerados. La prueba gráfica comprueba además que haya píxeles de profundidad, controles visibles y ausencia de errores OpenGL. La cifra de tiempo depende del equipo y de la carga del sistema.

30 Compilación y contenido de la carpeta ejecutable

El punto de entrada recomendado es build-demo.cmd. El archivo abre PowerShell y llama al script de compilación. La política de ejecución utilizada se aplica al proceso hijo. El script acepta Debug o Release, una opción Verify y una opción Package.

build-demo.ps1 localiza Visual Studio 2022 mediante vswhere. Prepara Mesa y llama a MSBuild sobre Galaga3D.sln en x64. Si la compilación falla, detiene el proceso. Después ejecuta Game3DTests y exige un resultado correcto.

Con Verify inicia Galaga3D.exe /verify-render en modo oculto, espera un máximo y revisa el código de salida y el informe. Antes elimina el informe anterior para que un archivo antiguo no se interprete como prueba de esa ejecución. Si se agota el tiempo, detiene el proceso creado por la verificación y comunica el fallo.

Con Package, que se utiliza en Release, copia el EXE, opengl32.dll, recursos, shaders y texturas a dist/Galaga3D. También copia los componentes de ejecución de Visual C++ y MFC y sus avisos de licencia. La carpeta completa es la unidad práctica de distribución.

El proyecto EntornVGI.vcxproj define el estándar C++17, el toolset v143 y la ubicación de salida. Incorpora los archivos propios a la compilación y referencia los proyectos de GLEW y SOIL2. Su target CopyDemoResources copia recursos después de construir la aplicación.

Game3D.cpp se compila sin la cabecera precompilada del entorno. Esto ayuda a mantener la lógica independiente de MFC. Los módulos de renderizado utilizan pch.h y los tipos gráficos proporcionados por la plantilla. Las pruebas enlazan directamente con la lógica.

La solución conserva configuraciones de la plantilla, pero el flujo documentado y comprobado utiliza x64. Para construir de nuevo se necesita Visual Studio con desarrollo de escritorio C++, MFC v143 y Windows SDK. La primera preparación de Mesa requiere red si sus archivos verificados todavía no están presentes.

31 Mesa y las dependencias explicadas

GLEW facilita acceder a las funciones modernas de OpenGL después de crear el contexto. GLM proporciona vectores, matrices, proyecciones y cuaterniones con una notación compatible con GLSL. SOIL2 carga imágenes del HUD y escribe capturas. MFC proporciona la estructura de ventanas y mensajes de Windows.

En el equipo de desarrollo, el controlador del sistema ofrecía inicialmente GDI Generic con OpenGL 1.1. El paquete incorpora Mesa LLVMpipe como biblioteca local opengl32.dll para disponer del pipeline moderno. LLVMpipe ejecuta el rasterizado mediante CPU. El contexto verificado indica OpenGL 4.6 Core Profile.

Las llamadas a VBO, VAO y shaders siguen formando parte del pipeline OpenGL. La biblioteca que implementa esas llamadas determina dónde se ejecutan físicamente. La presencia de buffers modernos no demuestra por sí sola aceleración en una GPU dedicada.

setup-mesa.ps1 fija una versión y hashes SHA256. Get-LocalSha256 abre un archivo como flujo, calcula su huella y libera los recursos .NET en finally. Get-VerifiedFile compara el archivo existente, descarga a un nombre temporal cuando hace falta y solo lo mueve al destino cuando coincide la huella esperada.

La DLL se extrae de un archivo comprimido y se verifica de nuevo. El script también conserva o restaura los avisos de licencia necesarios. Con Deploy copia la DLL a las carpetas de ejecución Debug y Release cuando corresponde.

Los hashes ayudan a detectar una descarga incompleta o un contenido distinto de la versión fijada. Si un proveedor cambia los archivos publicados, el fallo debe investigarse. La opción correcta consiste en validar el nuevo origen y sus licencias antes de actualizar una huella.

Las dependencias tienen su propia implementación y licencia. El anexo de esta guía recoge el código de integración y de la demo. El código interno de MFC, GLM, GLEW, SOIL2 y Mesa pertenece a sus respectivos proyectos y se consulta en sus fuentes cuando hace falta investigar una incidencia de esas capas.

32 Decisiones de diseño y evolución del código

La demo evolucionó desde un prototipo muy pequeño hacia una representación 3D dentro de la plantilla del curso. Se mantuvo una ronda para hacer comprobable el ciclo completo. El usuario concretó después el movimiento exclusivamente lateral, la profundidad real, la tercera persona, la formación estática y el fondo negro.

Las primeras disposiciones alejaban o reducían los modelos sin considerar suficientemente su tamaño en pantalla. La revisión combinó cámara, campo de visión, escala y margen del HUD. Se añadieron comprobaciones de encuadre para que un cambio posterior revele una formación demasiado pequeña o una nave recortada.

El problema de disparos descentrados se relacionó con tres decisiones distintas que no encajaban. El margen de selección era mayor que el radio de impacto y la bala conservaba velocidad lateral cero. La solución comparte el objetivo entre dibujo y disparo y calcula la trayectoria completa al emitir.

Las balas podían crecer visualmente al pasar detrás del jugador y acercarse a la cámara. Su retirada se trasladó a una franja inmediatamente posterior a la nave. La orientación también se corrigió para utilizar su velocidad XYZ completa.

La revisión de los materiales de VGI confirmó la estructura de mallas, matrices, texturas y ocultaciones. La iluminación se adaptó al modelo de Phong descrito en GL-6, con estructuras de luz y material y sus valores enviados desde C++. Se añadieron pruebas sobre píxeles reales para comprobar los términos.

El proyecto conserva algunos datos de estados y efectos anteriores para mantener compatibilidad entre módulos. Al ampliar funcionalidades conviene decidir qué campos se activan y cuáles pueden retirarse con seguridad. La existencia de un enum o de una malla adicional debe leerse junto a las ramas que los utilizan.

Los criterios de la demo son decisiones verificables del proyecto. La aceptación académica de una entrega depende también de su enunciado y de la revisión del profesorado. La documentación identifica el uso de asistencia de IA y las restricciones indicadas en la guía docente.

33 Cómo modificar el juego con criterio

Para cambiar la velocidad lateral, hay que localizar la multiplicación por 6 en update y revisar el test de movimiento y los límites de pantalla. Para cambiar la cadencia se modifica fireClock y se vuelve a calcular qué pasos discretos permiten disparar. La cifra observada puede diferir del valor continuo por la resolución de 1/60.

Al cambiar el número de enemigos deben revisarse el bucle de reset, las filas y columnas, los textos del menú y las pruebas que esperan dieciocho. La condición remaining()==0 seguirá sirviendo, pero la composición puede salir del campo si los nuevos valores no se proyectan dentro de sus límites.

Al modificar EnemyScale se debe comprobar EnemyHitRadius, la marca proyectada y las pruebas de impacto. Una malla grande con un volumen vulnerable muy pequeño puede resultar confusa. Una envolvente excesiva puede producir impactos que se perciben alejados de la nave.

Para cambiar la cámara se revisan eye, el punto de mirada, up y el campo de visión. Deben capturarse las posiciones central y extremas. La prueba de encuadre muestra medidas concretas. El HUD usa un lienzo virtual y su relación con el viewport debe conservarse al cambiar la proporción de pantalla.

Una nueva arma requiere datos o reglas de emisión en Game3D, una representación gráfica, controles o interfaz para seleccionarla y pruebas de daño, cadencia, consumo y reinicio. La lógica del arma y su apariencia deben documentarse como una misma decisión funcional.

Una nueva skin debería cambiar recursos o materiales y conservar las reglas que se hayan acordado para las naves. El sistema de monedas y cajas del GDD necesita además persistencia, operaciones de compra y casos que eviten duplicar recompensas. Esos componentes pertenecen al alcance futuro del proyecto completo.

Si se añade una malla hay que revisar ModelLibrary, sus nueve cargas esperadas, los arreglos y la liberación. Si se cambia un atributo del vértice, deben actualizarse la estructura C++, los offsets, el shader y verifyGpu. Esas modificaciones afectan un contrato compartido entre CPU y renderizado.

34 Diagnóstico de problemas y límites actuales

Un fallo de compilación se investiga desde el primer error que muestra MSBuild. Un error posterior puede ser consecuencia del primero. Los problemas de enlazado suelen estar relacionados con bibliotecas, símbolos o archivos de salida bloqueados por una aplicación todavía abierta.

Un fallo anterior a la pantalla inicial se investiga con startup.log. frame.log detalla etapas de creación del marco MFC. context.log indica formato de píxel, contexto OpenGL y resultado del renderizador. Si falta un shader o un PNG, se comprueba que la carpeta de ejecución esté completa y que la copia de recursos haya terminado.

Una pantalla con HUD pero sin naves requiere revisar el programa 3D, sus mallas, las matrices y el depth buffer. Una malla invisible desde un lado puede señalar orden incorrecto de vértices o una normal mal construida. Una textura invertida requiere revisar una sola convención de filas y coordenadas UV.

Una bala que parece atravesar un enemigo se analiza con sus posiciones XYZ, el objetivo marcado, el recorrido y el radio efectivo. La captura de pantalla representa una proyección. El test de mundo permite distinguir un fallo de trayectoria de una coincidencia visual entre objetos a diferentes distancias.

Se observó un fallo intermitente de creación del marco MFC. Se añadieron registros y las últimas series de arranque verificadas pasaron. Su causa raíz continúa pendiente. Los reintentos correctos aportan evidencia de funcionamiento, pero la incidencia se conserva abierta para reproducirla con información suficiente.

Los tests automáticos comprueban casos definidos. La verificación gráfica revisa el pipeline y varias composiciones. La valoración de una sesión manual por el jugador sigue siendo necesaria para decidir ritmo, comodidad y legibilidad durante una partida completa.

La versión documentada carece de varias ampliaciones del GDD, como tres oleadas, jefe jugable, armas adicionales, economía y skins. Su implementación futura debe conservar la claridad del alcance y ampliar las pruebas cuando cambien las reglas. El seguimiento de estas tareas se realiza con los documentos del equipo.

35 Correspondencia con la asignatura VGI

GL-2 desarrolla el pipeline programable, las etapas de shader y la comunicación mediante atributos y uniform. La aplicación utiliza vertex y fragment shader y comprueba compilación y enlace antes de jugar.

GL-3, especialmente sus páginas 7, 10, 28-38 y 160, presenta VBO, VAO y reutilización de geometría. Mesh3D aplica ese ciclo de carga, dibujo y liberación. La variante glDrawArrays está descrita en los materiales y permite representar las mallas trianguladas de esta demo.

GL-4, páginas 13-15, 25-30, 56 y 58, relaciona transformaciones GLM, lookAt, perspective y paso de matrices al shader. El orden T·R·S y projection·view·model se observa directamente en Renderer3D y galaga3d.vert.

GL-5, páginas 5-9 y 14-19, presenta orientación de caras, culling y Z-buffer. La pasada 3D configura CCW, descarta caras posteriores y utiliza GL_LESS. La pasada HUD se ejecuta en pantalla después de la geometría.

GL-6, páginas 6, 12-13, 18, 27-30, 37-38 y 49-52, explica normales, fuente direccional y términos de material. El cálculo en coordenadas de mundo está contemplado en sus páginas 25 y 27. La implementación calcula la normal inversa transpuesta en C++ y evalúa Phong por fragmento.

GL-7, páginas 5-10 y 18-20, presenta texturas, sampler y SOIL. El HUD carga PNG con SOIL2. La mezcla alfa coincide con GL-6, página 48. Los materiales del curso también tratan técnicas más avanzadas, cuyo uso depende del alcance concreto del proyecto.

VGI-7 incluye cuaterniones en las páginas 46-58 y envolventes esféricas en 133-134. VGI-8, páginas 79 y 86, distingue motor, gameplay y estados. VGI-G, páginas 11-15, aborda dificultad y coste de la IA. La demo utiliza esas bases en una implementación pequeña y reproducible.

La colección VGI - Teoria i Problemes contiene treinta PDF y 2.919 páginas. Se indexaron sus documentos y se contrastaron los apartados aplicables. El archivo LLEGEIX-ME delimita la colección y sitúa los enunciados y entornos prácticos fuera de ella. La rúbrica concreta sigue siendo la referencia para aceptar una entrega académica.

36 Glosario para leer el código

Datos y reglas

Estado es la situación actual de la partida, como menú o pausa. Entidad es un elemento del mundo, como una nave o una bala. Instancia es una aparición concreta de un modelo. ID es un identificador que permite distinguir un objeto de otros. Temporizador es un valor que mide cuánto falta para una acción. TTL es el tiempo de vida restante de un objeto.

Input agrupa la intención del jugador. Gameplay reúne las reglas de interacción. IA reactiva toma decisiones sencillas a partir de la situación presente. Cadencia es el intervalo entre disparos. Cooldown es el tiempo de espera hasta poder volver a actuar. Hitbox o collider es la forma simplificada utilizada para calcular contactos.

Espacio y geometría

Vértice es un punto de una malla con sus atributos. Triángulo es una superficie formada por tres vértices. Malla es un conjunto de superficies que representa un objeto. Normal es un vector perpendicular a una cara. Vector expresa una dirección o desplazamiento. Matriz representa una transformación que puede aplicarse a muchos puntos.

Traslación cambia la posición. Rotación cambia la orientación. Escala cambia el tamaño. Cuaternión representa una orientación o giro de manera compacta. Producto escalar permite medir la relación entre dos direcciones. Producto vectorial produce un vector perpendicular a dos direcciones y permite obtener normales.

Coordenadas locales describen un modelo respecto a su origen. Coordenadas de mundo sitúan los objetos en la escena. Coordenadas de cámara expresan la escena respecto al observador. Perspectiva hace depender el tamaño aparente de la distancia. Viewport es el rectángulo de pantalla donde se dibuja una vista.

Renderizado

Renderizar es producir una imagen a partir del estado y de los recursos. Frame es una imagen completa. Shader es un programa que participa en una etapa del pipeline gráfico. Vertex shader transforma vértices. Fragment shader calcula los valores de fragmentos que pueden convertirse en píxeles visibles.

VBO almacena datos de vértices. VAO conserva la configuración con la que se leen esos datos. EBO almacena índices de vértices. Uniform es un dato común para una llamada de dibujo. Atributo es un dato asociado a cada vértice. Interpolación calcula valores intermedios entre los vértices de un triángulo.

Textura es una imagen utilizada por el renderizador. UV son las coordenadas para buscar un punto de una textura. Sampler indica cómo acceder a una textura desde un shader. Alfa representa cobertura o transparencia. Blending mezcla el resultado nuevo con el color que ya existía.

Depth buffer guarda profundidad para decidir qué superficie es visible. Culling descarta caras según su orientación. Framebuffer agrupa destinos de dibujo, como color y profundidad. FBO es un framebuffer gestionado explícitamente por OpenGL. Doble buffer permite preparar una imagen antes de mostrarla.

Herramientas

Compilar transforma código fuente en unidades ejecutables por el sistema. Enlazar reúne las unidades y bibliotecas en un programa. DLL es una biblioteca cargada en ejecución. Debug es una configuración útil para investigar. Release aplica optimizaciones para distribuir. Commit registra una versión en Git. Hash identifica el contenido de un archivo. Fixture es un estado preparado para una prueba concreta.

Referencia de funciones del juego

Esta referencia permite encontrar rápidamente qué hace una función y qué datos modifica. Las explicaciones extensas están en los capítulos anteriores. Las firmas y los cuerpos exactos aparecen en el anexo de código.

Game3D.h y Game3D.cpp

Función	Responsabilidad y precaución
Bullet::forwardSpeed	Devuelve velocityY cuando se ha calculado una dirección. Para un proyectil sencillo con velocityY cero utiliza la velocidad del equipo, positiva para aliado y negativa para hostil.
aimTarget	Filtra enemigos vivos delante del morro y dentro del margen lateral. Devuelve el de menor Y. El resultado sirve tanto al disparo como al HUD y se consulta de nuevo cuando hace falta.
reset	Conserva selección y récord, reconstruye valores iniciales y crea los dieciocho enemigos. Iguala posiciones anteriores y actuales para evitar recorridos ficticios en el primer paso.
returnToMenu	Vuelve al estado inicial Menu conservando modelo elegido y récord. Vacía el mundo de la ronda anterior mediante la asignación de un Game3D nuevo.
selectShip	Acepta índices de cero a dos durante Menu. Las otras fases conservan la nave elegida para esa partida.
pause	Alterna Playing y Paused. La entrada de otros estados no cambia su fase.
remaining	Cuenta enemigos que mantienen alive verdadero. La colección puede conservar objetos muertos sin afectar al contador.
hitTime	Es una forma de llamar a hitTime3D con profundidades cero. Conserva una interfaz útil para casos planos.
hitTime3D	Resuelve la primera intersección de una trayectoria relativa con una esfera efectiva durante el paso. Devuelve t entre cero y uno, o -1 cuando no hay contacto válido.
clearCombo	Pone hitStreak a cero y multiplier a uno. Se utiliza al recibir daño y al detectar tiros fallados que abandonan la ronda.
damage	Comprueba las protecciones y vidas. Cuando el daño es válido resta una vida e inicia reaparición y protección.
update	Ejecuta la secuencia completa de un paso. Actualiza entrada, temporizadores, emisiones y movimientos, resuelve contactos, retira objetos y determina el resultado.

GameView3D.cpp

Función	Responsabilidad y precaución
ClearGameInput	Vacía teclas y acumulador y reinicia el reloj. Asegura una entrada limpia al pausar, cambiar de pantalla o perder el foco.
OnGameDemo	Activa la demo, oculta paneles docentes, limpia entrada, recupera el foco y solicita repintado.
OnGameEnvironment	Comprueba el contexto requerido por el entorno docente, pausa el combate y restaura sus paneles.
OnKillFocus	Pausa una partida activa, limpia teclas y comunica el cambio a la clase base MFC.
PreTranslateMessage	Intercepta las teclas de juego y dirige sus eventos a la vista, evitando que se consuman en otra función de la interfaz.
GameKeyDown	Distingue pulsaciones nuevas, realiza las transiciones de menú, pausa y reinicio y actualiza gameKeys.
GameTick	Mide tiempo real, comprueba foco y modo, acumula tiempo y ejecuta hasta cinco pasos fijos. Solicita el siguiente repintado.

Construcción de geometría en Mesh3D.cpp

Función	Responsabilidad y precaución
triangle	Calcula una normal por producto vectorial, descarta triángulos degenerados y añade tres vértices con color y UV.
quad	Divide un cuadrilátero en dos triángulos manteniendo su orden de vértices.
hull	Construye un sólido cerrado desde un contorno convexo. Corrige el sentido del contorno y añade tapa interior, bisel, laterales y base.
box	Genera seis caras a partir del centro y semitamaños del bloque.
engine	Construye un motor hexagonal alrededor del eje Y y colorea su salida trasera.
cockpit	Genera la cabina como un pequeño sólido elevado con contorno alargado.
eyes	Añade bloques oscuros y pequeños reflejos para los ojos del insecto.
flecha	Reúne alas, fuselaje, motores, detalles y cabina de la primera nave.
delta	Construye el ala triangular y los detalles de la segunda nave.
twinEngine	Forma dos cuerpos laterales, su travesaño y la cabina central.
bee y butterfly	Crean las dos siluetas de enemigo utilizadas en la ronda.
boss	Construye la malla adicional que aparece en la galería y queda reservada para una ampliación.
projectile	Genera la geometría longitudinal de la bala con color distinto según equipo.

Mesh3D y ModelLibrary

Función	Responsabilidad y precaución
Mesh3D::upload	Valida datos y crea VAO/VBO estáticos. Configura los atributos a partir de sizeof y offsetof.
Mesh3D::draw	Enlaza el VAO válido y emite glDrawArrays para todos sus triángulos.
Mesh3D::verifyGpu	Consulta memoria y atributos reales y restaura las asociaciones anteriores.
Mesh3D::release	Elimina los recursos de la malla con el contexto activo y pone los identificadores a cero.
valid y vertexCount	Permiten consultar si existe una malla cargada y cuántos vértices contiene.
ModelLibrary::initialize	Construye y carga las nueve mallas compartidas. Evita una segunda carga completa y limpia después de un fallo.
ModelLibrary::release	Libera todas las mallas y reinicia el contador de cargas.
ModelLibrary::verifyGpu	Verifica todas las mallas y comprueba el contador esperado y los errores OpenGL.
ModelLibrary::vertexCount	Suma los vértices de naves, enemigos y objetos auxiliares.

Renderer3D.cpp

Función	Responsabilidad y precaución
compileFile	Lee un shader junto al EXE, lo compila y obtiene el mensaje de error si no es válido.
transform	Construye la matriz T·R·S de una instancia con posición, escala y giro local.
initialize	Carga shaders, enlaza programa, comprueba uniform e inicializa biblioteca y HUD.
release	Libera HUD, mallas y programa gráfico.
begin3D	Fija profundidad, caras, programa, matrices comunes y parámetros de luz.
camera	Prepara la vista ortográfica de selección o la cámara en tercera persona de partida.
instance	Envía model, normalMatrix, tint y material. Dibuja una malla ya cargada y cuenta la llamada.
draw	Coordina fondo, escena, objetivo y HUD para la fase actual.
verifyGpu	Comprueba programa, mallas, cargas y uniform esenciales y escribe el informe.
verifyLighting	Dibuja un cubo en seis condiciones aisladas y compara un píxel leído con el resultado previsto de la ecuación.
verifyFraming	Proyecta volúmenes aproximados para medir legibilidad y márgenes. Se utiliza durante la verificación.
modelGallery	Dibuja seis modelos desde una orientación controlada para comprobar volumen y matrices.

Renderer2D.cpp

Función	Responsabilidad y precaución
executableDirectory	Obtiene la carpeta del EXE mediante GetModuleFileNameW y filesystem.
initialize	Crea shaders del HUD, buffers, textura blanca, atlas de texto y PNG del proyecto.
release	Elimina texturas, programa y buffers de la interfaz.
begin	Configura lienzo virtual, viewport, mezcla alfa y atributos de la pasada 2D.
finish	Cierra el lote pendiente, envía los vértices de la pasada y dibuja cada rango con su textura.
flush	Registra un lote pendiente y actualiza el comienzo del siguiente rango.
quad	Añade los seis vértices de un rectángulo con color y UV.
image	Agrupa un rectángulo con la textura indicada y conserva el orden de dibujo.
border	Construye cuatro tiras para el borde de un panel.
text	Convierte caracteres a celdas del atlas, aplica tamaño y centrado y crea sus rectángulos.
background	Limpia color y profundidad, dibuja fondo negro y tarjetas de selección si corresponde.
overlay	Dibuja los indicadores de la fase, la marca de objetivo y los paneles de pausa o resultado.

Integración y comprobación

Función	Responsabilidad y precaución
InitInstance	Construye la aplicación MFC, su marco y documento. Decide entre ejecución interactiva y verificación fuera de pantalla.
ExitInstance	Finaliza OLE y MFC y devuelve el resultado de verificación cuando corresponde.
OnCreate de la vista	Crea el contexto de dispositivo y OpenGL, carga el renderizador, lee récord e instala el temporizador.
OnPaint	Activa el contexto, dibuja el estado y realiza SwapBuffers en modo demo.
OnDestroy	Detiene el temporizador, guarda récord en cierre normal y libera renderizador y contexto.
SetDemoLayout	Muestra u oculta paneles docentes y recalcula el marco.
VerifyDemoRendering	Coordina el framebuffer, capturas, entradas MFC, checks gráficos y limpieza.
check y ticks	Comprueban una condición de test y repiten un número conocido de pasos de lógica.
pilot	Produce entradas laterales y de disparo para una partida completa automática.

Scripts de compilación

Función	Responsabilidad y precaución
build-demo.cmd	Lanza el script PowerShell con los argumentos recibidos o las opciones de compilación, verificación y paquete por defecto.
build-demo.ps1	Localiza herramientas, prepara dependencias, compila, ejecuta pruebas y empaqueta cuando se solicita.
Get-LocalSha256	Calcula la huella de un archivo mediante un flujo y libera sus objetos .NET.
Get-VerifiedFile	Reutiliza un archivo validado o descarga uno temporal, verifica su hash y lo mueve al destino.
setup-mesa.ps1	Prepara la versión fijada de Mesa y sus licencias y la despliega junto a Debug y Release.

Mapa de parámetros y valores actuales

Parámetro	Valor	Consecuencia
Step	1/60 s	La lógica avanza en pasos de tiempo iguales.
Velocidad lateral	6 unidades/s	Mover durante un segundo recorre aproximadamente seis unidades.
Límites laterales	-7,68 a +7,68	La posición se limita con clamp.
Jugador	Y=-4, Z=0	El desplazamiento controlado corresponde a X.
Formación	3 filas × 6 columnas	18 enemigos al iniciar.
Separación horizontal	2,6 unidades	X entre -6,5 y +6,5.
Filas	Y=14, 18, 22	Distancia hacia delante en el mundo.
Alturas	Z=-2,5, 0, +2,5	Desnivel entre las filas.
EnemyScale	1,45	Escala visual uniforme de los enemigos.
EnemyHitRadius	0,76	Radio efectivo utilizado en contacto bala aliada/enemigo.
AimTolerance	0,65	Margen lateral para seleccionar un objetivo.
Salida del disparo	Y del jugador +0,85	El proyectil nace delante de la punta de la nave.
PlayerShotSpeed	24 unidades/s	Rapidez total del proyectil dirigido.
EnemyShotSpeed	7 unidades/s	Rapidez total del proyectil hostil.
Cadencia aliada	0,22 s	El paso fijo acepta emisiones aproximadamente cada 0,233 s.
Límite aliado	8 proyectiles activos	Bloquea nuevas emisiones al alcanzar el límite.
Vida útil aliada	3,5 s	Límite adicional a los bordes de retirada.
Primer ataque enemigo	2,4 s	Tiempo de espera inicial.
Ataques siguientes	1,2 s	Intervalo del director de disparos.
Reaparición	0,75 s	Espera antes de volver al centro.
Protección	1,5 s	Se consume después de reaparecer.
Contacto hostil	0,42	Radio efectivo bala/jugador.
Contacto entre naves	0,68	Caso defensivo conservado en la simulación.
Retirada por detrás	Y < -4,8	Elimina los tiros que ya han pasado al jugador.
Retirada por delante	Y > 28	Elimina tiros que atraviesan la formación.

Parámetro	Valor	Consecuencia
Cámara	(0,15·X, -16, 6)	Seguimiento lateral parcial.
Mirada	(0,15·X, 8, 0)	La cámara apunta por delante del jugador.
Perspectiva	38°, 16:9, 0,1 a 100	Ángulo vertical, relación de aspecto y límites de profundidad.
Material reflectante	0,37 / 0,63 / 0,16	Coeficientes ambiente, difuso y especular.
Shininess	32	Concentración del reflejo especular.

Anexo de código y trazabilidad

Los listados se obtienen directamente de los archivos de esta revisión. Cada línea conserva su número original. Las líneas largas se continúan con una barra vertical. La referencia del repositorio anterior a esta publicación es 51b1f0a5e6d6. Los hashes permiten comprobar exactamente qué archivos se documentaron.

Código Game3D.h

Entrega_Amin_VGI\entornsGrafics\EntornVGI\Game3D.h

```
 1 | #pragma once
 2 | #include <vector>
 3 |
 4 | namespace galaga {
 5 | struct Vec2 { float x = 0, y = 0; };
 6 | enum class Phase { Menu, Playing, Paused, Victory, Defeat };
 7 | enum class EnemyPhase { Entering, Formation, Warning, Diving, Returning };
 8 | struct Input { bool left = false, right = false, fire = false; };
 9 | struct Enemy {
10 |     int id = 0, type = 0;
11 |     bool alive = true, shot = false;
12 |     EnemyPhase phase = EnemyPhase::Entering;
13 |     Vec2 pos, previous, slot, start;
14 |     float timer = 0, targetX = 0;
15 |     float z = 0, previousZ = 0;
16 | };
17 | struct Bullet {
18 |     int id = 0;
19 |     bool ally = true, alive = true;
20 |     Vec2 pos, previous;
21 |     float ttl = 3.5f;
22 |     float z = 0, previousZ = 0, velocityZ = 0;
23 |     float velocityX = 0;
24 |     float velocityY = 0;
25 |     float forwardSpeed() const;
26 | };
27 | struct Spark { Vec2 pos; float age = 0; bool player = false; float z = 0; };
28 |
29 | // Pure simulation: no MFC, GL, system clock or file dependencies.
30 | class Game3D {
31 | public:
32 |     static constexpr float Step = 1.0f / 60.0f;
33 |     static constexpr float EnemyScale = 1.45f;
34 |     static constexpr float EnemyHitRadius = .76f;
35 |     static constexpr float AimTolerance = .65f;
36 |     static constexpr float PlayerShotSpeed = 24.f;
```

```
    static constexpr float EnemyShotSpeed = 7.f;
    Phase phase = Phase::Menu;
    Vec2 player{0, -4}, previousPlayer{0, -4};
    int lives = 3, score = 0, shots = 0, kills = 0;
    int selectedShip = 0, bestScore = 0, hitStreak = 0, multiplier = 1;
    float time = 0, invulnerable = 1.5f, respawn = 0;
    std::vector<Enemy> enemies;
    std::vector<Bullet> bullets;
    std::vector<Spark> sparks;
    void reset();
    void returnToMenu();
    void selectShip(int ship);
    void pause();
    void update(const Input& input, float dt = Step);
    int remaining() const;
    const Enemy* aimTarget() const;
    static float hitTime(Vec2 a0, Vec2 a1, Vec2 b0, Vec2 b1, float radius);
    static float hitTime3D(Vec2 a0, Vec2 a1, float za0, float za1,
        Vec2 b0, Vec2 b1, float zb0, float zb1, float radius);
private:
    float fireClock = 0, attackClock = 2.4f;
    int nextId = 100, nextAttacker = 0;
    void damage();
    void clearCombo();
};
}
```

Código Game3D.cpp

Entrega_Amin_VGI\entornsGrafics\EntornVGI\Game3D.cpp

```
#include "Game3D.h"
#include <algorithm>
#include <cmath>

namespace galaga {
float Bullet::forwardSpeed() const {
    return velocityY != 0 ? velocityY : (ally ? Game3D::PlayerShotSpeed : -Game3D::EnemyShotSpeed);
}

const Enemy* Game3D::aimTarget() const {
    const Enemy* target = nullptr;
    for (const auto& enemy : enemies) {
        if (!enemy.alive || enemy.pos.y <= player.y + .85f)
            continue;
        if (std::abs(enemy.pos.x - player.x) > AimTolerance)
            continue;
        if (!target || enemy.pos.y < target->pos.y)
            target = &enemy;
    }
    return target;
}

void Game3D::reset() {
    const int ship = selectedShip, record = std::max(bestScore, score);
    *this = Game3D{};
    selectedShip = std::clamp(ship, 0, 2); bestScore = record;
    phase = Phase::Playing;
    for (int i=0; i<18; ++i) {
        Enemy e; e.id=i; e.type=i<12?0:1;
        e.slot={(i%6-2.5f)*2.6f,14.0f+(i/6)*4.0f};
        e.phase=EnemyPhase::Formation;
        e.pos=e.previous=e.slot;
        // Three persistent world depths, not camera layers or selectable planes.
        e.z=e.previousZ=float(i/6-1)*2.5f;
        enemies.push_back(e);
    }
}
void Game3D::returnToMenu() {
```

```
    const int ship = selectedShip, record = std::max(bestScore, score);
    *this = Game3D{};
    selectedShip = std::clamp(ship, 0, 2); bestScore = record;
}
void Game3D::selectShip(int ship) {
    if (phase==Phase::Menu && ship>=0 && ship<3) selectedShip=ship;
}
void Game3D::pause() {
    if (phase==Phase::Playing) phase=Phase::Paused;
    else if (phase==Phase::Paused) phase=Phase::Playing;
}
int Game3D::remaining() const {
    return static_cast<int>(std::count_if(enemies.begin(),enemies.end(),[](const Enemy& e){return
e.alive;}));
}
float Game3D::hitTime(Vec2 a0, Vec2 a1, Vec2 b0, Vec2 b1, float r) {
    return hitTime3D(a0,a1,0,0,b0,b1,0,0,r);
}
float Game3D::hitTime3D(Vec2 a0, Vec2 a1, float za0, float za1,
    Vec2 b0, Vec2 b1, float zb0, float zb1, float r) {
    const float x=a0.x-b0.x,y=a0.y-b0.y,z=za0-zb0;
    const float vx=a1.x-a0.x-b1.x+b0.x,vy=a1.y-a0.y-b1.y+b0.y,vz=za1-za0-zb1+zb0;
    const float c=x*x+y*y+z*z-r*r, a=vx*vx+vy*vy+vz*vz;
    if(c<=0) return 0;
    if(a<1e-10f) return -1;
    const float b=2*(x*vx+y*vy+z*vz),d=b*b-4*a*c;
    if(d<0) return -1;
    const float t=(-b-std::sqrt(d))/(2*a);
    return t>=0 && t<=1?t:-1;
}
void Game3D::clearCombo() { hitStreak=0; multiplier=1; }
void Game3D::damage() {
    if(invulnerable>0 || respawn>0 || lives<=0) return;
    --lives; clearCombo();
    respawn=.75f; invulnerable=1.5f;
}
void Game3D::update(const Input& input,float dt) {
    if(phase!=Phase::Playing || dt<=0) return;
    time+=dt;
    sparks.clear();
    previousPlayer=player;
    if(respawn>0) {
        respawn=std::max(0.0f,respawn-dt);
        if(respawn==0) player=previousPlayer={0,-4};
    } else {
        invulnerable=std::max(0.0f,invulnerable-dt);
        player.x=std::clamp(player.x+6.0f*dt*(int(input.right)-int(input.left)),-7.68f,7.68f);
        fireClock=std::max(0.0f,fireClock-dt);
        const auto active=std::count_if(bullets.begin(),bullets.end(),[](const Bullet& b){return
b.alive&&b.ally;});
        if(input.fire && fireClock==0 && active<8) {
            Vec2 p{player.x,player.y+.85f};
            Bullet shot{nextId++,true,true,p,p};
            if (const Enemy* target = aimTarget()) {
                const float dx = target->pos.x - p.x;
                const float dy = target->pos.y - p.y;
                const float dz = target->z;
                const float flight = std::sqrt(dx*dx + dy*dy + dz*dz) / PlayerShotSpeed;
                shot.velocityX = dx / flight;
                shot.velocityY = dy / flight;
                shot.velocityZ = dz / flight;
            }
            bullets.push_back(shot);
            fireClock=.22f; ++shots;
        }
    }
    // Basic AI: stationary formation; one enemy fires at a time.
    for(auto& e:enemies) {e.previous=e.pos;e.previousZ=e.z;}
    attackClock-=dt;
    if(attackClock<=0 && !enemies.empty()) {
        for(size_t n=0;n<enemies.size();++n) {
```

```
108 |             const size_t index=(nextAttacker+n)%enemies.size();
109 |             const auto& e=enemies[index];if(!e.alive)continue;
110 |             const float dx = player.x - e.pos.x;
111 |             const float dy = player.y - e.pos.y;
112 |             const float flight = std::sqrt(dx*dx + dy*dy + e.z*e.z) / EnemyShotSpeed;
113 |             Bullet shot{nextId++,false,true,e.pos,e.pos,flight+1.f,e.z,e.z,-e.z/flight};
114 |             // Aim once on emission; no tracking after the shot has left.
115 |             shot.velocityX=(player.x-e.pos.x)/flight;
116 |             shot.velocityY=dy/flight;
117 |             bullets.push_back(shot);nextAttacker=int((index+1)%enemies.size());break;
118 |         }
119 |         attackClock=1.2f;
120 |     }
121 |     for(auto& b:bullets) {
122 |         if (!b.alive || b.ttl <= 0) {
123 |             b.alive = false;
124 |             continue;
125 |         }
126 |         b.previous=b.pos;
127 |         b.previousZ=b.z;
128 |         const float activeTime=std::min(dt,std::max(0.0f,b.ttl));
129 |         b.pos.y+=b.forwardSpeed()*activeTime;
130 |         b.pos.x+=b.velocityX*activeTime;
131 |         b.z+=b.velocityZ*activeTime;b.ttl-=dt;
132 |     }
133 |     // Resolve swept contacts chronologically, then by stable IDs. Never erase while iterating.
134 |     struct Contact {float t;int bullet,enemy,id;};
135 |     std::vector<Contact> contacts;
136 |     for(int i=0;i<int(bullets.size());++i) {
137 |         const auto& b=bullets[i];
138 |         if(b.ally) {
139 |             for(int j=0;j<int(enemies.size());++j) {
140 |                 const auto& e=enemies[j]; if(!e.alive||e.timer<0)continue;
141 |                 float
    | t=hitTime3D(b.previous,b.pos,b.previousZ,b.z,e.previous,e.pos,e.previousZ,e.z,EnemyHitRadius);
142 |                 if(t>=0)contacts.push_back({t,i,j,e.id});
143 |             }
144 |         } else if(respawn==0) {
145 |             float t=hitTime3D(b.previous,b.pos,b.previousZ,b.z,previousPlayer,player,0,0,.42f);
146 |             if(t>=0)contacts.push_back({t,i,-1,b.id});
147 |         }
148 |     }
149 |     std::sort(contacts.begin(),contacts.end(),[](const Contact& a,const Contact& b){
150 |         if(a.t!=b.t)return a.t<b.t;if(a.id!=b.id)return a.id<b.id;return a.bullet<b.bullet;
151 |     });
152 |     for(const auto& c:contacts) {
153 |         auto& b=bullets[c.bullet];if(!b.alive)continue;
154 |         if(c.enemy<0) {b.alive=false;damage();}
155 |         else {
156 |             auto& e=enemies[c.enemy]; if(!e.alive)continue;
157 |             b.alive=false;e.alive=false;++kills;
158 |             ++hitStreak;multiplier=std::min(3,1+hitStreak/3);
159 |             int points=e.type?150:100;
160 |             score+=(e.phase==EnemyPhase::Diving?points*3/2:points)*multiplier;
161 |             bestScore=std::max(bestScore,score);
162 |
163 |         }
164 |     }
165 |     if(respawn==0) for(auto& e:enemies) {
166 |         if(e.alive && e.timer>=0 &&
    | hitTime3D(e.previous,e.pos,e.previousZ,e.z,previousPlayer,player,0,0,.68f)>=0) {
167 |             e.alive=false;damage();
168 |         }
169 |     }
170 |     for(const auto& b:bullets) {
171 |         if(b.ally && b.alive && (b.ttl<=0||b.pos.y>28||b.pos.y<-7)) clearCombo();
172 |     }
173 |     bullets.erase(std::remove_if(bullets.begin(),bullets.end(),[](const Bullet& b){
174 |         // Retire missed enemy shots immediately after they pass the player.
175 |         return !b.alive||b.ttl<=0||b.pos.y>28||b.pos.y<-4.8f;
176 |     }),bullets.end());
```

```
177 |     if(lives==0) phase=Phase::Defeat;
178 |     else if(remaining()==0) phase=Phase::Victory;
179 |     if(phase!=Phase::Playing) bullets.clear();
180 | }
181 | }
```

Código GameView3D.cpp

Entrega_Amin_VGI\entornsGrafics\EntornVGI\GameView3D.cpp

```
  1 | #include "pch.h"
  2 | #include "EntornVGIDoc.h"
  3 | #include "EntornVGIView.h"
  4 |
  5 | void CEntornVGIView::ClearGameInput() {
  6 |     gameKeys.fill(false); gameAccumulator=0;gameClock=std::chrono::steady_clock::now();
  7 | }
  8 | void CEntornVGIView::OnGameDemo() {
  9 |     demoMode=true;static_cast<CMainFrame*>(AfxGetMainWnd())-
    | >SetDemoLayout(true);ClearGameInput();SetFocus();Invalidate(FALSE);
 10 | }
 11 | void CEntornVGIView::OnGameEnvironment() {
 12 |     if(!GLEW_VERSION_4_6){AfxMessageBox(L"El entorno docente utiliza shaders OpenGL 4.6. La demo 3D
    | sigue disponible en OpenGL 3.3.");return;}
 13 |     if(game.phase==galaga::Phase::Playing)game.pause();
 14 |     demoMode=false;static_cast<CMainFrame*>(AfxGetMainWnd())-
    | >SetDemoLayout(false);ClearGameInput();SetFocus();Invalidate(FALSE);
 15 | }
 16 | void CEntornVGIView::OnKillFocus(CWnd* next) {
 17 |     if(game.phase==galaga::Phase::Playing)game.pause();
 18 |     ClearGameInput();CView::OnKillFocus(next);Invalidate(FALSE);
 19 | }
 20 | BOOL CEntornVGIView::PreTranslateMessage(MSG* message) {
 21 |     if(demoMode && (message->message==WM_KEYDOWN||message->message==WM_KEYUP)) {
 22 |         UINT key=static_cast<UINT>(message->wParam);
 23 |         if(key=='A'||key=='D'||key==VK_LEFT||key==VK_RIGHT||key==VK_SPACE||key=='P'||key=='R'||
 24 |             key=='1'||key=='2'||key=='3'||key==VK_RETURN||key==VK_ESCAPE) {
 25 |             if(message->message==WM_KEYDOWN)OnKeyDown(key,1,UINT(message->lParam>>16));
 26 |             else OnKeyUp(key,1,UINT(message->lParam>>16));
 27 |             return TRUE;
 28 |         }
 29 |     }
 30 |     return CView::PreTranslateMessage(message);
 31 | }
 32 | void CEntornVGIView::GameKeyDown(UINT key,UINT flags) {
 33 |     if(key>=gameKeys.size())return;
 34 |     bool fresh=(flags&0x4000)==0 && !gameKeys[key];
 35 |     if(fresh && key=='P') {game.pause();ClearGameInput();}
 36 |     if(fresh && key>='1' && key<='3') game.selectShip(int(key-'1'));
 37 |     if(fresh && key==VK_ESCAPE) {
 38 |         if(game.phase==galaga::Phase::Playing) game.pause();
 39 |         else if(game.phase!=galaga::Phase::Menu) game.returnToMenu();
 40 |         ClearGameInput();
 41 |     }
 42 |     if(fresh && ((key==VK_RETURN&&game.phase==galaga::Phase::Menu)||
 43 |         (key=='R'&&(game.phase==galaga::Phase::Victory||game.phase==galaga::Phase::Defeat)))) {
 44 |         game.reset();ClearGameInput();
 45 |     }
 46 |     gameKeys[key]=true;Invalidate(FALSE);
 47 | }
 48 | void CEntornVGIView::GameTick() {
 49 |     auto now=std::chrono::steady_clock::now();
 50 |     double elapsed=std::chrono::duration<double>(now-gameClock).count();gameClock=now;
 51 |     if(!demoMode||game.phase!=galaga::Phase::Playing) {gameAccumulator=0;return;}
 52 |     // An inactive app or an inactive MDI document must never keep simulating.
 53 |     if(GetForegroundWindow()!=AfxGetMainWnd() || GetFocus()!=this) {
 54 |         game.pause();ClearGameInput();Invalidate(FALSE);return;
 55 |     }
 56 |     gameAccumulator+=std::min(elapsed,.1);
```

```
57 |     galaga::Input
   | input{gameKeys['A']||gameKeys[VK_LEFT],gameKeys['D']||gameKeys[VK_RIGHT],gameKeys[VK_SPACE]};
58 |     int steps=0;
59 |     while(gameAccumulator>=galaga::Game3D::Step && steps++<5) {
60 |         game.update(input);gameAccumulator-=galaga::Game3D::Step;
61 |         if(game.phase!=galaga::Phase::Playing){gameAccumulator=0;break;}
62 |     }
63 |     if(gameAccumulator>=galaga::Game3D::Step)gameAccumulator=std::fmod(gameAccumulator,double(galaga
   | ::Game3D::Step));
64 |     Invalidate(FALSE);
65 | }
```

Código Mesh3D.h

Entrega_Amin_VGI\entornsGrafics\EntornVGI\Mesh3D.h

```
 1 | #pragma once
 2 | #include <glm/glm.hpp>
 3 | #include <cstddef>
 4 | #include <vector>
 5 |
 6 | namespace galaga {
 7 |
 8 | // The shader uses this same interleaved layout: position, normal, UV, colour.
 9 | struct MeshVertex {
10 |     glm::vec3 position;
11 |     glm::vec3 normal;
12 |     glm::vec2 uv;
13 |     glm::vec4 color;
14 | };
15 |
16 | class Mesh3D {
17 | public:
18 |     Mesh3D() = default;
19 |     Mesh3D(const Mesh3D&) = delete;
20 |     Mesh3D& operator=(const Mesh3D&) = delete;
21 |     void upload(const std::vector<MeshVertex>& vertices);
22 |     void draw() const;
23 |     // Inspects actual GPU storage and all four VAO attributes; restores bindings.
24 |     bool verifyGpu() const;
25 |     // Call while the owning OpenGL context is current.
26 |     void release();
27 |     std::size_t vertexCount() const { return count; }
28 |     bool valid() const { return vao != 0 && vbo != 0 && count != 0; }
29 | private:
30 |     unsigned int vao = 0, vbo = 0;
31 |     std::size_t count = 0;
32 | };
33 |
34 | class ModelLibrary {
35 | public:
36 |     Mesh3D ships[3];       // Flecha, Delta, Doble Motor; noses point to +Y.
37 |     Mesh3D enemies[3];     // Abeja, Mariposa, Jefe; heads point to -Y.
38 |     Mesh3D shot, enemyShot, particle;
39 |     void initialize();
40 |     void release();
41 |     bool verifyGpu() const;
42 |     unsigned int uploadCount() const { return uploads; }
43 |     std::size_t vertexCount() const;
44 | private:
45 |     unsigned int uploads = 0;
46 | };
47 |
48 | }
```

Código Mesh3D.cpp

Entrega_Amin_VGI\entornsGrafics\EntornVGI\Mesh3D.cpp

```
#include "pch.h"
#include "Mesh3D.h"
#include <algorithm>
#include <cmath>
#include <limits>
#include <stdexcept>
#include <type_traits>

namespace galaga {
namespace {
using V2 = glm::vec2;
using V3 = glm::vec3;
using Color = glm::vec4;
using Vertices = std::vector<MeshVertex>;
const Color white(.94f, .96f, 1.f, 1.f), blue(.13f, .37f, .96f, 1.f);
const Color red(.96f, .12f, .18f, 1.f), yellow(1.f, .79f, .12f, 1.f);
const Color green(.26f, .86f, .37f, 1.f), purple(.64f, .24f, .89f, 1.f);
const Color metal(.22f, .28f, .40f, 1.f), dark(.025f, .045f, .09f, 1.f);
const Color glass(.24f, .88f, 1.f, 1.f), flame(1.f, .46f, .12f, 1.f);

void triangle(Vertices& out, V3 a, V3 b, V3 c, Color color) {
    V3 n = glm::cross(b - a, c - a);
    const float length = glm::length(n);
    if (length < 1e-8f) return;
    n /= length;
    for (V3 p : {a, b, c}) out.push_back({p, n, V2(p.x + .5f, p.y + .5f), color});
}

void quad(Vertices& out, V3 a, V3 b, V3 c, V3 d, Color color) {
    triangle(out, a, b, c, color);
    triangle(out, a, c, d, color);
}

// A closed convex hull with a raised inset face, bevel and underside.
// All polygons below are convex; use separate pieces for concave silhouettes.
void hull(Vertices& out, std::vector<V2> polygon, float bottom, float top,
          Color color, float inset = .78f) {
    if (polygon.size() < 3) return;
    float area = 0;
    V2 center(0.f);
    for (std::size_t i = 0; i < polygon.size(); ++i) {
        const V2 a = polygon[i], b = polygon[(i + 1) % polygon.size()];
        area += a.x * b.y - b.x * a.y;
        center += a;
    }
    if (area < 0) std::reverse(polygon.begin(), polygon.end());
    center /= static_cast<float>(polygon.size());
    const float shoulder = bottom + (top - bottom) * .35f;
    const Color edge(color.r * .82f, color.g * .82f, color.b * .82f, color.a);
    for (std::size_t i = 0; i < polygon.size(); ++i) {
        const V2 a = polygon[i], b = polygon[(i + 1) % polygon.size()];
        const V2 ai = center + (a - center) * inset, bi = center + (b - center) * inset;
        triangle(out, V3(center, top), V3(ai, top), V3(bi, top), color);
        quad(out, V3(a, shoulder), V3(b, shoulder), V3(bi, top), V3(ai, top), color);
        quad(out, V3(a, bottom), V3(b, bottom), V3(b, shoulder), V3(a, shoulder), edge);
        triangle(out, V3(center, bottom), V3(b, bottom), V3(a, bottom), edge);
    }
}

void box(Vertices& out, V3 center, V3 half, Color color) {
    const V3 a = center - half, b = center + half;
    quad(out, {a.x,a.y,b.z}, {b.x,a.y,b.z}, {b.x,b.y,b.z}, {a.x,b.y,b.z}, color);
    quad(out, {b.x,a.y,a.z}, {a.x,a.y,a.z}, {a.x,b.y,a.z}, {b.x,b.y,a.z}, color);
    quad(out, {b.x,a.y,b.z}, {b.x,a.y,a.z}, {b.x,b.y,a.z}, {b.x,b.y,b.z}, color);
```

```
    quad(out, {a.x,a.y,a.z}, {a.x,a.y,b.z}, {a.x,b.y,b.z}, {a.x,b.y,a.z}, color);
    quad(out, {a.x,b.y,b.z}, {b.x,b.y,b.z}, {b.x,b.y,a.z}, {a.x,b.y,a.z}, color);
    quad(out, {a.x,a.y,a.z}, {b.x,a.y,a.z}, {b.x,a.y,b.z}, {a.x,a.y,b.z}, color);
}

// Six-sided motor, long axis Y. Flat-shaded faces show its real thickness.
void engine(Vertices& out, float x, float rear, float front, float radius, Color color) {
    const float tau = 6.28318530718f;
    for (int i = 0; i < 6; ++i) {
        const float a = tau * i / 6.f, b = tau * (i + 1) / 6.f;
        V3 ar(x + radius * std::cos(a), rear, radius * std::sin(a) - .04f);
        V3 br(x + radius * std::cos(b), rear, radius * std::sin(b) - .04f);
        V3 af(ar.x, front, ar.z), bf(br.x, front, br.z);
        quad(out, ar, af, bf, br, color);
        triangle(out, {x, front, -.04f}, bf, af, color);
        triangle(out, {x, rear, -.04f}, ar, br, flame);
    }
}

void cockpit(Vertices& out, float x, float y, float width, float length, float z) {
    hull(out, {{x-width,y-length}, {x+width,y-length}, {x+width*.65f,y+length},
               {x,y+length*1.22f}, {x-width*.65f,y+length}}, z, z + .065f, glass, .65f);
}

Vertices flecha() {
    Vertices out;
    for (float side : {-1.f, 1.f}) {
        hull(out, {{side*.09f,.34f}, {side*.58f,-.34f}, {side*.49f,-.46f},
                   {side*.08f,-.28f}}, -.075f, .075f, white);
        hull(out, {{side*.31f,-.12f}, {side*.49f,-.32f}, {side*.45f,-.37f},
                   {side*.27f,-.19f}}, .077f, .095f, red, .88f);
        engine(out, side * .29f, -.51f, -.29f, .074f, metal);
    }
    hull(out, {{0.f,.72f}, {-.135f,-.45f}, {.135f,-.45f}}, -.13f, .17f, blue, .76f);
    hull(out, {{0.f,.70f}, {-.035f,.43f}, {.035f,.43f}}, .06f, .19f, white);
    cockpit(out, 0, .16f, .065f, .16f, .18f);
    return out;
}

Vertices delta() {
    Vertices out;
    hull(out, {{0.f,.70f}, {-.59f,-.45f}, {.59f,-.45f}}, -.10f, .095f, red, .84f);
    for (float side : {-1.f, 1.f}) {
        hull(out, {{side*.22f,-.10f}, {side*.47f,-.36f}, {side*.20f,-.31f}}, .10f, .13f, white);
        engine(out, side * .23f, -.51f, -.33f, .07f, metal);
    }
    hull(out, {{0.f,.63f}, {-.10f,-.43f}, {.10f,-.43f}}, .05f, .19f, red, .75f);
    cockpit(out, 0.f, .10f, .06f, .17f, .19f);
    return out;
}

Vertices twinEngine() {
    Vertices out;
    box(out, {0.f,-.12f,-.08f}, {.39f,.065f,.06f}, metal);
    for (float side : {-1.f, 1.f}) {
        const float x = side * .40f;
        hull(out, {{x-.12f,-.48f}, {x+.12f,-.48f}, {x+.12f,.39f},
                   {x+.055f,.56f}, {x-.055f,.56f}, {x-.12f,.39f}}, -.12f, .14f, green);
        hull(out, {{x-.035f,-.34f}, {x+.035f,-.34f}, {x+.035f,.29f},
                   {x-.035f,.29f}}, .145f, .157f, white, .9f);
        engine(out, x, -.59f, -.42f, .085f, metal);
    }
    hull(out, {{-.15f,-.35f}, {.15f,-.35f}, {.14f,.40f}, {0.f,.58f}, {-.14f,.40f}},
         -.13f, .18f, white);
    hull(out, {{-.085f,-.34f}, {.085f,-.34f}, {.075f,.36f}, {0.f,.53f}, {-.075f,.36f}},
         .15f, .19f, green);
    cockpit(out, 0.f, .09f, .06f, .15f, .19f);
    return out;
}

void eyes(Vertices& out, float separation, float y, float z) {
```

```
136 |     for (float side : {-1.f, 1.f}) {
137 |         box(out, {side*separation,y,z}, {.046f,.052f,.017f}, dark);
138 |         box(out, {side*separation,y-.015f,z+.02f}, {.018f,.023f,.01f}, white);
139 |     }
140 | }
141 |
142 | Vertices bee() {
143 |     Vertices out;
144 |     for (float side : {-1.f, 1.f}) {
145 |         hull(out, {{side*.10f,.20f}, {side*.43f,.37f}, {side*.54f,.12f},
146 |                    {side*.30f,-.22f}, {side*.13f,-.15f}}, -.09f, .10f, blue);
147 |         hull(out, {{side*.30f,.18f}, {side*.45f,.29f}, {side*.50f,.12f},
148 |                    {side*.34f,-.02f}}, .10f, .135f, yellow);
149 |         hull(out, {{side*.12f,-.17f}, {side*.27f,-.37f}, {side*.23f,-.43f},
150 |                    {side*.08f,-.29f}}, -.02f, .085f, red);
151 |     }
152 |     hull(out, {{0.f,-.47f}, {.18f,-.27f}, {.13f,.31f}, {0.f,.43f},
153 |                {-.13f,.31f}, {-.18f,-.27f}}, -.15f, .18f, yellow);
154 |     box(out, {0.f,.12f,.185f}, {.105f,.045f,.017f}, blue);
155 |     box(out, {0.f,.29f,.165f}, {.072f,.034f,.016f}, blue);
156 |     eyes(out, .077f, -.23f, .195f);
157 |     return out;
158 | }
159 |
160 | Vertices butterfly() {
161 |     Vertices out;
162 |     for (float side : {-1.f, 1.f}) {
163 |         hull(out, {{side*.10f,.18f}, {side*.40f,.46f}, {side*.53f,.30f},
164 |                    {side*.49f,-.17f}, {side*.17f,-.31f}}, -.10f, .12f, red);
165 |         hull(out, {{side*.25f,.03f}, {side*.43f,.28f}, {side*.43f,-.08f},
166 |                    {side*.29f,-.17f}}, .12f, .15f, white);
167 |         hull(out, {{side*.08f,-.21f}, {side*.21f,-.45f}, {side*.14f,-.49f},
168 |                    {side*.04f,-.29f}}, -.04f, .06f, blue);
169 |     }
170 |     hull(out, {{0.f,-.43f}, {.16f,-.23f}, {.10f,.32f}, {0.f,.43f},
171 |                {-.10f,.32f}, {-.16f,-.23f}}, -.15f, .19f, white);
172 |     hull(out, {{-.13f,-.17f}, {.13f,-.17f}, {.10f,.04f}, {-.10f,.04f}}, .19f, .205f, red);
173 |     eyes(out, .067f, -.24f, .215f);
174 |     return out;
175 | }
176 |
177 | Vertices boss() {
178 |     Vertices out;
179 |     for (float side : {-1.f, 1.f}) {
180 |         hull(out, {{side*.12f,.15f}, {side*.42f,.43f}, {side*.56f,.28f},
181 |                    {side*.52f,-.19f}, {side*.20f,-.35f}}, -.10f, .14f, purple);
182 |         hull(out, {{side*.29f,-.13f}, {side*.48f,-.08f}, {side*.45f,.21f},
183 |                    {side*.29f,.10f}}, .14f, .19f, green);
184 |         hull(out, {{side*.09f,-.23f}, {side*.24f,-.46f}, {side*.14f,-.50f},
185 |                    {side*.05f,-.31f}}, -.01f, .10f, yellow);
186 |     }
187 |     hull(out, {{-.19f,-.24f}, {0.f,-.45f}, {.19f,-.24f}, {.18f,.23f},
188 |                {0.f,.43f}, {-.18f,.23f}}, -.16f, .22f, green);
189 |     hull(out, {{0.f,.35f}, {-.10f,.07f}, {.10f,.07f}}, .22f, .24f, yellow);
190 |     eyes(out, .085f, -.19f, .235f);
191 |     return out;
192 | }
193 |
194 | Vertices projectile(bool hostile) {
195 |     Vertices out;
196 |     const Color color = hostile ? Color(1.f,.22f,.08f,1.f) : Color(.25f,.85f,1.f,1.f);
197 |     hull(out, {{0.f,.25f}, {-.045f,.14f}, {-.045f,-.21f}, {0.f,-.25f},
198 |                {.045f,-.21f}, {.045f,.14f}}, -.04f, .04f, color, .7f);
199 |     box(out, {0.f,0.f,.043f}, {.016f,.17f,.012f}, white);
200 |     return out;
201 | }
202 | }
203 |
204 | void Mesh3D::upload(const std::vector<MeshVertex>& vertices) {
205 |     static_assert(std::is_standard_layout<MeshVertex>::value, "Vertex offsets must be stable");
206 |     if (vertices.empty() || vertices.size() % 3 != 0 ||
```

```
207 |         vertices.size() > static_cast<std::size_t>((std::numeric_limits<GLsizei>::max)()))
208 |         throw std::runtime_error("Malla 3D sin triangulos validos.");
209 |     for (const MeshVertex& v : vertices) {
210 |         for (int k = 0; k < 3; ++k)
211 |             if (!std::isfinite(v.position[k]) || !std::isfinite(v.normal[k]))
212 |                 throw std::runtime_error("La malla 3D contiene valores no finitos.");
213 |         if (glm::dot(v.normal,v.normal) < .9f)
214 |             throw std::runtime_error("La malla 3D contiene una normal degenerada.");
215 |     }
216 |     release();
217 |     glGenVertexArrays(1, &vao);
218 |     glGenBuffers(1, &vbo);
219 |     glBindVertexArray(vao);
220 |     glBindBuffer(GL_ARRAY_BUFFER, vbo);
221 |     glBufferData(GL_ARRAY_BUFFER, static_cast<GLsizeiptr>(vertices.size() * sizeof(MeshVertex)),
222 |                  vertices.data(), GL_STATIC_DRAW);
223 |     glEnableVertexAttribArray(0);
224 |     glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, sizeof(MeshVertex),
225 |                           reinterpret_cast<void*>(offsetof(MeshVertex,position)));
226 |     glEnableVertexAttribArray(1);
227 |     glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, sizeof(MeshVertex),
228 |                           reinterpret_cast<void*>(offsetof(MeshVertex,normal)));
229 |     glEnableVertexAttribArray(2);
230 |     glVertexAttribPointer(2, 2, GL_FLOAT, GL_FALSE, sizeof(MeshVertex),
231 |                           reinterpret_cast<void*>(offsetof(MeshVertex,uv)));
232 |     glEnableVertexAttribArray(3);
233 |     glVertexAttribPointer(3, 4, GL_FLOAT, GL_FALSE, sizeof(MeshVertex),
234 |                           reinterpret_cast<void*>(offsetof(MeshVertex,color)));
235 |     glBindVertexArray(0);
236 |     glBindBuffer(GL_ARRAY_BUFFER,0);
237 |     if (!vao || !vbo || glGetError() != GL_NO_ERROR) {
238 |         release();
239 |         throw std::runtime_error("No se ha podido cargar una malla 3D en VAO/VBO.");
240 |     }
241 |     count = vertices.size();
242 | }
243 |
244 | void Mesh3D::draw() const {
245 |     if (!valid()) return;
246 |     glBindVertexArray(vao);
247 |     glDrawArrays(GL_TRIANGLES, 0, static_cast<GLsizei>(count));
248 | }
249 |
250 | bool Mesh3D::verifyGpu() const {
251 |     if (!valid() || !glIsVertexArray || !glIsBuffer ||
252 |         glIsVertexArray(vao) != GL_TRUE || glIsBuffer(vbo) != GL_TRUE) return false;
253 |
254 |     GLint previousVao = 0, previousBuffer = 0;
255 |     glGetIntegerv(GL_VERTEX_ARRAY_BINDING, &previousVao);
256 |     glGetIntegerv(GL_ARRAY_BUFFER_BINDING, &previousBuffer);
257 |     glBindVertexArray(vao);
258 |     glBindBuffer(GL_ARRAY_BUFFER, vbo);
259 |
260 |     GLint bytes = 0, usage = 0;
261 |     glGetBufferParameteriv(GL_ARRAY_BUFFER, GL_BUFFER_SIZE, &bytes);
262 |     glGetBufferParameteriv(GL_ARRAY_BUFFER, GL_BUFFER_USAGE, &usage);
263 |     bool correct = bytes > 0 && static_cast<std::size_t>(bytes) == count * sizeof(MeshVertex)
264 |                    && usage == GL_STATIC_DRAW;
265 |     const GLint sizes[] = {3, 3, 2, 4};
266 |     const std::size_t offsets[] = {offsetof(MeshVertex, position), offsetof(MeshVertex, normal),
267 |                                    offsetof(MeshVertex, uv), offsetof(MeshVertex, color)};
268 |     for (GLuint index = 0; index < 4; ++index) {
269 |         GLint enabled = 0, size = 0, type = 0, binding = 0, stride = 0, normalized = 0;
270 |         void* offset = nullptr;
271 |         glGetVertexAttribiv(index, GL_VERTEX_ATTRIB_ARRAY_ENABLED, &enabled);
272 |         glGetVertexAttribiv(index, GL_VERTEX_ATTRIB_ARRAY_SIZE, &size);
273 |         glGetVertexAttribiv(index, GL_VERTEX_ATTRIB_ARRAY_TYPE, &type);
274 |         glGetVertexAttribiv(index, GL_VERTEX_ATTRIB_ARRAY_BUFFER_BINDING, &binding);
275 |         glGetVertexAttribiv(index, GL_VERTEX_ATTRIB_ARRAY_STRIDE, &stride);
276 |         glGetVertexAttribiv(index, GL_VERTEX_ATTRIB_ARRAY_NORMALIZED, &normalized);
277 |         glGetVertexAttribPointerv(index, GL_VERTEX_ATTRIB_ARRAY_POINTER, &offset);
```

```
        if (enabled != GL_TRUE || size != sizes[index] || type != GL_FLOAT ||
            static_cast<GLuint>(binding) != vbo || stride != sizeof(MeshVertex) ||
            normalized != GL_FALSE || reinterpret_cast<std::size_t>(offset) != offsets[index])
            correct = false;
    }

    glBindVertexArray(static_cast<GLuint>(previousVao));
    glBindBuffer(GL_ARRAY_BUFFER, static_cast<GLuint>(previousBuffer));
    const GLenum error = glGetError();
    return correct && error == GL_NO_ERROR;
}

void Mesh3D::release() {
    if (vao) glDeleteVertexArrays(1, &vao);
    if (vbo) glDeleteBuffers(1, &vbo);
    vao = vbo = 0;
    count = 0;
}

void ModelLibrary::initialize() {
    if (uploads == 9) return;
    release();
    try {
        auto upload = [this](Mesh3D& mesh, const Vertices& vertices) {
            mesh.upload(vertices);
            ++uploads;
        };
        upload(ships[0],flecha());
        upload(ships[1],delta());
        upload(ships[2],twinEngine());
        upload(enemies[0],bee());
        upload(enemies[1],butterfly());
        upload(enemies[2],boss());
        upload(shot,projectile(false));
        upload(enemyShot,projectile(true));
        Vertices cube;
        box(cube,V3(0.f),V3(.5f),white);
        upload(particle,cube);
    } catch (...) {
        release();
        throw;
    }
}

void ModelLibrary::release() {
    for (auto& mesh : ships) mesh.release();
    for (auto& mesh : enemies) mesh.release();
    shot.release();
    enemyShot.release();
    particle.release();
    uploads = 0;
}

bool ModelLibrary::verifyGpu() const {
    bool correct = uploads == 9;
    // Check every mesh even if an earlier one fails, including its GL errors.
    for (const auto& mesh : ships) if (!mesh.verifyGpu()) correct = false;
    for (const auto& mesh : enemies) if (!mesh.verifyGpu()) correct = false;
    if (!shot.verifyGpu()) correct = false;
    if (!enemyShot.verifyGpu()) correct = false;
    if (!particle.verifyGpu()) correct = false;
    const GLenum error = glGetError();
    return correct && error == GL_NO_ERROR;
}

std::size_t ModelLibrary::vertexCount() const {
    std::size_t total = shot.vertexCount() + enemyShot.vertexCount() + particle.vertexCount();
    for (const auto& mesh : ships) total += mesh.vertexCount();
    for (const auto& mesh : enemies) total += mesh.vertexCount();
    return total;
}
```

```
349 | }
```

Código Renderer3D.h

Entrega_Amin_VGI\entornsGrafics\EntornVGI\Renderer3D.h

```
  1 | #pragma once
  2 | #include "Mesh3D.h"
  3 | #include "Renderer2D.h"
  4 | #include <ostream>
  5 | namespace galaga {
  6 | class Renderer3D {
  7 | public:
  8 |     void initialize();
  9 |     void release();
 10 |     void draw(const Game3D&,int width,int height);
 11 |     bool modern() const { return program!=0; }
 12 |     bool verifyGpu(std::ostream&) const;
 13 |     bool verifyFraming(const Game3D&,std::ostream&);
 14 |     bool verifyLighting(std::ostream&);
 15 |     unsigned int uploadCount() const {return models.uploadCount();}
 16 |     unsigned int drawCount() const {return draws;}
 17 |     // Same geometry and shader, different camera: verifies real solid models.
 18 |     void modelGallery(int width,int height,float angle);
 19 | private:
 20 |     Renderer2D hud;
 21 |     ModelLibrary models;
 22 |     unsigned int program=0,draws=0;
 23 |     int modelLocation=-1,viewLocation=-1,projectionLocation=-1,normalLocation=-1,tintLocation=-
    | 1,eyeLocation=-1;
 24 |     int
    | emissionLocation=-1,ambientLocation=-1,diffuseLocation=-1,specularLocation=-1,shininessLocation=-1;
 25 |     int
    | lightPositionLocation=-1,lightDiffuseLocation=-1,lightSpecularLocation=-1,ambientLightLocation=-1;
 26 |     glm::mat4 view{1},projection{1};
 27 |     glm::vec3 eye{0,-4.5f,22};
 28 |     void begin3D();
 29 |     void camera(bool menu,float playerX=0);
 30 |     void instance(const Mesh3D&,const glm::mat4&,glm::vec4 tint=glm::vec4(1),float emission=0);
 31 | };
 32 | }
```

Código Renderer3D.cpp

Entrega_Amin_VGI\entornsGrafics\EntornVGI\Renderer3D.cpp

```
  1 | #include "pch.h"
  2 | #include "Renderer3D.h"
  3 | #include <glm/gtc/type_ptr.hpp>
  4 | #include <glm/gtc/matrix_transform.hpp>
  5 | #include <glm/gtx/quaternion.hpp>
  6 | #include <algorithm>
  7 | #include <cmath>
  8 | #include <fstream>
  9 | #include <iterator>
 10 | #include <stdexcept>
 11 | namespace galaga {
 12 | namespace {
 13 | GLuint compileFile(GLenum type,const wchar_t* name){
 14 |     const auto path=Renderer2D::executableDirectory()/L"shaders"/name;
 15 |     std::ifstream input(path,std::ios::binary);
 16 |     std::string source((std::istreambuf_iterator<char>(input)),{});
 17 |     if(source.empty())throw std::runtime_error("Falta un shader Galaga3D junto al ejecutable.");
 18 |     const char* p=source.c_str();GLuint
    | sh=glCreateShader(type);glShaderSource(sh,1,&p,nullptr);glCompileShader(sh);
 19 |     GLint ok=0;glGetShaderiv(sh,GL_COMPILE_STATUS,&ok);
 20 |     if(!ok){char log[4096]{};glGetShaderInfoLog(sh,sizeof(log),nullptr,log);glDeleteShader(sh);throw
```

```
|  std::runtime_error(log);}
21 |     return sh;
22 | }
23 | glm::mat4 transform(glm::vec3 position,glm::vec3 scale=glm::vec3(1),float angle=0){
24 |     // Column vectors: scale locally, rotate locally, translate in world space.
25 |     return glm::translate(glm::mat4(1),position)*glm::rotate(glm::mat4(1),angle,glm::vec3(0,0,1))*gl
|  m::scale(glm::mat4(1),scale);
26 | }
27 | }
28 | void Renderer3D::initialize(){
29 |     if(!GLEW_VERSION_3_3)throw std::runtime_error("Falta el motor grafico incluido. Ejecuta build-
|  demo.cmd o usa la carpeta completa dist/Galaga3D, con opengl32.dll.");
30 |     GLuint v=0,f=0;
31 |     try{
32 |
|  v=compileFile(GL_VERTEX_SHADER,L"galaga3d.vert");f=compileFile(GL_FRAGMENT_SHADER,L"galaga3d.frag");
33 |         program=glCreateProgram();glAttachShader(program,v);glAttachShader(program,f);glLinkProgram(
|  program);
34 |         glDeleteShader(v);v=0;glDeleteShader(f);f=0;
35 |         GLint ok=0;glGetProgramiv(program,GL_LINK_STATUS,&ok);
36 |         if(!ok){char log[4096]{};glGetProgramInfoLog(program,sizeof(log),nullptr,log);throw
|  std::runtime_error(log);}
37 |         auto uniform=[&](const char* name){int loc=glGetUniformLocation(program,name);if(loc<0)throw
|   std::runtime_error(std::string("Uniform no activo: ")+name);return loc;};
38 |         modelLocation=uniform("model");viewLocation=uniform("view");projectionLocation=uniform("proj
|  ection");
39 |
|  normalLocation=uniform("normalMatrix");tintLocation=uniform("tint");eyeLocation=uniform("eye");
40 |         emissionLocation=uniform("material.emission");
41 |         ambientLocation=uniform("material.ambient");
42 |         diffuseLocation=uniform("material.diffuse");
43 |         specularLocation=uniform("material.specular");
44 |         shininessLocation=uniform("material.shininess");
45 |         lightPositionLocation=uniform("light.position");
46 |         lightDiffuseLocation=uniform("light.diffuse");
47 |         lightSpecularLocation=uniform("light.specular");
48 |         ambientLightLocation=uniform("ambientLight");
49 |         models.initialize();hud.initialize();
50 |     }catch(...){if(v)glDeleteShader(v);if(f)glDeleteShader(f);release();throw;}
51 | }
52 | void
|  Renderer3D::release(){hud.release();models.release();if(program)glDeleteProgram(program);program=0;}
53 | void Renderer3D::begin3D(){
54 |     glDisable(GL_BLEND);glEnable(GL_DEPTH_TEST);glDepthFunc(GL_LESS);glDepthMask(GL_TRUE);
55 |     glEnable(GL_CULL_FACE);glCullFace(GL_BACK);glFrontFace(GL_CCW);glUseProgram(program);draws=0;
56 |     glUniformMatrix4fv(viewLocation,1,GL_FALSE,glm::value_ptr(view));
57 |     glUniformMatrix4fv(projectionLocation,1,GL_FALSE,glm::value_ptr(projection));
58 |     glUniform3fv(eyeLocation,1,glm::value_ptr(eye));
59 |     // One directional source, using the material/light uniforms from GL-6.
60 |     glUniform4f(lightPositionLocation,-.45f,.7f,1.1f,0.f);
61 |     glUniform3f(lightDiffuseLocation,1.f,1.f,1.f);
62 |     glUniform3f(lightSpecularLocation,1.f,1.f,1.f);
63 |     glUniform3f(ambientLightLocation,1.f,1.f,1.f);
64 | }
65 | void Renderer3D::camera(bool menu,float playerX){
66 |     if(menu){eye={0,0,12};view=glm::lookAt(eye,glm::vec3(0),glm::vec3(0,1,0));projection=glm::ortho(
|  -6.4f,6.4f,-3.6f,3.6f,.1f,40.f);}
67 |     else{
68 |         // Behind the ship: +Y is forward and +Z is up in the game world.
69 |         const float follow=playerX*.15f;
70 |         eye={follow,-16,6};
71 |         view=glm::lookAt(eye,glm::vec3(follow,8,0),glm::vec3(0,0,1));
72 |         projection=glm::perspective(glm::radians(38.f),1280.f/720.f,.1f,100.f);
73 |     }
74 | }
75 | void Renderer3D::instance(const Mesh3D& mesh,const glm::mat4& model,glm::vec4 tint,float emission){
76 |     // No vertex uploads here. The same VAO/VBO is reused by every instance.
77 |     const glm::mat3 normal=glm::transpose(glm::inverse(glm::mat3(model)));
78 |     glUniformMatrix4fv(modelLocation,1,GL_FALSE,glm::value_ptr(model));
79 |     glUniformMatrix3fv(normalLocation,1,GL_FALSE,glm::value_ptr(normal));
80 |     glUniform4fv(tintLocation,1,glm::value_ptr(tint));
```

```
 81 |     const float emissive = std::clamp(emission,0.f,1.f);
 82 |     const float reflected = 1.f-emissive;
 83 |     glUniform3f(emissionLocation,emissive,emissive,emissive);
 84 |     glUniform3f(ambientLocation,.37f*reflected,.37f*reflected,.37f*reflected);
 85 |     glUniform3f(diffuseLocation,.63f*reflected,.63f*reflected,.63f*reflected);
 86 |     glUniform3f(specularLocation,.16f*reflected,.16f*reflected,.16f*reflected);
 87 |     glUniform1f(shininessLocation,32.f);
 88 |     mesh.draw();++draws;
 89 | }
 90 | void Renderer3D::draw(const Game3D& g,int width,int height){
 91 |     if(width<=0||height<=0||!program)return;
 92 |     hud.background(g,width,height);camera(g.phase==Phase::Menu,g.player.x);begin3D();
 93 |     if(g.phase==Phase::Menu){
 94 |         for(int i=0;i<3;++i){
 95 |             glm::mat4 model=glm::translate(glm::mat4(1),glm::vec3((i-1)*3.f,-.08f,0));
 96 |             model=glm::rotate(model,glm::radians(-18.f),glm::vec3(1,0,0));
 97 |             model=glm::rotate(model,glm::radians(i==g.selectedShip?20.f:-12.f),glm::vec3(0,1,0));
 98 |             model=glm::scale(model,glm::vec3(1.35f));
 99 |             instance(models.ships[i],model);
100 |         }
101 |     }else{
102 |         for(const auto& e:g.enemies)if(e.alive&&e.timer>=0){
103 |             auto model=transform({e.pos.x,e.pos.y,e.z},glm::vec3(Game3D::EnemyScale));
104 |             // A fixed pitch keeps the silhouette readable from the chase camera.
105 |             model=glm::rotate(model,glm::radians(35.f),glm::vec3(1,0,0));
106 |             instance(models.enemies[e.type%2],model);
107 |         }
108 |         for(const auto& b:g.bullets){
109 |             auto model=transform({b.pos.x,b.pos.y,b.z});
110 |             const glm::vec3 direction =
    | glm::normalize(glm::vec3(b.velocityX,b.forwardSpeed(),b.velocityZ));
111 |             model *= glm::toMat4(glm::rotation(glm::vec3(0,1,0),direction));
112 |             instance(b.ally?models.shot:models.enemyShot,model,glm::vec4(1),1.f);
113 |         }
114 |         if(g.respawn==0)
115 |             instance(models.ships[g.selectedShip],transform({g.player.x,g.player.y,0}));
116 |     }
117 |     glBindVertexArray(0);
118 |     Vec2 targetScreen{-1,-1};
119 |     float targetRadius = 0;
120 |     if (g.phase == Phase::Playing && g.respawn == 0) {
121 |         if (const Enemy* target = g.aimTarget()) {
122 |             const glm::vec4 clip = projection * view *
    | glm::vec4(target->pos.x,target->pos.y,target->z,1);
123 |             if (clip.w > 0) {
124 |                 targetScreen = {(clip.x/clip.w+1)*640,(1-clip.y/clip.w)*360};
125 |                 targetRadius = 360 * projection[1][1] * Game3D::EnemyHitRadius / clip.w + 5;
126 |             }
127 |         }
128 |     }
129 |     hud.overlay(g,width,height,targetScreen,targetRadius);
130 | }
131 | bool Renderer3D::verifyGpu(std::ostream& report) const{
132 |     GLint linked=0;glGetProgramiv(program,GL_LINK_STATUS,&linked);
133 |     bool ok=linked&&models.verifyGpu()&&models.uploadCount()==9&&modelLocation>=0&&viewLocation>=0&&
    | projectionLocation>=0;
134 |     report<<(ok?"PASS ":"FAIL ")<<"3D GPU meshes: "<<models.uploadCount()<<" static uploads,
    | "<<models.vertexCount()/3
135 |           <<" triangles; VAO attributes 0/1/2/3; model/view/projection uniforms active\n";
136 |     return ok;
137 | }
138 | bool Renderer3D::verifyLighting(std::ostream& report) {
139 |     // The caller supplies a 64x64 RGBA8 framebuffer with a depth attachment.
140 |     // Sample the +Z face of the existing cube under isolated GL-6 terms.
141 |     camera(true);
142 |     glViewport(0,0,64,64);
143 |     begin3D();
144 |     instance(models.particle,glm::scale(glm::mat4(1),glm::vec3(3)));
145 |     struct Sample {
146 |         const char* name;
147 |         float ambient,diffuse,specular,emission,lightZ,expected;
```

```
148 |     };
149 |     const Sample samples[] = {
150 |         {"ambient",.25f,0,0,0,1,.25f},
151 |         {"Lambert front",0,.5f,0,0,1,.5f},
152 |         {"unlit back",0,.5f,.5f,0,-1,0},
153 |         {"Phong reflection",0,0,.4f,0,1,.4f},
154 |         {"emissive",0,0,0,.7f,-1,.7f},
155 |         {"additive terms",.2f,.3f,0,.1f,1,.6f}
156 |     };
157 |     const float base[] = {.94f,.96f,1.f};
158 |     bool correct=true;
159 |     for(const auto& sample:samples) {
160 |         glClearColor(0,0,0,1);
161 |         glClear(GL_COLOR_BUFFER_BIT|GL_DEPTH_BUFFER_BIT);
162 |         glUniform4f(lightPositionLocation,0,0,sample.lightZ,0);
163 |         glUniform3f(ambientLocation,sample.ambient,sample.ambient,sample.ambient);
164 |         glUniform3f(diffuseLocation,sample.diffuse,sample.diffuse,sample.diffuse);
165 |         glUniform3f(specularLocation,sample.specular,sample.specular,sample.specular);
166 |         glUniform3f(emissionLocation,sample.emission,sample.emission,sample.emission);
167 |         models.particle.draw();
168 |         unsigned char pixel[4]{};
169 |         glReadPixels(32,32,1,1,GL_RGBA,GL_UNSIGNED_BYTE,pixel);
170 |         bool matches=true;
171 |         for(int channel=0;channel<3;++channel)
172 |             matches &= std::abs(float(pixel[channel])-255.f*base[channel]*sample.expected)<3.f;
173 |         report<<(matches?"PASS ":"FAIL ")<<"GL-6 shader term "<<sample.name<<"\n";
174 |         correct &= matches;
175 |     }
176 |     glBindVertexArray(0);
177 |     return correct && glGetError()==GL_NO_ERROR;
178 | }
179 | bool Renderer3D::verifyFraming(const Game3D& game,std::ostream& report) {
180 |     camera(false,game.player.x);
181 |     auto project=[&](glm::vec3 point) {
182 |         const glm::vec4 clip=projection*view*glm::vec4(point,1);
183 |         return glm::vec2((clip.x/clip.w+1)*640,(1-clip.y/clip.w)*360);
184 |     };
185 |     float minimumWidth=1280,formationLeft=1280,formationRight=0;
186 |     bool readable=true;
187 |     for(const auto& enemy:game.enemies) {
188 |         if(!enemy.alive)continue;
189 |         const glm::vec3 centre(enemy.pos.x,enemy.pos.y,enemy.z);
190 |         const auto left=project(centre-glm::vec3(Game3D::EnemyHitRadius,0,0));
191 |         const auto right=project(centre+glm::vec3(Game3D::EnemyHitRadius,0,0));
192 |         const auto screen=project(centre);
193 |         minimumWidth=std::min(minimumWidth,right.x-left.x);
194 |         formationLeft=std::min(formationLeft,left.x);
195 |         formationRight=std::max(formationRight,right.x);
196 |         readable &= screen.y>145 && screen.y<480 && left.x>20 && right.x<1260;
197 |     }
198 |     for(float x:{-.65f,.65f})for(float y:{-.72f,.72f})for(float z:{-.26f,.26f}) {
199 |         const auto corner=project(glm::vec3(game.player.x+x,game.player.y+y,z));
200 |         readable &= corner.x>12 && corner.x<1268 && corner.y>480 && corner.y<640;
201 |     }
202 |     readable &= minimumWidth>=34 && formationRight-formationLeft>=400;
203 |     report<<(readable?"PASS ":"FAIL ")<<"framing at player X="<<game.player.x
204 |           <<"; smallest target "<<minimumWidth<<"px; formation width "<<formationRight-
    | formationLeft<<"px\n";
205 |     return readable;
206 | }
207 | void Renderer3D::modelGallery(int width,int height,float angle){
208 |     Game3D menu;hud.background(menu,width,height);camera(true);begin3D();
209 |     for(int i=0;i<6;++i){
210 |         auto model=glm::translate(glm::mat4(1),glm::vec3((i%3-1)*3.4f,i<3?1.5f:-1.5f,0));
211 |         model=glm::rotate(model,glm::radians(-24.f),glm::vec3(1,0,0));
212 |         model=glm::rotate(model,angle,glm::vec3(0,1,0));model=glm::scale(model,glm::vec3(1.6f));
213 |         instance(i<3?models.ships[i]:models.enemies[i-3],model);
214 |     }
215 |     glBindVertexArray(0);glUseProgram(0);
216 | }
217 | }
```

Código Renderer2D.h

Entrega_Amin_VGI\entornsGrafics\EntornVGI\Renderer2D.h

```
#pragma once
#include "Game3D.h"
#include <filesystem>
#include <string>
#include <vector>
namespace galaga {
// Screen-space HUD; all ships and projectiles are solid meshes in Renderer3D.
class Renderer2D {
public:
    void initialize();
    void release();
    void background(const Game3D&,int width,int height);
    void overlay(const Game3D&,int width,int height,Vec2 targetScreen={-1,-1},float targetRadius=0);
    static std::filesystem::path executableDirectory();
private:
    struct Vertex {float x,y,u,v,r,g,b,a;};
    struct Color {float r,g,b,a=1;};
    struct Batch {unsigned int texture;int first,count;};
    unsigned int program=0,vao=0,vbo=0,white=0,font=0;
    unsigned int projectHud[7]{};
    std::vector<Vertex> vertices;
    std::vector<Batch> batches;
    int pendingStart=0;
    void begin(int width,int height);
    void finish();
    void flush(unsigned int texture=0);
    void image(unsigned int texture,float x,float y,float w,float h);
    void quad(float x,float y,float w,float h,Color,float u1=0,float v1=0,float u2=1,float v2=1);
    void border(float x,float y,float w,float h,Color);
    void text(const std::string&,float x,float y,int height,Color,bool center=false);
};
}
```

Código Renderer2D.cpp

Entrega_Amin_VGI\entornsGrafics\EntornVGI\Renderer2D.cpp

```
#include "pch.h"
#include "Renderer2D.h"
#include <gl/SOIL2.h>
#include <algorithm>
#include <array>
#include <cmath>
#include <stdexcept>
namespace galaga {
std::filesystem::path Renderer2D::executableDirectory() {
    wchar_t path[32768]{};GetModuleFileNameW(nullptr,path,32768);
    return std::filesystem::path(path).parent_path();
}
void Renderer2D::initialize() {
    const char* vs=R"(#version 330 core
layout(location=0) in vec2 position;
layout(location=1) in vec2 uv;
layout(location=2) in vec4 color;
out vec2 texCoord;out vec4 tint;
void
main(){gl_Position=vec4(position.x/640.0-1.0,1.0-position.y/360.0,0,1);texCoord=uv;tint=color;})";
    const char* fs=R"(#version 330 core
in vec2 texCoord;in vec4 tint;out vec4 outColor;uniform sampler2D image;
void main(){outColor=texture(image,texCoord)*tint;})";
    auto compile=[](GLenum type,const char* source) {
        GLuint sh=glCreateShader(type);glShaderSource(sh,1,&source,nullptr);glCompileShader(sh);
```

```
25 |         GLint ok=0;glGetShaderiv(sh,GL_COMPILE_STATUS,&ok);
26 |         if(!ok){char log[2048]{};glGetShaderInfoLog(sh,2048,nullptr,log);glDeleteShader(sh);throw
   | std::runtime_error(log);}return sh;
27 |     };
28 |     GLuint v=compile(GL_VERTEX_SHADER,vs),f=compile(GL_FRAGMENT_SHADER,fs);
29 |     program=glCreateProgram();glAttachShader(program,v);glAttachShader(program,f);glLinkProgram(prog
   | ram);
30 |     glDeleteShader(v);glDeleteShader(f);
31 |     GLint ok=0;glGetProgramiv(program,GL_LINK_STATUS,&ok);
32 |     if(!ok)throw std::runtime_error("No se ha podido enlazar el shader HUD.");
33 |     glGenVertexArrays(1,&vao);glGenBuffers(1,&vbo);
34 |     glBindVertexArray(vao);glBindBuffer(GL_ARRAY_BUFFER,vbo);
35 |
   | glEnableVertexAttribArray(0);glVertexAttribPointer(0,2,GL_FLOAT,GL_FALSE,sizeof(Vertex),nullptr);
36 |     glEnableVertexAttribArray(1);glVertexAttribPointer(1,2,GL_FLOAT,GL_FALSE,sizeof(Vertex),(void*)(
   | 2*sizeof(float)));
37 |     glEnableVertexAttribArray(2);glVertexAttribPointer(2,4,GL_FLOAT,GL_FALSE,sizeof(Vertex),(void*)(
   | 4*sizeof(float)));
38 |     auto texture=[](GLuint& id,int w,int h,const unsigned char* data){
39 |         glGenTextures(1,&id);glBindTexture(GL_TEXTURE_2D,id);
40 |         glTexImage2D(GL_TEXTURE_2D,0,GL_RGBA8,w,h,0,GL_RGBA,GL_UNSIGNED_BYTE,data);
41 |         glTexParameteri(GL_TEXTURE_2D,GL_TEXTURE_MIN_FILTER,GL_NEAREST);glTexParameteri(GL_TEXTURE_2
   | D,GL_TEXTURE_MAG_FILTER,GL_NEAREST);
42 |         glTexParameteri(GL_TEXTURE_2D,GL_TEXTURE_WRAP_S,GL_CLAMP_TO_EDGE);glTexParameteri(GL_TEXTURE
   | _2D,GL_TEXTURE_WRAP_T,GL_CLAMP_TO_EDGE);
43 |     };
44 |     unsigned char pixel[]={255,255,255,255};texture(white,1,1,pixel);
45 |     // Original 5x7 glyph atlas generated once. No installed-font dependency.
46 |     const char* glyphs="ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789:/.-+[]!<>";
47 |     const unsigned char rows[][7]={
48 |       {14,17,17,31,17,17,17},{30,17,17,30,17,17,30},{15,16,16,16,16,16,15},
49 |       {30,17,17,17,17,17,30},{31,16,16,30,16,16,31},{31,16,16,30,16,16,16},
50 |       {15,16,16,23,17,17,15},{17,17,17,31,17,17,17},{31,4,4,4,4,4,31},
51 |       {7,2,2,2,18,18,12},{17,18,20,24,20,18,17},{16,16,16,16,16,16,31},
52 |       {17,27,21,21,17,17,17},{17,25,25,21,19,19,17},{14,17,17,17,17,17,14},
53 |       {30,17,17,30,16,16,16},{14,17,17,17,21,18,13},{30,17,17,30,20,18,17},
54 |       {15,16,16,14,1,1,30},{31,4,4,4,4,4,4},{17,17,17,17,17,17,14},
55 |       {17,17,17,17,17,10,4},{17,17,17,21,21,21,10},{17,17,10,4,10,17,17},
56 |       {17,17,10,4,4,4,4},{31,1,2,4,8,16,31},
57 |       {14,17,19,21,25,17,14},{4,12,4,4,4,4,14},{14,17,1,2,4,8,31},
58 |       {30,1,1,14,1,1,30},{2,6,10,18,31,2,2},{31,16,16,30,1,1,30},
59 |       {14,16,16,30,17,17,14},{31,1,2,4,8,8,8},{14,17,17,14,17,17,14},{14,17,17,15,1,1,14},
60 |       {0,4,4,0,4,4,0},{1,1,2,4,8,16,16},{0,0,0,0,0,6,6},{0,0,0,31,0,0,0},
61 |       {0,4,4,31,4,4,0},{14,8,8,8,8,8,14},{14,2,2,2,2,2,14},{4,4,4,4,4,0,4},
62 |       {2,4,8,16,8,4,2},{8,4,2,1,2,4,8}};
63 |     std::array<unsigned char,128*64*4> atlas{};
64 |     for(int i=0;glyphs[i];++i){unsigned ch=unsigned(glyphs[i]);int x=(ch%16)*8,y=(ch/16)*8;
65 |         for(int r=0;r<7;++r)for(int c=0;c<5;++c)if(rows[i][r]&(1<<(4-c))){auto
   | p=((y+r)*128+x+c)*4;atlas[p]=atlas[p+1]=atlas[p+2]=atlas[p+3]=255;}}
66 |     texture(font,128,64,atlas.data());glBindVertexArray(0);
67 |     const wchar_t* names[]={L"etiqueta_puntos",L"etiqueta_record",L"etiqueta_vidas",L"vida_nave",
68 |         L"etiqueta_control_mover",L"etiqueta_control_disparar",L"etiqueta_control_pausa"};
69 |     for(int i=0;i<7;++i){
70 |         auto
   | path=executableDirectory()/L"Recursos_Galaga3D"/L"hud"/(std::wstring(names[i])+L".png");
71 |
   | projectHud[i]=SOIL_load_OGL_texture(path.string().c_str(),SOIL_LOAD_RGBA,SOIL_CREATE_NEW_ID,0);
72 |         if(!projectHud[i])throw std::runtime_error("Falta un PNG del HUD del proyecto.");
73 |     }
74 | }
75 | void Renderer2D::release(){glDeleteTextures(7,projectHud);if(font)glDeleteTextures(1,&font);if(white
   | )glDeleteTextures(1,&white);if(vbo)glDeleteBuffers(1,&vbo);if(vao)glDeleteVertexArrays(1,&vao);if(pr
   | ogram)glDeleteProgram(program);font=white=vbo=vao=program=0;}
76 | void Renderer2D::begin(int width,int height){
77 |     glDisable(GL_SCISSOR_TEST);glDisable(GL_DEPTH_TEST);glDepthMask(GL_TRUE);glDisable(GL_CULL_FACE)
   | ;glPolygonMode(GL_FRONT_AND_BACK,GL_FILL);
78 |     float s=std::min(width/1280.f,height/720.f);int vw=int(1280*s),vh=int(720*s);glViewport((width-
   | vw)/2,(height-vh)/2,vw,vh);
79 |     glEnable(GL_BLEND);glBlendFunc(GL_SRC_ALPHA,GL_ONE_MINUS_SRC_ALPHA);
80 |     glUseProgram(program);glUniform1i(glGetUniformLocation(program,"image"),0);glActiveTexture(GL_TE
   | XTURE0);
```

```
    glBindVertexArray(vao);glBindBuffer(GL_ARRAY_BUFFER,vbo);
}
void Renderer2D::finish(){
    flush();
    // One upload for the complete HUD pass; retain draw ordering for overlays.
    glBufferData(GL_ARRAY_BUFFER,vertices.size()*sizeof(Vertex),vertices.data(),GL_STREAM_DRAW);
    for(const auto& batch:batches){glBindTexture(GL_TEXTURE_2D,batch.texture);glDrawArrays(GL_TRIANGLES,batch.first,batch.count);}
    vertices.clear();batches.clear();pendingStart=0;
    glBindVertexArray(0);glUseProgram(0);glDisable(GL_BLEND);
}
void Renderer2D::flush(unsigned int texture){
    int count=int(vertices.size())-pendingStart;if(count<=0)return;
    batches.push_back({texture?texture:white,pendingStart,count});pendingStart+=count;
}
void Renderer2D::quad(float x,float y,float w,float h,Color c,float u1,float v1,float u2,float v2){
    Vertex a{x,y,u1,v1,c.r,c.g,c.b,c.a},b{x+w,y,u2,v1,c.r,c.g,c.b,c.a},d{x,y+h,u1,v2,c.r,c.g,c.b,c.a},e{x+w,y+h,u2,v2,c.r,c.g,c.b,c.a};vertices.insert(vertices.end(),{a,b,e,a,e,d});
}
void Renderer2D::image(unsigned int texture,float x,float y,float w,float h){
    flush();quad(x,y,w,h,{1,1,1,1});flush(texture);
}
void Renderer2D::border(float x,float y,float w,float h,Color c){quad(x,y,w,2,c);quad(x,y+h-2,w,2,c);quad(x,y,2,h,c);quad(x+w-2,y,2,h,c);}
void Renderer2D::text(const std::string& value,float x,float y,int height,Color c,bool center){
    flush();float unit=height/7.f,advance=unit*6;if(center)x-=(value.size()*advance-unit)*.5f;
    for(unsigned char ch:value){if(ch>='a'&&ch<='z')ch-=32;if(ch>127)ch=' ';float u=float((ch%16)*8)/128,v=float((ch/16)*8)/64;quad(x,y,unit*5,float(height),c,u,v,u+5.f/128,v+7.f/64);x+=advance;}flush(font);
}
void Renderer2D::background(const Game3D& g,int width,int height){
    begin(width,height);glClearColor(0,0,0,1);glClear(GL_COLOR_BUFFER_BIT|GL_DEPTH_BUFFER_BIT);
    quad(0,0,1280,720,{0,0,0});
    if(g.phase==Phase::Menu){for(int i=0;i<3;++i){float x=215+i*300.f;bool selected=i==g.selectedShip;quad(x,257,250,242,selected?Color{.043f,.055f,.083f,.94f}:Color{.022f,.029f,.045f,.9f});border(x,257,250,242,selected?Color{1,.8f,.18f}:Color{.16f,.20f,.27f});if(selected)quad(x,257,250,4,{1,.8f,.18f});}}
    finish();
}
void Renderer2D::overlay(const Game3D& g,int width,int height,Vec2 targetScreen,float targetRadius){
    begin(width,height);
    const Color whiteC{.91f,.94f,1},yellow{1,.83f,.16f},red{1,.22f,.28f},blue{.3f,.7f,1},muted{.44f,.51f,.63f};
    const char* ships[]={"FLECHA","DELTA","DOBLE MOTOR"};
    if(g.phase==Phase::Menu){
        text("GALAGA",640,65,70,yellow,true);text("3 D",640,154,28,blue,true);
        text("ELIGE TU NAVE",640,216,18,whiteC,true);
        for(int i=0;i<3;++i){float x=340+i*300.f;text(std::to_string(i+1),x-100,277,14,i==g.selectedShip?yellow:muted);text(ships[i],x,461,17,i==g.selectedShip?yellow:whiteC,true);}
        text("1 / 2 / 3  SELECCIONAR",640,530,14,muted,true);
        text("ENTER  EMPEZAR",640,578,24,yellow,true);
        text("A/D MOVER   ESPACIO DISPARAR   P PAUSA",640,643,12,whiteC,true);
        text("UNA RONDA - 18 ENEMIGOS - 3 VIDAS",640,682,11,muted,true);
    }else{
        quad(0,0,1280,94,{0,0,0,.95f});
        image(projectHud[0],55,18,103.5f,36);text(std::to_string(g.score),55,57,22,whiteC);
        image(projectHud[1],300,18,105,36);text(std::to_string(std::max(g.score,g.bestScore)),300,57,22,whiteC);

text("ENEMIGOS",640,27,12,muted,true);text(std::to_string(g.remaining()),640,57,22,whiteC,true);

text("COMBO",915,27,12,muted,true);text("X"+std::to_string(g.multiplier),915,57,22,yellow,true);
        text("RONDA",1150,27,12,muted,true);text("01 / 01",1150,57,20,yellow,true);
        text(ships[g.selectedShip],55,109,10,muted);
        text(targetRadius>0?"OBJETIVO FIJADO":"ALINEA LA NAVE CON UN ENEMIGO",640,109,10,targetRadius>0?yellow:muted,true);
        if (targetRadius > 0) {
            for (float side : {-1.f,1.f}) {
                const float x = targetScreen.x + side * targetRadius;
                for (float end : {-1.f,1.f}) {
```

```
137 |                     const float y = targetScreen.y + end * targetRadius;
138 |                     quad(x-(side>0?8:0),y,8,2,yellow);
139 |                     quad(x,y-(end>0?8:0),2,8,yellow);
140 |                 }
141 |             }
142 |         }
143 |         quad(0,642,1280,78,{0,0,0,.95f});
144 |         image(projectHud[2],65,638,79.5f,36);
145 |         for(int i=0;i<g.lives;++i)image(projectHud[3],155+i*35.f,652,32,32);
146 |         image(projectHud[4],475,649,165,36);
147 |         image(projectHud[5],660,649,244.5f,36);
148 |         image(projectHud[6],930,649,115.5f,36);
149 |
    | if(g.invulnerable>0&&g.respawn==0&&g.phase==Phase::Playing)text("ESCUDO",330,675,10,blue,true);
150 |         if(g.phase==Phase::Paused||g.phase==Phase::Victory||g.phase==Phase::Defeat){
151 |             quad(0,100,1280,522,{0,0,.015f,.73f});quad(360,240,560,252,{.025f,.035f,.06f,.96f});bord
    | er(360,240,560,252,{.24f,.3f,.4f});
152 |             text(g.phase==Phase::Paused?"PAUSA":g.phase==Phase::Victory?"RONDA SUPERADA":"GAME
    | OVER",640,278,30,g.phase==Phase::Defeat?red:yellow,true);
153 |             text(g.phase==Phase::Paused?"TU NAVE TE ESPERA":"PUNTOS
    | "+std::to_string(g.score),640,345,17,whiteC,true);
154 |             text(g.phase==Phase::Paused?"P  CONTINUAR":"R  VOLVER A
    | JUGAR",640,412,16,whiteC,true);text("ESC  ELEGIR NAVE",640,453,12,muted,true);
155 |         }
156 |     }
157 |     finish();
158 | }
159 | }
```

Código galaga3d.vert

Entrega_Amin_VGI\entornsGrafics\EntornVGI\shaders\galaga3d.vert

```
  1 | #version 330 core
  2 | // Same locations as the course GL-3 examples and MeshVertex's interleaved VBO.
  3 | layout(location=0) in vec3 position;
  4 | layout(location=1) in vec3 normal;
  5 | layout(location=2) in vec2 uv;
  6 | layout(location=3) in vec4 color;
  7 | uniform mat4 model;
  8 | uniform mat4 view;
  9 | uniform mat4 projection;
 10 | uniform mat3 normalMatrix;
 11 | out vec3 worldPosition;
 12 | out vec3 worldNormal;
 13 | out vec4 vertexColor;
 14 | void main() {
 15 |     vec4 world = model * vec4(position, 1.0);
 16 |     worldPosition = world.xyz;
 17 |     worldNormal = normalMatrix * normal;
 18 |     vertexColor = color;
 19 |     gl_Position = projection * view * world;
 20 | }
```

Código galaga3d.frag

Entrega_Amin_VGI\entornsGrafics\EntornVGI\shaders\galaga3d.frag

```
  1 | #version 330 core
  2 | in vec3 worldPosition;
  3 | in vec3 worldNormal;
  4 | in vec4 vertexColor;
  5 | uniform vec3 eye;
  6 | uniform vec4 tint;
  7 |
  8 | // GL-6, pp. 18, 38 and 49-52. Lighting is evaluated in world coordinates.
  9 | struct Light {
```

```
    vec4 position; // w=0 for the directional light used by this scene.
    vec3 diffuse;
    vec3 specular;
};
struct Material {
    vec3 emission;
    vec3 ambient;
    vec3 diffuse;
    vec3 specular;
    float shininess;
};
uniform Light light;
uniform Material material;
uniform vec3 ambientLight;
out vec4 outColor;
void main() {
    vec3 n = normalize(worldNormal);
    vec3 lightDirection = normalize(light.position.xyz - worldPosition * light.position.w);
    vec3 viewDirection = normalize(eye - worldPosition);
    vec3 base = vertexColor.rgb * tint.rgb;

    vec3 emissive = material.emission * base;
    vec3 ambient = material.ambient * base * ambientLight;
    float cosine = max(dot(n, lightDirection), 0.0);
    vec3 diffuse = material.diffuse * base * light.diffuse * cosine;

    vec3 specular = vec3(0.0);
    if (cosine > 0.0) {
        vec3 reflected = reflect(-lightDirection, n);
        float highlight = pow(max(dot(reflected, viewDirection), 0.0), material.shininess);
        specular = material.specular * base * light.specular * highlight;
    }
    outColor = vec4(clamp(emissive + ambient + diffuse + specular, 0.0, 1.0),
                    vertexColor.a * tint.a);
}
```

Código DemoVerification.cpp

Entrega_Amin_VGI\entornsGrafics\EntornVGI\DemoVerification.cpp

```
#include "pch.h"
#include "EntornVGIDoc.h"
#include "EntornVGIView.h"
#include <algorithm>
#include <chrono>
#include <fstream>
#include <stdexcept>

// /verify-render exercises the real shader/VAO/VBO pipeline in an offscreen FBO.
// No desktop screenshots or synthetic OS input; MFC handlers are invoked directly.
bool CEntornVGIView::VerifyDemoRendering() {
    auto folder=galaga::Renderer2D::executableDirectory()/L"verification";
    std::filesystem::create_directories(folder);
    std::ofstream report(folder/L"render-report.txt");
    GLuint fbo=0,color=0,depth=0;
    auto cleanup=[&]{
        glBindFramebuffer(GL_FRAMEBUFFER,0);
        if(fbo)glDeleteFramebuffers(1,&fbo);if(color)glDeleteTextures(1,&color);if(depth)glDeleteRen
derbuffers(1,&depth);
        wglMakeCurrent(nullptr,nullptr);
    };
    try {
        if(!gameRendererReady)throw std::runtime_error("renderer not initialized");
        wglMakeCurrent(m_pDC->GetSafeHdc(),m_hRC);
        report<<"OpenGL: "<<glGetString(GL_VERSION)<<"\nRenderer: "<<glGetString(GL_RENDERER)<<"\n";
        if(!gameRenderer.verifyGpu(report))throw std::runtime_error("invalid GPU model buffers");
        const auto uploads=gameRenderer.uploadCount();
        glGenFramebuffers(1,&fbo);glGenTextures(1,&color);glGenRenderbuffers(1,&depth);glBindFramebu
ffer(GL_FRAMEBUFFER,fbo);
```

```
        auto target=[&](int width,int height){
            glBindTexture(GL_TEXTURE_2D,color);glTexImage2D(GL_TEXTURE_2D,0,GL_RGBA8,width,height,0,GL_RGBA,GL_UNSIGNED_BYTE,nullptr);
            glFramebufferTexture2D(GL_FRAMEBUFFER,GL_COLOR_ATTACHMENT0,GL_TEXTURE_2D,color,0);
            glBindRenderbuffer(GL_RENDERBUFFER,depth);glRenderbufferStorage(GL_RENDERBUFFER,GL_DEPTH_COMPONENT24,width,height);
            glFramebufferRenderbuffer(GL_FRAMEBUFFER,GL_DEPTH_ATTACHMENT,GL_RENDERBUFFER,depth);
            if(glCheckFramebufferStatus(GL_FRAMEBUFFER)!=GL_FRAMEBUFFER_COMPLETE)throw std::runtime_error("incomplete framebuffer");
        };
        target(64,64);
        if(!gameRenderer.verifyLighting(report))throw std::runtime_error("GL-6 lighting verification failed");
        auto capture=[&](const char* name,int width,int height,bool gallery=false,float angle=0.f) {
            target(width,height);
            if(!gallery && game.phase==galaga::Phase::Playing && !gameRenderer.verifyFraming(game,report))
                throw std::runtime_error("gameplay framing is unreadable or clipped");
            if(gallery)gameRenderer.modelGallery(width,height,angle);else gameRenderer.draw(game,width,height);
            glFinish();
            auto error=glGetError();if(error!=GL_NO_ERROR)throw std::runtime_error("OpenGL error "+std::to_string(error));
            std::vector<unsigned char> data(size_t(width)*height*4),flipped(data.size());
            glReadPixels(0,0,width,height,GL_RGBA,GL_UNSIGNED_BYTE,data.data());
            for(int y=0;y<height;++y)memcpy(flipped.data()+size_t(y)*width*4,data.data()+size_t(height-1-y)*width*4,size_t(width)*4);
            // Captures represent an opaque window, including blended HUD glyphs.
            for(size_t pixel=3;pixel<flipped.size();pixel+=4)flipped[pixel]=255;
            if(!gallery&&width==1280&&height==720&&game.phase==galaga::Phase::Playing){
                size_t visibleFooter=0;
                for(int y=640;y<710;++y)for(int x=25;x<1230;++x){const auto* p=flipped.data()+(size_t(y)*width+x)*4;if(std::max({p[0],p[1],p[2]})>150)++visibleFooter;}
                if(visibleFooter<1500)throw std::runtime_error("HUD controls or lives missing from framebuffer");
            }
            if(!SOIL_save_image((folder/(std::string(name)+".png")).string().c_str(),SOIL_SAVE_TYPE_PNG,width,height,4,flipped.data()))throw std::runtime_error("could not write PNG");
            std::vector<float> z(size_t(width)*height);glReadPixels(0,0,width,height,GL_DEPTH_COMPONENT,GL_FLOAT,z.data());
            size_t samples=std::count_if(z.begin(),z.end(),[](float v){return v<.99999f;});
            if(gameRenderer.drawCount()>0&&samples<30)throw std::runtime_error("solid meshes did not write depth");
            unsigned long long hash=14695981039346656037ull;for(auto c:data)hash=(hash^c)*1099511628211ull;
            report<<"PASS "<<name<<" "<<width<<"x"<<height<<"; "<<samples<<" depth pixels\n";
            return hash;
        };
        auto key=[&](UINT code){GameKeyDown(code,0);OnKeyUp(code,1,0);};
        game=galaga::Game3D{};capture("01-menu",1280,720);
        key('2');if(game.selectedShip!=1)throw std::runtime_error("ship selection failed");
        capture("02-delta-selected",1280,720);
        key('3');if(game.selectedShip!=2)throw std::runtime_error("third ship selection failed");
        key(VK_RETURN);if(game.phase!=galaga::Phase::Playing||game.selectedShip!=2)throw std::runtime_error("Enter did not preserve selected ship");
        for(int i=0;i<390;++i)game.update({});
        capture("03-combat",1280,720);capture("04-combat-4x3",1024,768);
        game.player.x=-7.68f;capture("04-left-limit",1280,720);
        game.player.x=7.68f;capture("04-right-limit",1280,720);game.player.x=0;
        bool hasNear=false,hasFar=false;
        for(const auto& e:game.enemies){hasNear|=e.z>2;hasFar|=e.z<-2;}
        if(!hasNear||!hasFar)throw std::runtime_error("missing persistent enemy depth levels");
        auto still=capture("05-depth-levels",1280,720);
        if(still!=capture("05-static-repeat",1280,720))throw std::runtime_error("static scene changed without simulation");
        auto savedGame=game;
        game.reset();game.player.x=game.enemies[2].pos.x+.5f;
        if(!game.aimTarget())throw std::runtime_error("missing aim target at lateral margin");
        capture("11-target-marker",1280,720);
        game.update({false,false,true});
        for(int step=0;step<20;++step)game.update({});
```

```
        capture("12-shot-in-flight",1280,720);
        for(int step=0;step<50;++step)game.update({});
        if(game.kills!=1)throw std::runtime_error("marked target was not hit");
        capture("13-confirmed-hit",1280,720);
        game=savedGame;
        key('P');float time=game.time;for(int i=0;i<180;++i)game.update({false,true,true});
        if(game.time!=time||game.phase!=galaga::Phase::Paused)throw std::runtime_error("pause integration failed");
        capture("06-pause",1280,720);key('P');
        if(game.phase!=galaga::Phase::Playing)throw std::runtime_error("resume integration failed");
        OnKillFocus(nullptr);
        if(game.phase!=galaga::Phase::Paused)throw std::runtime_error("focus loss did not pause");
        key(VK_ESCAPE);if(game.phase!=galaga::Phase::Menu)throw std::runtime_error("return to selection failed");
        key('1');key(VK_RETURN);
        game.phase=galaga::Phase::Victory;game.score=2100;for(auto& e:game.enemies)e.alive=false;
        capture("07-victory-fixture",1280,720);
        key('R');if(game.remaining()!=18||game.score!=0||game.lives!=3)throw std::runtime_error("restart integration failed");
        game.phase=galaga::Phase::Defeat;game.lives=0;game.score=750;
        capture("08-defeat-fixture",1280,720);
        auto front=capture("09-models-front",1280,720,true,.25f);
        auto side=capture("10-models-side",1280,720,true,1.05f);
        if(front==side)throw std::runtime_error("model matrix rotation did not alter rendering");
        report<<"PASS model matrix: rotated solid meshes produce different frames\n";
        game.reset();for(int i=0;i<390;++i)game.update({});
        target(1280,720);
        auto start=std::chrono::steady_clock::now();
        for(int i=0;i<60;++i){gameRenderer.draw(game,1280,720);glFinish();}
        auto ms=std::chrono::duration<double,std::milli>(std::chrono::steady_clock::now()-start).count()/60;
        report<<"Average render (60 frames, 1280x720): "<<ms<<" ms\n";
        if(gameRenderer.uploadCount()!=uploads||!gameRenderer.verifyGpu(report))throw std::runtime_error("mesh buffers changed across frames");
        if(glGetError()!=GL_NO_ERROR)throw std::runtime_error("OpenGL error after verification");
        report<<"PASS MFC handlers: 3 ships / start / pause / resume / focus loss / menu / restart\n";
        report<<"PASS static meshes reused through all captures and 60 frames; XYZ depth buffer active\n";
        cleanup();return true;
    }catch(const std::exception& e){report<<"FAIL "<<e.what()<<"\n";}
    cleanup();return false;
}
```

Código Game3DTests.cpp

Entrega_Amin_VGI\entornsGrafics\tests\Game3DTests.cpp

```
#include "../EntornVGI/Game3D.h"
#include <cmath>
#include <iostream>
#include <stdexcept>
using namespace galaga;
int checks=0;
void check(bool ok,const char* name){if(!ok)throw std::runtime_error(name);++checks;std::cout<<"PASS  "<<name<<'\n';}
void ticks(Game3D& g,int n,Input input={}){for(int i=0;i<n;++i)g.update(input);}
Input pilot(const Game3D& g){
 const Enemy* t=nullptr;
 for(const auto& e:g.enemies)if(e.alive&&(!t||std::abs(e.pos.x-g.player.x)<std::abs(t->pos.x-g.player.x)))t=&e;
 if(!t)return {};
 return {g.player.x>t->pos.x+.05f,g.player.x<t->pos.x-.05f,true};
}
int main(){try{
 Game3D g;g.reset();check(g.remaining()==18&&g.lives==3,"fresh round");
 bool levels[3]{};for(const auto& e:g.enemies)levels[int(e.z/2.5f)+1]=true;
 check(levels[0]&&levels[1]&&levels[2],"three persistent world depths");
 auto initial=g.enemies;ticks(g,120);bool stationary=true;
```

```
 for(size_t i=0;i<initial.size();++i)stationary&=g.enemies[i].pos.x==initial[i].pos.x&&g.enemies[i].pos.y==initial[i].pos.y&&g.enemies[i].z==initial[i].z;
 check(stationary&&g.sparks.empty(),"static formation without animation");
 g.reset();ticks(g,60,{false,true,false});check(std::abs(g.player.x-6)<.001f,"lateral speed");
 ticks(g,120,{false,true,false});check(g.player.x==7.68f,"right boundary");
 ticks(g,240,{true,false,false});check(g.player.x==-7.68f,"left boundary");
 check(g.player.y==-4,"only lateral player movement");
 g.reset();ticks(g,60,{true,true,false});check(g.player.x==0,"opposite keys cancel");
 g.reset();ticks(g,120,{false,false,true});check(g.shots==9,"fire cooldown");
 float time=g.time;auto bullets=g.bullets;g.pause();ticks(g,120,{false,true,true});check(g.time==time&&g.bullets.size()==bullets.size(),"pause freezes world");
 g.pause();g.update({});check(g.time>time,"resume");
 check(Game3D::hitTime3D({0,-2},{0,2},2,2,{0,0},{0,0},0,0,.46f)<0,"screen overlap at different depths is not a hit");
 check(Game3D::hitTime3D({0,0},{0,0},-2,2,{0,0},{0,0},0,0,.46f)>=0,"swept depth collision");
 for(float z:{-2.5f,0.f,2.5f}){
  g.reset();g.enemies.resize(1);auto& e=g.enemies[0];e.pos=e.previous=e.slot={0,4};e.z=e.previousZ=z;
  ticks(g,60,{false,false,true});check(g.phase==Phase::Victory&&g.kills==1,"aligned enemy reachable at each depth");
 }
 g.reset();ticks(g,146);check(!g.bullets.empty()&&!g.bullets[0].ally,"stationary AI fires");
 auto b=g.bullets[0];float flight=(g.player.y-b.pos.y)/b.forwardSpeed();
 check(std::abs(b.z+b.velocityZ*flight)<.001f&&std::abs(b.pos.x+b.velocityX*flight-g.player.x)<.001f,"enemy shot aims at player in XYZ");
 for (float depth : {-2.5f,0.f,2.5f}) {
  for (float offset : {-.64f,.50f,.64f}) {
   g.reset();g.enemies.resize(1);
   auto& enemy=g.enemies[0];enemy.pos=enemy.previous={offset,14};enemy.z=enemy.previousZ=depth;
   check(g.aimTarget()==&enemy,"target marker and fire share acquisition at margin");
   g.update({false,false,true});
   auto shot=g.bullets[0];
   float speed=std::sqrt(shot.velocityX*shot.velocityX+shot.forwardSpeed()*shot.forwardSpeed()+shot.velocityZ*shot.velocityZ);
   check(std::abs(speed-Game3D::PlayerShotSpeed)<.001f,"constant shot speed in XYZ");
   g.player.x=-5; ticks(g,100);
   check(g.kills==1,"off-centre marked shot hits every depth after player moves");
  }
 }
 g.reset();g.enemies.resize(1);g.enemies[0].pos=g.enemies[0].previous={1.1f,4};g.enemies[0].z=g.enemies[0].previousZ=0;
 check(g.aimTarget()==nullptr,"outside capture margin has no target marker");
 g.update({false,false,true});ticks(g,30);check(g.kills==0,"real near miss does not kill");
 g.reset();g.player.x=g.enemies[0].pos.x;
 check(g.aimTarget()==&g.enemies[0],"front row is targeted first");
 g.enemies[0].alive=false;check(g.aimTarget()==&g.enemies[6],"next depth becomes target after front dies");
 g.reset();g.enemies.resize(1);g.enemies[0].pos=g.enemies[0].previous={0,4};g.enemies[0].z=g.enemies[0].previousZ=0;
 g.bullets={{100,true,true,{0,3.8f},{0,3.8f}},{101,true,true,{0,3.8f},{0,3.8f}}};g.update({});
 check(g.kills==1&&g.score==100,"simultaneous hits award one kill");
 g.reset();g.invulnerable=1;g.bullets={{100,false,true,{2,-4.7f},{2,-4.7f}}};ticks(g,2);
 check(g.bullets.empty(),"missed enemy shot retired behind player before camera");
 g.reset();g.enemies.resize(1);g.enemies[0].pos=g.enemies[0].previous={0,4};g.enemies[0].z=g.enemies[0].previousZ=0;
 g.bullets={{100,true,true,{0,4},{0,4},0}};g.update({});check(g.kills==0,"expired shot cannot cause a fresh hit");
 g.reset();g.invulnerable=0;g.bullets={{100,false,true,g.player,g.player},{101,false,true,g.player,g.player}};g.update({});
 check(g.lives==2&&g.respawn>0,"simultaneous hits cost one life");
 g.reset();g.selectedShip=2;g.score=100;g.reset();check(g.selectedShip==2&&g.bestScore==100&&g.score==0,"restart preserves selection and record");
 for(int i=0;i<5;++i){ticks(g,100,{false,true,true});g.reset();}check(g.remaining()==18&&g.bullets.empty()&&g.time==0&&g.lives==3,"five clean restarts");
 g.returnToMenu();g.selectShip(1);check(g.phase==Phase::Menu&&g.selectedShip==1,"menu selection");
 g.reset();for(int i=0;i<60*120&&g.phase==Phase::Playing;++i)g.update(pilot(g));
 std::cout<<"PILOT lives="<<g.lives<<" kills="<<g.kills<<" seconds="<<g.time<<'\n';
 check(g.phase==Phase::Victory&&g.kills==18,"full round won using ordinary movement and fire");
 g.reset();ticks(g,60*120);check(g.phase==Phase::Defeat&&g.lives==0,"idle player loses naturally");
 std::cout<<checks<<" checks passed\n";return 0;
}catch(const std::exception& e){std::cerr<<"FAIL "<<e.what()<<'\n';return 1;}}
```

Código build-demo.cmd

build-demo.cmd

```
@echo off
setlocal
rem ExecutionPolicy applies only to this child process, never to Windows settings.
if "%~1"=="" (
    powershell.exe -NoProfile -ExecutionPolicy Bypass -File "%~dp0scripts\build-demo.ps1" -Configuration Release -Verify -Package
) else (
    powershell.exe -NoProfile -ExecutionPolicy Bypass -File "%~dp0scripts\build-demo.ps1" %*
)
exit /b %errorlevel%
```

Código build-demo.ps1

scripts\build-demo.ps1

```
param(
    [ValidateSet('Debug','Release')][string]$Configuration = 'Release',
    [switch]$Verify,
    [switch]$Package
)
$ErrorActionPreference = 'Stop'
$repo = Split-Path -Parent $PSScriptRoot
$source = Join-Path $repo 'Entrega_Amin_VGI\entornsGrafics'
if ($Package -and $Configuration -ne 'Release') { throw 'El paquete distribuible requiere Release.' }
$locator = Join-Path ${env:ProgramFiles(x86)} 'Microsoft Visual Studio\Installer\vswhere.exe'
if (!(Test-Path -LiteralPath $locator)) { throw 'Instala Visual Studio 2022 con C++ de escritorio y MFC v143.' }
$installation = & $locator -latest -version '[17.0,18.0)' -requires Microsoft.VisualStudio.Component.VC.Tools.x86.x64 -property installationPath
if (!$installation) { throw 'No se encuentra Visual Studio 2022 con el compilador C++.' }
$msbuild = Join-Path $installation 'MSBuild\Current\Bin\MSBuild.exe'
$solution = Join-Path $source 'Galaga3D.sln'
& (Join-Path $PSScriptRoot 'setup-mesa.ps1') -Deploy
& $msbuild $solution /t:Build "/p:Configuration=$Configuration" /p:Platform=x64 /verbosity:minimal /nologo
if ($LASTEXITCODE -ne 0) { throw 'La compilacion ha fallado.' }
$tests = Join-Path $source "bin\tests\$Configuration\Game3DTests.exe"
& $tests
if ($LASTEXITCODE -ne 0) { throw 'Han fallado las pruebas de logica.' }
$binary = Join-Path $source "bin\x64\$Configuration"
$executable = Join-Path $binary 'Galaga3D.exe'
if (!(Test-Path -LiteralPath $executable)) { throw "Falta el ejecutable esperado: $executable" }
if ($Verify) {
    $report = Join-Path $binary 'verification\render-report.txt'
    if (Test-Path -LiteralPath $report) { Remove-Item -LiteralPath $report -Force }
    $process = Start-Process -FilePath $executable -ArgumentList '/verify-render' -WorkingDirectory $binary -WindowStyle Hidden -PassThru
    if (!$process.WaitForExit(60000)) {
        Stop-Process -Id $process.Id -ErrorAction SilentlyContinue
        throw 'La verificacion grafica ha superado 60 segundos. Consulta startup.log y context.log.'
    }
    $process.Refresh()
    if ($process.ExitCode -ne 0 -or !(Test-Path -LiteralPath $report)) { throw 'Ha fallado la verificacion grafica. Consulta startup.log y context.log junto al ejecutable.' }
    Get-Content -LiteralPath $report
}
if ($Package) {
    $destination = Join-Path $repo 'dist\Galaga3D'
    New-Item -ItemType Directory -Path $destination -Force | Out-Null
    Copy-Item -LiteralPath $executable -Destination $destination -Force
    Copy-Item -LiteralPath (Join-Path $binary 'opengl32.dll') -Destination $destination -Force
```

```
    foreach ($folder in @('Recursos_Galaga3D','shaders','textures')) {
        Copy-Item -LiteralPath (Join-Path $binary $folder) -Destination $destination -Recurse -Force
    }
    # App-local redistribution of the VS runtime; no system installation needed.
    $redist = Get-ChildItem -LiteralPath (Join-Path $installation 'VC\Redist\MSVC') -Directory |
        Where-Object { $_.Name -match '^14\.' } | Sort-Object { [version]$_.Name } -Descending |
Select-Object -First 1
    if (!$redist) { throw 'No se encuentran los redistribuibles de Visual C++.' }
    foreach ($component in @('Microsoft.VC143.CRT','Microsoft.VC143.MFC')) {
        $folder = Join-Path $redist.FullName "x64\$component"
        if (!(Test-Path -LiteralPath $folder)) { throw "Falta el redistribuible $component." }
        Get-ChildItem -LiteralPath $folder -Filter '*.dll' | Copy-Item -Destination $destination
-Force
    }
    $licenses = Join-Path $destination 'licenses'
    New-Item -ItemType Directory -Path $licenses -Force | Out-Null
    Copy-Item -LiteralPath (Join-Path $source 'third_party\glew\LICENSE.txt') -Destination (Join-
Path $licenses 'GLEW.txt') -Force
    Copy-Item -LiteralPath (Join-Path $source 'third_party\soil2\LICENSE') -Destination (Join-Path
$licenses 'SOIL2.txt') -Force
    # GLM 0.9.9.7: https://raw.githubusercontent.com/g-truc/glm/0.9.9.7/copying.txt
    Copy-Item -LiteralPath (Join-Path $source 'third_party\GLM-COPYING.txt') -Destination (Join-Path
 $licenses 'GLM.txt') -Force
    $mesaLicenses = Join-Path $licenses 'Mesa'
    New-Item -ItemType Directory -Path $mesaLicenses -Force | Out-Null
    Get-ChildItem -LiteralPath (Join-Path $source 'third_party\mesa\licenses') | Copy-Item
-Destination $mesaLicenses -Recurse -Force
    Copy-Item -LiteralPath (Join-Path $source 'third_party\mesa\README.md') -Destination (Join-Path
$licenses 'MESA-ORIGEN.md') -Force
    Copy-Item -LiteralPath (Join-Path $repo 'DEMO_3D.md') -Destination (Join-Path $destination
'LEEME.md') -Force
    Write-Output "Demo lista: $destination\Galaga3D.exe"
}
```

Código setup-mesa.ps1

scripts\setup-mesa.ps1

```
param([switch]$Deploy)

$ErrorActionPreference = 'Stop'
$repo = Split-Path -Parent $PSScriptRoot
$source = Join-Path $repo 'Entrega_Amin_VGI\entornsGrafics'
$mesa = Join-Path $source 'third_party\mesa'
$cache = Join-Path $mesa 'cache'
$runtime = Join-Path $mesa 'bin'
$licenses = Join-Path $mesa 'licenses'
$version = '26.2.5'
$dllSha256 = '8ddc061ee752fff06ae682de0ac2e010926d76a93c1aecc4c206adee771856e1'

# TLS selection and downloaded files are local to this process/project.
[Net.ServicePointManager]::SecurityProtocol = [Net.SecurityProtocolType]::Tls12
foreach ($folder in @($cache, $runtime, $licenses)) {
    New-Item -ItemType Directory -Path $folder -Force | Out-Null
}

function Get-LocalSha256([string]$Path) {
    $stream = [IO.File]::OpenRead($Path)
    $hash = [Security.Cryptography.SHA256]::Create()
    try { return [BitConverter]::ToString($hash.ComputeHash($stream)).Replace('-',
'').ToLowerInvariant() }
    finally { $hash.Dispose(); $stream.Dispose() }
}

function Get-VerifiedFile([string]$Url, [string]$Path, [string]$Sha256) {
    if ((Test-Path -LiteralPath $Path) -and
        (Get-LocalSha256 $Path) -eq $Sha256) { return }
    $temporary = "$Path.download"
    Write-Output "Descargando $Url"
```

```
    try {
        Invoke-WebRequest -UseBasicParsing -Uri $Url -OutFile $temporary
        if ((Get-LocalSha256 $temporary) -ne $Sha256) {
            throw "El SHA256 no coincide: $Url"
        }
        Move-Item -LiteralPath $temporary -Destination $Path -Force
    } finally {
        if (Test-Path -LiteralPath $temporary) { Remove-Item -LiteralPath $temporary -Force }
    }
}

$dll = Join-Path $runtime 'opengl32.dll'
if (!(Test-Path -LiteralPath $dll) -or
    (Get-LocalSha256 $dll) -ne $dllSha256) {
    $archive = Join-Path $cache "mesa-llvmpipe-x64-$version.7z"
    Get-VerifiedFile "https://github.com/mmozeiko/build-mesa/releases/download/$version/mesa-
llvmpipe-x64-$version.7z" `
        $archive 'd1b713ca2a7fba47a50c058099f13c435ddb76a16a422c334e4d787f9755a31d'
    & tar.exe -xf $archive -C $runtime opengl32.dll
    if ($LASTEXITCODE -ne 0) { throw 'No se ha podido extraer Mesa con tar.exe.' }
    if ((Get-LocalSha256 $dll) -ne $dllSha256) {
        throw 'El SHA256 de opengl32.dll no coincide con la version fijada.'
    }
}

# Notices are checked in. Restore them from pinned upstream sources if absent.
if (!(Test-Path -LiteralPath (Join-Path $licenses 'license.rst')) -or
    !(Test-Path -LiteralPath (Join-Path $licenses 'MIT'))) {
    $archive = Join-Path $cache "mesa-$version.tar.xz"
    Get-VerifiedFile "https://archive.mesa3d.org/mesa-$version.tar.xz" `
        $archive '794b3d1729075dae7ebd47c7283b7ca0cdb5dec668f7f11cc0a5d82f6138a0a5'
    & tar.exe -xf $archive -C $licenses --strip-components=2 "mesa-$version/docs/license.rst"
"mesa-$version/licenses"
    if ($LASTEXITCODE -ne 0) { throw 'No se han podido extraer las licencias de Mesa.' }
}
Get-VerifiedFile 'https://raw.githubusercontent.com/llvm/llvm-project/llvmorg-23.1.3/LICENSE.TXT' `
    (Join-Path $licenses 'LLVM.txt')
'8d85c1057d742e597985c7d4e6320b015a9139385cff4cbae06ffc0ebe89afee'

if ($Deploy) {
    foreach ($configuration in @('Debug', 'Release')) {
        $destination = Join-Path $source "bin\x64\$configuration"
        New-Item -ItemType Directory -Path $destination -Force | Out-Null
        $target = Join-Path $destination 'opengl32.dll'
        if (!(Test-Path -LiteralPath $target) -or
            (Get-LocalSha256 $target) -ne $dllSha256) {
            Copy-Item -LiteralPath $dll -Destination $target -Force
        }
    }
}
Write-Output "Mesa $version LLVMpipe x64 listo (OpenGL local; sin cambiar controladores)."
```

Integración con la plantilla MFC

Los siguientes fragmentos corresponden a los puntos que conectan la demo con la ventana y el contexto de EntornVGI. Se incluyen con números de línea originales. El resto del entorno docente conserva sus controles y utilidades de modelado.

EntornVGI.cpp · BOOL CEntornVGIApp::InitInstance

```
BOOL CEntornVGIApp::InitInstance()
{
    SetCurrentDirectoryW(galaga::Renderer2D::executableDirectory().c_str());
    std::ofstream startup("startup.log");startup << "InitInstance" << std::endl;
    // Windows XP requiere InitCommonControlsEx() si un manifiesto de
    // aplicación especifica el uso de ComCtl32.dll versión 6 o posterior para habilitar
    // estilos visuales.  De lo contrario, se generará un error al crear ventanas.
    INITCOMMONCONTROLSEX InitCtrls;
```

```
74 |     InitCtrls.dwSize = sizeof(InitCtrls);
75 |     // Establecer para incluir todas las clases de control comunes que desee utilizar
76 |     // en la aplicación.
77 |     InitCtrls.dwICC = ICC_WIN95_CLASSES;
78 |     InitCommonControlsEx(&InitCtrls);
79 |
80 |     CWinAppEx::InitInstance();
81 |     startup << "MFC initialized" << std::endl;
82 |
83 |
84 |     // Inicializar bibliotecas OLE
85 |     if (!AfxOleInit())
86 |     {
87 |         AfxMessageBox(IDP_OLE_INIT_FAILED);
88 |         return FALSE;
89 |     }
90 |
91 |     AfxEnableControlContainer();
92 |
93 |     EnableTaskbarInteraction();
94 |
95 |     // Se necesita AfxInitRichEdit2() para usar el control RichEdit
96 |     // AfxInitRichEdit2();
97 |
98 |     // Inicialización estándar
99 |     // Si no utiliza estas funcionalidades y desea reducir el tamaño
100|     // del archivo ejecutable final, debe quitar
101|     // las rutinas de inicialización específicas que no necesite
102|     // Cambie la clave del Registro en la que se almacena la configuración
103|     // TODO: debe modificar esta cadena para que contenga información correcta
104|     // como el nombre de su compañía u organización
105|     SetRegistryKey(_T("VGI Galaga3D Demo"));
106|     LoadStdProfileSettings(4);  // Cargar opciones de archivo INI estándar (incluidas las de la
    | lista MRU)
107|
108|
109|     InitContextMenuManager();
110|
111|     InitKeyboardManager();
112|
113|     InitTooltipManager();
114|     CMFCToolTipInfo ttParams;
115|     ttParams.m_bVislManagerTheme = TRUE;
116|     theApp.GetTooltipManager()->SetTooltipParams(AFX_TOOLTIP_TYPE_ALL,
117|         RUNTIME_CLASS(CMFCToolTipCtrl), &ttParams);
118|
119|     // Registrar las plantillas de documento de la aplicación.  Las plantillas de documento
120|     //  sirven como conexión entre documentos, ventanas de marco y vistas
121|     CMultiDocTemplate* pDocTemplate;
122|     pDocTemplate = new CMultiDocTemplate(IDR_EntornVGITYPE,
123|         RUNTIME_CLASS(CEntornVGIDoc),
124|         RUNTIME_CLASS(CChildFrame), // Marco MDI secundario personalizado
125|         RUNTIME_CLASS(CEntornVGIView));
126|     if (!pDocTemplate)
127|         return FALSE;
128|     AddDocTemplate(pDocTemplate);
129|
130|     // Crear ventana de marco principal MDI
131|     startup << "Creating frame" << std::endl;
132|     CMainFrame* pMainFrame = new CMainFrame;
133|     if (!pMainFrame || !pMainFrame->LoadFrame(IDR_MAINFRAME))
134|     {
135|         startup << "LoadFrame failed, Win32 error " << GetLastError() << std::endl;
136|         delete pMainFrame;
137|         return FALSE;
138|     }
139|     m_pMainWnd = pMainFrame;
140|     startup << "Frame ready" << std::endl;
141|
142|
143|     // Analizar línea de comandos para comandos Shell estándar, DDE, Archivo Abrir
```

```
144 |     CCommandLineInfo cmdInfo;
145 |     ParseCommandLine(cmdInfo);
146 |     startup << "Shell command " << cmdInfo.m_nShellCommand << std::endl;
147 |
148 |
149 |
150 |     // Enviar comandos especificados en la línea de comandos.  Devolverá FALSE si
151 |     // la aplicación se inició con los modificadores /RegServer, /Register, /Unregserver o
    | /Unregister.
152 |     if (!ProcessShellCommand(cmdInfo))
153 |         return FALSE;
154 |     // La ventana principal se ha inicializado; mostrarla y actualizarla
155 |     startup << "Document created" << std::endl;
156 |     pMainFrame->SetDemoLayout(true);
157 |     if(CString(GetCommandLineW()).Find(L"/verify-render")>=0) {
158 |         auto child=pMainFrame->MDIGetActive();
159 |         auto view=child?DYNAMIC_DOWNCAST(CEntornVGIView,child->GetActiveView()):nullptr;
160 |         startup << "View " << (view?"ready":"missing") << std::endl;
161 |         bool passed=view && view->VerifyDemoRendering();
162 |         startup << "Verification " << passed << std::endl;
163 |         verificationExitCode=passed?0:1;
164 |         pMainFrame->DestroyWindow();
165 |         return FALSE;
166 |     }
167 |     pMainFrame->SetTitle(L"Galaga 3D");
168 |     if(auto child=pMainFrame->MDIGetActive())if(auto
    | doc=child->GetActiveDocument())doc->SetTitle(L"Galaga 3D");
169 |     pMainFrame->ShowWindow(m_nCmdShow);
170 |     pMainFrame->SetWindowPos(nullptr,0,0,1320,840,SWP_NOMOVE|SWP_NOZORDER);
171 |     if(auto child=pMainFrame->MDIGetActive()){child->MDIMaximize();if(auto
    | view=child->GetActiveView())view->SetFocus();}
172 |     pMainFrame->UpdateWindow();
173 |
174 |     return TRUE;
175 | }
```

EntornVGI.cpp · int CEntornVGIApp::ExitInstance

```
177 | int CEntornVGIApp::ExitInstance()
178 | {
179 |     //TODO: controlar recursos adicionales que se hayan podido agregar
180 |     AfxOleTerm(FALSE);
181 |
182 |     int result=CWinAppEx::ExitInstance();
183 |     return verificationExitCode>=0?verificationExitCode:result;
184 | }
```

EntornVGIView.cpp · int CEntornVGIView::OnCreate

```
663 | int CEntornVGIView::OnCreate(LPCREATESTRUCT lpCreateStruct)
664 | {
665 |     if(CView::OnCreate(lpCreateStruct)==-1)return -1;
666 |     std::ofstream trace("context.log");trace << "OnCreate" << std::endl;
667 |     m_pDC=new CClientDC(this);
668 |     PIXELFORMATDESCRIPTOR pfd{};
669 |     pfd.nSize=sizeof(pfd);pfd.nVersion=1;
670 |     pfd.dwFlags=PFD_DOUBLEBUFFER|PFD_SUPPORT_OPENGL|PFD_DRAW_TO_WINDOW;
671 |     pfd.iPixelType=PFD_TYPE_RGBA;pfd.cColorBits=32;pfd.cDepthBits=24;
672 |     int format=ChoosePixelFormat(m_pDC->GetSafeHdc(),&pfd);
673 |     if(!format||!SetPixelFormat(m_pDC->GetSafeHdc(),format,&pfd))return -1;
674 |     trace << "Pixel format " << format << std::endl;
675 |     HGLRC temporary=wglCreateContext(m_pDC->GetSafeHdc());
676 |     if(!temporary||!wglMakeCurrent(m_pDC->GetSafeHdc(),temporary)) {
677 |         if(temporary)wglDeleteContext(temporary);return -1;
678 |     }
679 |     trace << "Legacy context ready" << std::endl;
680 |     glewExperimental=GL_TRUE;
681 |     if(glewInit()!=GLEW_OK){wglMakeCurrent(nullptr,nullptr);wglDeleteContext(temporary);return -1;}
682 |     auto create=(PFNWGLCREATECONTEXTATTRIBSARBPROC)wglGetProcAddress("wglCreateContextAttribsARB");
683 |     // Preserve the teaching shaders on 4.6 hardware; the demo itself needs only 3.3.
684 |     if(create)for(int version : {46,33}) {
```

```
685 |         int
    | attributes[]={WGL_CONTEXT_MAJOR_VERSION_ARB,version/10,WGL_CONTEXT_MINOR_VERSION_ARB,version%10,
686 |             WGL_CONTEXT_PROFILE_MASK_ARB,WGL_CONTEXT_CORE_PROFILE_BIT_ARB,0};
687 |         m_hRC=create(m_pDC->GetSafeHdc(),nullptr,attributes);if(m_hRC)break;
688 |     }
689 |     if(m_hRC) {
690 |         wglMakeCurrent(nullptr,nullptr);wglDeleteContext(temporary);
691 |         if(!wglMakeCurrent(m_pDC->GetSafeHdc(),m_hRC))return -1;
692 |     } else {
693 |         // Initialization below checks 3.3 and reports a missing app-local runtime.
694 |         m_hRC=temporary;
695 |     }
696 |     trace << "OpenGL " << glGetString(GL_VERSION) << " / " << glGetString(GL_RENDERER) << std::endl;
697 |     m_hrc=m_hRC;
698 |     if(glewInit()!=GLEW_OK)return -1;
699 |     for(int i=0;i<16 && glGetError()!=GL_NO_ERROR;++i){}
700 |     if(GLEW_VERSION_3_3)InitAPI();
701 |     try {gameRenderer.initialize();gameRendererReady=true;trace << "Renderer ready" << std::endl;}
702 |     catch(const std::exception& e) {CString message(e.what());AfxMessageBox(message);return -1;}
703 |     game.bestScore=AfxGetApp()->GetProfileInt(L"Game",L"BestScore",0);
704 |     ClearGameInput();SetTimer(45003,16,nullptr);
705 |     wglMakeCurrent(nullptr,nullptr);
706 |     return 0;
707 | }
```

EntornVGIView.cpp · void CEntornVGIView::OnDestroy

```
842 | void CEntornVGIView::OnDestroy()
843 | {
844 |     KillTimer(45003);
845 |     if(CString(GetCommandLineW()).Find(L"/verify-render")<0)
846 |         AfxGetApp()->WriteProfileInt(L"Game",L"BestScore",std::max(game.bestScore,game.score));
847 |     if(m_hRC) {
848 |         wglMakeCurrent(m_pDC->GetSafeHdc(),m_hRC);
849 |         gameRenderer.release();gameRendererReady=false;
850 |         wglMakeCurrent(nullptr,nullptr);wglDeleteContext(m_hRC);m_hRC=nullptr;m_hrc=nullptr;
851 |     }
852 |     delete m_pDC;m_pDC=nullptr;
853 |     CView::OnDestroy();
854 | }
```

MainFrm.cpp · void CMainFrame::SetDemoLayout

```
454 | void CMainFrame::SetDemoLayout(bool demo)
455 | {
456 |     ShowPane(&m_wndFileView,!demo,FALSE,FALSE);
457 |     ShowPane(&m_wndClassView,!demo,FALSE,FALSE);
458 |     ShowPane(&m_wndOutput,!demo,FALSE,FALSE);
459 |     ShowPane(&m_wndProperties,!demo,FALSE,FALSE);
460 |     ShowPane(&m_wndToolBar,!demo,FALSE,FALSE);
461 |     ShowPane(&m_wndStatusBar,!demo,FALSE,FALSE);
462 |     RecalcLayout();
463 | }
```

void CEntornVGIView::OnPaint · fragmento de conexión

```
894 | void CEntornVGIView::OnPaint()
895 | {
896 |     CPaintDC dc(this); // device context for painting
897 |     if(demoMode) {
898 |         if(gameRendererReady) {
899 |             wglMakeCurrent(m_pDC->GetSafeHdc(),m_hRC);
900 |             gameRenderer.draw(game,w,h);
901 |             SwapBuffers(m_pDC->GetSafeHdc());
902 |             wglMakeCurrent(nullptr,nullptr);
903 |         }
904 |         return;
905 |     }
906 | // TODO: Agregue aquí su código de controlador de mensajes
907 | // No llama a CView::OnPaint() para mensajes de pintura
908 |     GLdouble vpv[3] = { 0.0, 0.0, 1.0 };
```

```
 909 |
 910 | // Entorn VGI: Activació el contexte OpenGL
 911 |     wglMakeCurrent(m_pDC->GetSafeHdc(), m_hRC);
 912 |
 913 | // EntornVGI: FPS
```

void CEntornVGIView::OnKeyDown · fragmento de conexión

```
1437 | void CEntornVGIView::OnKeyDown(UINT nChar, UINT nRepCnt, UINT nFlags)
1438 | {
1439 |     if(demoMode){GameKeyDown(nChar,nFlags);return;}
1440 | // TODO: Agregue aquí su código de controlador de mensajes o llame al valor predeterminado
1441 |     const double incr = 0.025f;
1442 |     double modul = 0;
1443 |     GLdouble vdir[3] = { 0, 0, 0 };
1444 |
1445 |     if (nChar == 'F') this->OnVistaFullScreen();    // Activació-Desactivació Full Screen
1446 |     else if (objecte == C_BEZIER || objecte == C_BSPLINE || objecte == C_LEMNISCATA || objecte ==
     | C_HERMITTE
1447 |         || objecte == C_CATMULL_ROM) Teclat_PasCorbes(nChar, nRepCnt);
```

void CEntornVGIView::OnKeyUp · fragmento de conexión

```
1479 | void CEntornVGIView::OnKeyUp(UINT nChar, UINT nRepCnt, UINT nFlags)
1480 | {
1481 |     if(demoMode){if(nChar<gameKeys.size())gameKeys[nChar]=false;return;}
1482 | // TODO: Agregue aquí su código de controlador de mensajes o llame al valor predeterminado
1483 |
1484 |     CView::OnKeyUp(nChar, nRepCnt, nFlags);
1485 | }
1486 |
1487 | // Teclat_ColorObjecte: Teclat pels canvis de color de l'objecte per teclat.
```

void CEntornVGIView::OnTimer · fragmento de conexión

```
3690 | void CEntornVGIView::OnTimer(UINT_PTR nIDEvent)
3691 | {
3692 |     if(nIDEvent==45003){GameTick();return;}
3693 | // TODO: Agregue aquí su código de controlador de mensajes o llame al valor predeterminado
3694 |     if (anima) {
3695 |         // Codi de tractament de l'animació quan transcorren els ms. del crono.
3696 |
3697 |         // Crida a OnPaint() per redibuixar l'escena
3698 |         InvalidateRect(NULL, false);
```

Configuración de los proyectos

Entrega_Amin_VGI\entornsGrafics\Galaga3D.sln

```
  1 | Microsoft Visual Studio Solution File, Format Version 12.00
  2 | # Visual Studio Version 17
  3 | VisualStudioVersion = 17.14.36603.0
  4 | MinimumVisualStudioVersion = 10.0.40219.1
  5 | Project("{8BC9CEB8-8B4A-11D0-8D11-00A0C91BC942}") = "EntornVGI", "EntornVGI\EntornVGI.vcxproj",
    | "{719E4E88-7616-3F72-F458-B584AB24CA32}"
  6 | EndProject
  7 | Project("{8BC9CEB8-8B4A-11D0-8D11-00A0C91BC942}") = "GLEW", "third_party\glew\glew.vcxproj",
    | "{017E497F-72AC-47B0-844C-5DA40B005001}"
  8 | EndProject
  9 | Project("{8BC9CEB8-8B4A-11D0-8D11-00A0C91BC942}") = "SOIL2", "third_party\soil2\soil2.vcxproj",
    | "{017E497F-72AC-47B0-844C-5DA40B005002}"
 10 | EndProject
 11 | Project("{8BC9CEB8-8B4A-11D0-8D11-00A0C91BC942}") = "Game3DTests", "tests\Game3DTests.vcxproj",
    | "{017E497F-72AC-47B0-844C-5DA40B005003}"
 12 | EndProject
 13 | Global
 14 |     GlobalSection(SolutionConfigurationPlatforms) = preSolution
 15 |         Debug|x64 = Debug|x64
 16 |         Release|x64 = Release|x64
```

```
    EndGlobalSection
    GlobalSection(ProjectConfigurationPlatforms) = postSolution
        {719E4E88-7616-3F72-F458-B584AB24CA32}.Debug|x64.ActiveCfg = Debug|x64
        {719E4E88-7616-3F72-F458-B584AB24CA32}.Debug|x64.Build.0 = Debug|x64
        {719E4E88-7616-3F72-F458-B584AB24CA32}.Release|x64.ActiveCfg = Release|x64
        {719E4E88-7616-3F72-F458-B584AB24CA32}.Release|x64.Build.0 = Release|x64
        {017E497F-72AC-47B0-844C-5DA40B005001}.Debug|x64.ActiveCfg = Debug|x64
        {017E497F-72AC-47B0-844C-5DA40B005001}.Debug|x64.Build.0 = Debug|x64
        {017E497F-72AC-47B0-844C-5DA40B005001}.Release|x64.ActiveCfg = Release|x64
        {017E497F-72AC-47B0-844C-5DA40B005001}.Release|x64.Build.0 = Release|x64
        {017E497F-72AC-47B0-844C-5DA40B005002}.Debug|x64.ActiveCfg = Debug|x64
        {017E497F-72AC-47B0-844C-5DA40B005002}.Debug|x64.Build.0 = Debug|x64
        {017E497F-72AC-47B0-844C-5DA40B005002}.Release|x64.ActiveCfg = Release|x64
        {017E497F-72AC-47B0-844C-5DA40B005002}.Release|x64.Build.0 = Release|x64
        {017E497F-72AC-47B0-844C-5DA40B005003}.Debug|x64.ActiveCfg = Debug|x64
        {017E497F-72AC-47B0-844C-5DA40B005003}.Debug|x64.Build.0 = Debug|x64
        {017E497F-72AC-47B0-844C-5DA40B005003}.Release|x64.ActiveCfg = Release|x64
        {017E497F-72AC-47B0-844C-5DA40B005003}.Release|x64.Build.0 = Release|x64
    EndGlobalSection
EndGlobal
```

Entrega_Amin_VGI\entornsGrafics\EntornVGI\EntornVGI.vcxproj

```
<?xml version="1.0" encoding="utf-8"?>
<Project DefaultTargets="Build" xmlns="http://schemas.microsoft.com/developer/msbuild/2003">
  <ItemGroup Label="ProjectConfigurations">
    <ProjectConfiguration Include="Debug|Win32">
      <Configuration>Debug</Configuration>
      <Platform>Win32</Platform>
    </ProjectConfiguration>
    <ProjectConfiguration Include="Release|Win32">
      <Configuration>Release</Configuration>
      <Platform>Win32</Platform>
    </ProjectConfiguration>
    <ProjectConfiguration Include="Debug|x64">
      <Configuration>Debug</Configuration>
      <Platform>x64</Platform>
    </ProjectConfiguration>
    <ProjectConfiguration Include="Release|x64">
      <Configuration>Release</Configuration>
      <Platform>x64</Platform>
    </ProjectConfiguration>
  </ItemGroup>
  <PropertyGroup Label="Globals">
    <VCProjectVersion>18.0</VCProjectVersion>
    <ProjectGuid>{719E4E88-7616-3F72-F458-B584AB24CA32}</ProjectGuid>
    <Keyword>MFCProj</Keyword>
    <RootNamespace>EntornVGI</RootNamespace>
    <WindowsTargetPlatformVersion>10.0</WindowsTargetPlatformVersion>
  </PropertyGroup>
  <Import Project="$(VCTargetsPath)\Microsoft.Cpp.Default.props" />
  <PropertyGroup Condition="'$(Configuration)|$(Platform)'=='Debug|Win32'" Label="Configuration">
    <ConfigurationType>Application</ConfigurationType>
    <UseDebugLibraries>true</UseDebugLibraries>
    <PlatformToolset>v143</PlatformToolset>
    <CharacterSet>Unicode</CharacterSet>
    <UseOfMfc>Dynamic</UseOfMfc>
  </PropertyGroup>
  <PropertyGroup Condition="'$(Configuration)|$(Platform)'=='Release|Win32'" Label="Configuration">
    <ConfigurationType>Application</ConfigurationType>
    <UseDebugLibraries>false</UseDebugLibraries>
    <PlatformToolset>v143</PlatformToolset>
    <WholeProgramOptimization>true</WholeProgramOptimization>
    <CharacterSet>Unicode</CharacterSet>
    <UseOfMfc>Dynamic</UseOfMfc>
  </PropertyGroup>
  <PropertyGroup Condition="'$(Configuration)|$(Platform)'=='Debug|x64'" Label="Configuration">
    <ConfigurationType>Application</ConfigurationType>
    <UseDebugLibraries>true</UseDebugLibraries>
    <PlatformToolset>v143</PlatformToolset>
    <CharacterSet>Unicode</CharacterSet>
```

```
49 |     <UseOfMfc>Dynamic</UseOfMfc>
50 |   </PropertyGroup>
51 |   <PropertyGroup Condition="'$(Configuration)|$(Platform)'=='Release|x64'" Label="Configuration">
52 |     <ConfigurationType>Application</ConfigurationType>
53 |     <UseDebugLibraries>false</UseDebugLibraries>
54 |     <PlatformToolset>v143</PlatformToolset>
55 |     <WholeProgramOptimization>true</WholeProgramOptimization>
56 |     <CharacterSet>Unicode</CharacterSet>
57 |     <UseOfMfc>Dynamic</UseOfMfc>
58 |   </PropertyGroup>
59 |   <Import Project="$(VCTargetsPath)\Microsoft.Cpp.props" />
60 |   <ImportGroup Label="ExtensionSettings">
61 |   </ImportGroup>
62 |   <ImportGroup Label="Shared">
63 |   </ImportGroup>
64 |   <ImportGroup Label="PropertySheets" Condition="'$(Configuration)|$(Platform)'=='Debug|Win32'">
65 |     <Import Project="$(UserRootDir)\Microsoft.Cpp.$(Platform).user.props"
   | Condition="exists('$(UserRootDir)\Microsoft.Cpp.$(Platform).user.props')"
   | Label="LocalAppDataPlatform" />
66 |   </ImportGroup>
67 |   <ImportGroup Label="PropertySheets" Condition="'$(Configuration)|$(Platform)'=='Release|Win32'">
68 |     <Import Project="$(UserRootDir)\Microsoft.Cpp.$(Platform).user.props"
   | Condition="exists('$(UserRootDir)\Microsoft.Cpp.$(Platform).user.props')"
   | Label="LocalAppDataPlatform" />
69 |   </ImportGroup>
70 |   <ImportGroup Label="PropertySheets" Condition="'$(Configuration)|$(Platform)'=='Debug|x64'">
71 |     <Import Project="$(UserRootDir)\Microsoft.Cpp.$(Platform).user.props"
   | Condition="exists('$(UserRootDir)\Microsoft.Cpp.$(Platform).user.props')"
   | Label="LocalAppDataPlatform" />
72 |   </ImportGroup>
73 |   <ImportGroup Label="PropertySheets" Condition="'$(Configuration)|$(Platform)'=='Release|x64'">
74 |     <Import Project="$(UserRootDir)\Microsoft.Cpp.$(Platform).user.props"
   | Condition="exists('$(UserRootDir)\Microsoft.Cpp.$(Platform).user.props')"
   | Label="LocalAppDataPlatform" />
75 |   </ImportGroup>
76 |   <PropertyGroup Label="UserMacros" />
77 |   <PropertyGroup><OutDir>$(ProjectDir)..\bin\$(Platform)\$(Configuration)\</OutDir><IntDir>$(Project
   | Dir)..\build\app\$(Platform)\$(Configuration)\</IntDir><TargetName>Galaga3D</TargetName></PropertyGr
   | oup>
78 |   <ItemDefinitionGroup><ClCompile><LanguageStandard>stdcpp17</LanguageStandard></ClCompile></ItemDef
   | initionGroup>
79 |   <ItemGroup>
80 |     <ProjectReference Include="..\third_party\glew\glew.vcxproj"><Project>{017E497F-72AC-47B0-844C-
   | 5DA40B005001}</Project></ProjectReference>
81 |     <ProjectReference Include="..\third_party\soil2\soil2.vcxproj"><Project>{017E497F-72AC-47B0-
   | 844C-5DA40B005002}</Project></ProjectReference>
82 |     <ClCompile Include="Game3D.cpp"><PrecompiledHeader>NotUsing</PrecompiledHeader></ClCompile>
83 |     <ClCompile Include="Renderer2D.cpp"/><ClCompile Include="Renderer3D.cpp"/><ClCompile
   | Include="Mesh3D.cpp"/><ClCompile Include="GameView3D.cpp"/><ClCompile
   | Include="DemoVerification.cpp"/>
84 |     <ClInclude Include="Game3D.h"/><ClInclude Include="Renderer2D.h"/><ClInclude
   | Include="Renderer3D.h"/><ClInclude Include="Mesh3D.h"/>
85 |   </ItemGroup>
86 |   <PropertyGroup Condition="'$(Configuration)|$(Platform)'=='Debug|x64'">
87 |     <LinkIncremental>true</LinkIncremental>
88 |   </PropertyGroup>
89 |   <PropertyGroup Condition="'$(Configuration)|$(Platform)'=='Debug|Win32'">
90 |     <LinkIncremental>true</LinkIncremental>
91 |   </PropertyGroup>
92 |   <PropertyGroup Condition="'$(Configuration)|$(Platform)'=='Release|Win32'">
93 |     <LinkIncremental>false</LinkIncremental>
94 |   </PropertyGroup>
95 |   <PropertyGroup Condition="'$(Configuration)|$(Platform)'=='Release|x64'">
96 |     <LinkIncremental>false</LinkIncremental>
97 |   </PropertyGroup>
98 |   <ItemDefinitionGroup Condition="'$(Configuration)|$(Platform)'=='Debug|x64'">
99 |     <ClCompile>
100 |       <PrecompiledHeader>Use</PrecompiledHeader>
101 |       <WarningLevel>Level3</WarningLevel>
102 |       <SDLCheck>true</SDLCheck>
103 |       <PreprocessorDefinitions>_WINDOWS;_DEBUG;_CRT_SECURE_NO_WARNINGS;%(PreprocessorDefinitions)</P
```

```
    | reprocessorDefinitions>
104 |       <PrecompiledHeaderFile>pch.h</PrecompiledHeaderFile>
105 |       <AdditionalIncludeDirectories>../include;../third_party/soil2/src/SOIL2;</AdditionalIncludeDir
    | ectories>
106 |       <DebugInformationFormat>ProgramDatabase</DebugInformationFormat>
107 |     </ClCompile>
108 |     <Link>
109 |       <SubSystem>Windows</SubSystem>
110 |       <AdditionalLibraryDirectories>../lib/x64;</AdditionalLibraryDirectories>
111 |       <AdditionalDependencies>opengl32.lib;%(AdditionalDependencies)</AdditionalDependencies>
112 |
113 |       <AdditionalOptions>/ignore:4099 %(AdditionalOptions)</AdditionalOptions>
114 |     </Link>
115 |     <Midl>
116 |       <MkTypLibCompatible>false</MkTypLibCompatible>
117 |       <ValidateAllParameters>true</ValidateAllParameters>
118 |       <PreprocessorDefinitions>_DEBUG;%(PreprocessorDefinitions)</PreprocessorDefinitions>
119 |     </Midl>
120 |     <ResourceCompile>
121 |       <Culture>0x0c0a</Culture>
122 |       <PreprocessorDefinitions>_DEBUG;%(PreprocessorDefinitions)</PreprocessorDefinitions>
123 |       <AdditionalIncludeDirectories>$(IntDir);%(AdditionalIncludeDirectories)</AdditionalIncludeDire
    | ctories>
124 |     </ResourceCompile>
125 |   </ItemDefinitionGroup>
126 |   <ItemDefinitionGroup Condition="'$(Configuration)|$(Platform)'=='Debug|Win32'">
127 |     <ClCompile>
128 |       <PrecompiledHeader>Use</PrecompiledHeader>
129 |       <WarningLevel>Level3</WarningLevel>
130 |       <SDLCheck>true</SDLCheck>
131 |
    | <PreprocessorDefinitions>WIN32;_WINDOWS;_DEBUG;%(PreprocessorDefinitions)</PreprocessorDefinitions>
132 |       <PrecompiledHeaderFile>pch.h</PrecompiledHeaderFile>
133 |     </ClCompile>
134 |     <Link>
135 |       <SubSystem>Windows</SubSystem>
136 |     </Link>
137 |     <Midl>
138 |       <MkTypLibCompatible>false</MkTypLibCompatible>
139 |       <ValidateAllParameters>true</ValidateAllParameters>
140 |       <PreprocessorDefinitions>_DEBUG;%(PreprocessorDefinitions)</PreprocessorDefinitions>
141 |     </Midl>
142 |     <ResourceCompile>
143 |       <Culture>0x0c0a</Culture>
144 |       <PreprocessorDefinitions>_DEBUG;%(PreprocessorDefinitions)</PreprocessorDefinitions>
145 |       <AdditionalIncludeDirectories>$(IntDir);%(AdditionalIncludeDirectories)</AdditionalIncludeDire
    | ctories>
146 |     </ResourceCompile>
147 |   </ItemDefinitionGroup>
148 |   <ItemDefinitionGroup Condition="'$(Configuration)|$(Platform)'=='Release|Win32'">
149 |     <ClCompile>
150 |       <PrecompiledHeader>Use</PrecompiledHeader>
151 |       <WarningLevel>Level3</WarningLevel>
152 |       <FunctionLevelLinking>true</FunctionLevelLinking>
153 |       <IntrinsicFunctions>true</IntrinsicFunctions>
154 |       <SDLCheck>true</SDLCheck>
155 |
    | <PreprocessorDefinitions>WIN32;_WINDOWS;NDEBUG;%(PreprocessorDefinitions)</PreprocessorDefinitions>
156 |       <PrecompiledHeaderFile>pch.h</PrecompiledHeaderFile>
157 |     </ClCompile>
158 |     <Link>
159 |       <SubSystem>Windows</SubSystem>
160 |       <EnableCOMDATFolding>true</EnableCOMDATFolding>
161 |       <OptimizeReferences>true</OptimizeReferences>
162 |     </Link>
163 |     <Midl>
164 |       <MkTypLibCompatible>false</MkTypLibCompatible>
165 |       <ValidateAllParameters>true</ValidateAllParameters>
166 |       <PreprocessorDefinitions>NDEBUG;%(PreprocessorDefinitions)</PreprocessorDefinitions>
167 |     </Midl>
168 |     <ResourceCompile>
```

```
169 |       <Culture>0x0c0a</Culture>
170 |       <PreprocessorDefinitions>NDEBUG;%(PreprocessorDefinitions)</PreprocessorDefinitions>
171 |       <AdditionalIncludeDirectories>$(IntDir);%(AdditionalIncludeDirectories)</AdditionalIncludeDire
    | ctories>
172 |     </ResourceCompile>
173 |   </ItemDefinitionGroup>
174 |   <ItemDefinitionGroup Condition="'$(Configuration)|$(Platform)'=='Release|x64'">
175 |     <ClCompile>
176 |       <PrecompiledHeader>Use</PrecompiledHeader>
177 |       <WarningLevel>Level3</WarningLevel>
178 |       <FunctionLevelLinking>true</FunctionLevelLinking>
179 |       <IntrinsicFunctions>true</IntrinsicFunctions>
180 |       <SDLCheck>true</SDLCheck>
181 |       <PreprocessorDefinitions>_WINDOWS;NDEBUG;_CRT_SECURE_NO_WARNINGS;%(PreprocessorDefinitions)</P
    | reprocessorDefinitions>
182 |       <PrecompiledHeaderFile>pch.h</PrecompiledHeaderFile>
183 |       <AdditionalIncludeDirectories>../include;../third_party/soil2/src/SOIL2;</AdditionalIncludeDir
    | ectories>
184 |     </ClCompile>
185 |     <Link>
186 |       <SubSystem>Windows</SubSystem>
187 |       <EnableCOMDATFolding>true</EnableCOMDATFolding>
188 |       <OptimizeReferences>true</OptimizeReferences>
189 |       <AdditionalLibraryDirectories>../lib/x64;</AdditionalLibraryDirectories>
190 |
191 |       <AdditionalDependencies>opengl32.lib;%(AdditionalDependencies)</AdditionalDependencies>
192 |       <AdditionalOptions>/ignore:4099 %(AdditionalOptions)</AdditionalOptions>
193 |       <LinkTimeCodeGeneration>UseLinkTimeCodeGeneration</LinkTimeCodeGeneration>
194 |     </Link>
195 |     <Midl>
196 |       <MkTypLibCompatible>false</MkTypLibCompatible>
197 |       <ValidateAllParameters>true</ValidateAllParameters>
198 |       <PreprocessorDefinitions>NDEBUG;%(PreprocessorDefinitions)</PreprocessorDefinitions>
199 |     </Midl>
200 |     <ResourceCompile>
201 |       <Culture>0x0c0a</Culture>
202 |       <PreprocessorDefinitions>NDEBUG;%(PreprocessorDefinitions)</PreprocessorDefinitions>
203 |       <AdditionalIncludeDirectories>$(IntDir);%(AdditionalIncludeDirectories)</AdditionalIncludeDire
    | ctories>
204 |     </ResourceCompile>
205 |   </ItemDefinitionGroup>
206 |   <ItemGroup>
207 |     <ClInclude Include="ChildFrm.h" />
208 |     <ClInclude Include="ClassView.h" />
209 |     <ClInclude Include="constants.h" />
210 |     <ClInclude Include="EntornVGI.h" />
211 |     <ClInclude Include="EntornVGIDoc.h" />
212 |     <ClInclude Include="EntornVGIView.h" />
213 |     <ClInclude Include="escena.h" />
214 |     <ClInclude Include="FileView.h" />
215 |     <ClInclude Include="framework.h" />
216 |     <ClInclude Include="glut_geometry.h" />
217 |     <ClInclude Include="MainFrm.h" />
218 |     <ClInclude Include="objLoader.h" />
219 |     <ClInclude Include="OutputWnd.h" />
220 |     <ClInclude Include="pch.h" />
221 |     <ClInclude Include="PropertiesWnd.h" />
222 |     <ClInclude Include="Resource.h" />
223 |     <ClInclude Include="shader.h" />
224 |     <ClInclude Include="targetver.h" />
225 |     <ClInclude Include="ViewTree.h" />
226 |     <ClInclude Include="visualitzacio.h" />
227 |   </ItemGroup>
228 |   <ItemGroup>
229 |     <ClCompile Include="ChildFrm.cpp" />
230 |     <ClCompile Include="ClassView.cpp" />
231 |     <ClCompile Include="EntornVGI.cpp" />
232 |     <ClCompile Include="EntornVGIDoc.cpp" />
233 |     <ClCompile Include="EntornVGIView.cpp" />
234 |     <ClCompile Include="escena.cpp" />
235 |     <ClCompile Include="FileView.cpp" />
```

```
236 |     <ClCompile Include="glut_geometry.cpp" />
237 |     <ClCompile Include="MainFrm.cpp" />
238 |     <ClCompile Include="material.cpp" />
239 |     <ClCompile Include="objLoader.cpp" />
240 |     <ClCompile Include="OutputWnd.cpp" />
241 |     <ClCompile Include="pch.cpp">
242 |       <PrecompiledHeader
    | Condition="'$(Configuration)|$(Platform)'=='Debug|x64'">Create</PrecompiledHeader>
243 |       <PrecompiledHeader
    | Condition="'$(Configuration)|$(Platform)'=='Debug|Win32'">Create</PrecompiledHeader>
244 |       <PrecompiledHeader
    | Condition="'$(Configuration)|$(Platform)'=='Release|Win32'">Create</PrecompiledHeader>
245 |       <PrecompiledHeader
    | Condition="'$(Configuration)|$(Platform)'=='Release|x64'">Create</PrecompiledHeader>
246 |     </ClCompile>
247 |     <ClCompile Include="PropertiesWnd.cpp" />
248 |     <ClCompile Include="shader.cpp" />
249 |     <ClCompile Include="ViewTree.cpp" />
250 |     <ClCompile Include="visualitzacio.cpp" />
251 |   </ItemGroup>
252 |   <ItemGroup>
253 |     <ResourceCompile Include="EntornVGI.rc" />
254 |   </ItemGroup>
255 |   <ItemGroup>
256 |     <Image Include="res\classview.bmp" />
257 |     <Image Include="res\classview_hc.bmp" />
258 |     <Image Include="res\class_view.ico" />
259 |     <Image Include="res\class_view_hc.ico" />
260 |     <Image Include="res\EntornVGI.ico" />
261 |     <Image Include="res\EntornVGIDoc.ico" />
262 |     <Image Include="res\explorer.bmp" />
263 |     <Image Include="res\explorer_hc.bmp" />
264 |     <Image Include="res\fileview.bmp" />
265 |     <Image Include="res\fileview_hc.bmp" />
266 |     <Image Include="res\file_view.ico" />
267 |     <Image Include="res\file_view_hc.ico" />
268 |     <Image Include="res\menuimages.bmp" />
269 |     <Image Include="res\menuimages_hc.bmp" />
270 |     <Image Include="res\output_wnd.ico" />
271 |     <Image Include="res\output_wnd_hc.ico" />
272 |     <Image Include="res\properties.bmp" />
273 |     <Image Include="res\properties_hc.bmp" />
274 |     <Image Include="res\properties_wnd.ico" />
275 |     <Image Include="res\properties_wnd_hc.ico" />
276 |     <Image Include="res\sort.bmp" />
277 |     <Image Include="res\sort_hc.bmp" />
278 |     <Image Include="res\Toolbar.bmp" />
279 |     <Image Include="res\Toolbar256.bmp" />
280 |     <Image Include="res\userimages.bmp" />
281 |   </ItemGroup>
282 |   <ItemGroup>
283 |     <None Include="res\EntornVGI.rc2" />
284 |   </ItemGroup>
285 |   <ItemGroup>
286 |     <DemoAssets Include="$(ProjectDir)..\..\..\Recursos_Galaga3D\hud\*.png"><DemoFolder>hud</DemoFol
    | der></DemoAssets>
287 |     <DemoAssets Include="$(ProjectDir)..\..\..\Recursos_Galaga3D\fonts\*"><DemoFolder>fonts</DemoFol
    | der></DemoAssets>
288 |     <MesaRuntime Include="$(ProjectDir)..\third_party\mesa\bin\opengl32.dll"
    | Condition="Exists('$(ProjectDir)..\third_party\mesa\bin\opengl32.dll')"/>
289 |     <TeachingShaders Include="$(ProjectDir)shaders\*"/>
290 |     <TeachingTextures Include="$(ProjectDir)textures\**\*"/>
291 |   </ItemGroup>
292 |   <Target Name="CopyDemoResources" AfterTargets="Build">
293 |     <Copy SourceFiles="@(MesaRuntime)" DestinationFolder="$(OutDir)" SkipUnchangedFiles="true"/>
294 |     <Copy SourceFiles="@(DemoAssets)"
    | DestinationFiles="@(DemoAssets->'$(OutDir)Recursos_Galaga3D\%(DemoFolder)\%(Filename)%(Extension)')"
    |  SkipUnchangedFiles="true"/>
295 |     <Copy SourceFiles="@(TeachingShaders)" DestinationFolder="$(OutDir)shaders"
    | SkipUnchangedFiles="true"/>
296 |     <Copy SourceFiles="@(TeachingTextures)"
```

```
    | DestinationFiles="@(TeachingTextures->'$(OutDir)textures\%(RecursiveDir)%(Filename)%(Extension)')"
    | SkipUnchangedFiles="true"/>
297 |   </Target>
298 |   <Import Project="$(VCTargetsPath)\Microsoft.Cpp.targets" />
299 |   <ImportGroup Label="ExtensionTargets">
300 |   </ImportGroup>
301 | </Project>
```

Entrega_Amin_VGI\entornsGrafics\tests\Game3DTests.vcxproj

```
  1 | <?xml version="1.0" encoding="utf-8"?>
  2 | <Project DefaultTargets="Build" xmlns="http://schemas.microsoft.com/developer/msbuild/2003">
  3 |   <ItemGroup Label="ProjectConfigurations"><ProjectConfiguration Include="Debug|x64"><Configuration>
    | Debug</Configuration><Platform>x64</Platform></ProjectConfiguration><ProjectConfiguration Include="R
    | elease|x64"><Configuration>Release</Configuration><Platform>x64</Platform></ProjectConfiguration></I
    | temGroup>
  4 |   <PropertyGroup Label="Globals"><ProjectGuid>{017E497F-72AC-47B0-844C-
    | 5DA40B005003}</ProjectGuid><WindowsTargetPlatformVersion>10.0</WindowsTargetPlatformVersion></Proper
    | tyGroup>
  5 |   <Import Project="$(VCTargetsPath)\Microsoft.Cpp.Default.props"/>
  6 |   <PropertyGroup Label="Configuration"><ConfigurationType>Application</ConfigurationType><PlatformTo
    | olset>v143</PlatformToolset></PropertyGroup>
  7 |   <Import Project="$(VCTargetsPath)\Microsoft.Cpp.props"/>
  8 |   <PropertyGroup><OutDir>$(ProjectDir)..\bin\tests\$(Configuration)\</OutDir><IntDir>$(ProjectDir)..
    | \build\tests\$(ProjectName)\$(Configuration)\</IntDir></PropertyGroup>
  9 |   <ItemDefinitionGroup><ClCompile><LanguageStandard>stdcpp17</LanguageStandard><WarningLevel>Level4<
    | /WarningLevel></ClCompile><Link><SubSystem>Console</SubSystem></Link></ItemDefinitionGroup>
 10 |   <ItemGroup><ClCompile Include="Game3DTests.cpp"/><ClCompile
    | Include="..\EntornVGI\Game3D.cpp"/></ItemGroup>
 11 |   <Import Project="$(VCTargetsPath)\Microsoft.Cpp.targets"/>
 12 | </Project>
```

Identificación de los archivos documentados

SHA256 es una huella del contenido. Un cambio de cualquier byte produce normalmente una huella distinta. Los nombres siguientes corresponden al repositorio y permiten localizar los listados o su integración.

Entrega_Amin_VGI\entornsGrafics\EntornVGI\Game3D.h

```
85de99792df341fde4af07fa204b43d1b55c3abb6d4a095fe8aa78ef7600254f
```

Entrega_Amin_VGI\entornsGrafics\EntornVGI\Game3D.cpp

```
7c08c2babf8a631daadb22009c02ca6f19a00f38e7c2ccfeed4a002340779aa4
```

Entrega_Amin_VGI\entornsGrafics\EntornVGI\GameView3D.cpp

```
b943df9a93d2edf64b4e7a1b832886521b817cdc3ff630d41be94d20567137a6
```

Entrega_Amin_VGI\entornsGrafics\EntornVGI\Mesh3D.h

```
e85eafd1c56388850ef38e2d757d494a16713b901dca690930b1194e52d66ba3
```

Entrega_Amin_VGI\entornsGrafics\EntornVGI\Mesh3D.cpp

```
93417246583ad55642cec6cd5bb8fcf207f7de6a73b1fad9fe731d9e8d11cb57
```

Entrega_Amin_VGI\entornsGrafics\EntornVGI\Renderer3D.h

```
e5ca3f62f79ff61e24724a219e0d7f65d571932e5b8c7efaf4a8afa91c8318fe
```

Entrega_Amin_VGI\entornsGrafics\EntornVGI\Renderer3D.cpp

```
4d5d20efd157fd18e882199263731f7e18f1f0d041daaa92f7d99276733c799d
```

Entrega_Amin_VGI\entornsGrafics\EntornVGI\Renderer2D.h

```
c331761e6a36cadf414c02f0e00c89d7f59935348abe03b8846ec6fcf2a059f1
```

Entrega_Amin_VGI\entornsGrafics\EntornVGI\Renderer2D.cpp

```
d43806b044ae20bdfc5eb74bd75bdae7ce3d8b8a8f2d92adf55c9ccfab3ff655
```

Entrega_Amin_VGI\entornsGrafics\EntornVGI\shaders\galaga3d.vert

```
6b9b65fb4be093a72417b2e5944d35eed78523c691af1ec6c270e40d61297830
```

Entrega_Amin_VGI\entornsGrafics\EntornVGI\shaders\galaga3d.frag

```
cfa36427d62bf84cbbce6c50e7d5edbc10b6acfac3bed3a1ee883ed0a7c3de28
```

Entrega_Amin_VGI\entornsGrafics\EntornVGI\DemoVerification.cpp

```
3420a5273904100271ad2380eaf7684f17aeafbc57e77eb340405e055f4ae35a
```

Entrega_Amin_VGI\entornsGrafics\tests\Game3DTests.cpp

```
a6095e270a450024abf1f209dd71550eb9ee7d86236f3672dd2b1cc7fae772a2
```

build-demo.cmd

```
8078b42bf80a1e9ab82e07fb8e6153ff4b3d5f8d0ce12a996442a0f4a4561921
```

scripts\build-demo.ps1

```
d8a8f23ff1f89e7e0cd6aa94d05e9d951a19b503a6c0477e1a317bb2dd08cb6f
```

scripts\setup-mesa.ps1

```
7b2871d3790eed891fb702704e2ac399ca1233b2ada307d9f08dc71b7164182d
```

Entrega_Amin_VGI\entornsGrafics\EntornVGI\EntornVGI.cpp

```
99c7c06497d90afaba6e1aa4f18110b299ee624a7c7e896912d8262c76f414ad
```

Entrega_Amin_VGI\entornsGrafics\EntornVGI\EntornVGIView.cpp

```
76d6f2393d03db6ed55e276d9a8085902dfa54458b258cf36301e0962e32cb6d
```

Entrega_Amin_VGI\entornsGrafics\EntornVGI\MainFrm.cpp

```
6413dc44c4360125223e30a8d50171220b2ec82243ac99057c6c5052d1519ab2
```

Entrega_Amin_VGI\entornsGrafics\Galaga3D.sln

```
00d2c7e1d8db82dd96b01cc4603558a6c10f4fa186e7fd5b32218c513b7fa68a
```

Entrega_Amin_VGI\entornsGrafics\EntornVGI\EntornVGI.vcxproj

```
453cdc1714b535d0ab212bd63e9f6949b6d52c170d824b8b5251c3a0b66128b6
```

Entrega_Amin_VGI\entornsGrafics\tests\Game3DTests.vcxproj

```
ddae87fa6cc222381b1c915e17da83db635dfe5c85d1781faf22430797defde7
```